Lavavel
V9 框架中文文档

编　写　人：	翻译
日　　期：	2026-03-30
版　　本：	V1.0.3

修改历史

日期	版本	作者	修改内容
2026-03-30	1.0.3.01	翻译	初始版本

目录

第一章 序言 41

1.1. 发行说明 41

1.1.1. 版本化方案 41

1.1.2. 支持策略 41

1.1.3. Laravel 9 42

1.1.3.1. PHP 8.0 43

1.1.3.2. Symfony Mailer 43

1.1.3.3. Flysystem 3.x 43

1.1.3.4. Eloquent 访问器/修改器改进 43

1.1.3.5. Enum Eloquent 属性转换 44

1.1.3.6. 使用枚举的隐式路由绑定 44

1.1.3.7. 路由绑定的强制作用域名 44

1.1.3.8. 控制器路由组 44

1.1.3.9. 全文索引/Where 子名 44

1.1.3.10. Laravel Scout 数据库引擎 45

1.2. 升级说明 45

1.2.1. 升级指南 45

1.2.1.1. 从 8.x 升级到 9.0 45

1.2.2. 高影响变化 45

1.2.2.1. 更新依赖 45

1.2.2.2. Flysystem 3.x 45

1.2.2.3. Symfony Mailer 45

1.2.3. 中影响变化 46

1.2.3.1. Belongs To Many 46
1.3. 贡献导引 46
1.3.1. Bug 报告 46
1.3.2. 帮助支持 47
1.3.3. 核心发展讨论 48
1.3.4. 哪个分支 48
1.3.5. 编译文件 48
1.3.6. 安全漏洞 49
1.3.7. 编码风格 49
1.3.7.1. PHP 文档 49
1.3.7.2. StyleCI 50
1.3.8. 行为准则 50
第二章 快速入门 50
2.1. 安装 50
2.1.1. 认识 Laravel 50
2.1.1.1. 为什么选择 Laravel? 51
2.1.2. 你的第一个 Laravel 项目 51
2.1.2.1. macOS 入门 51
2.1.2.2. Windows 入门 51
2.1.2.3. 大 Linux 使用 Laravel Sail 51
2.1.3. 初始配置 51
2.1.3.1. 基于环境的配置 52
2.1.3.2. 目录配置 52
2.2. 配置 52

2.2.1. 介绍 52
2.2.2. 环境配置 52
2.2.2.1. 环境变量类型 53
2.2.2.2. 获取环境配置 53
2.2.2.3. 获取当前环境配置 54
2.2.3. 访问配置值 54
2.2.4. 配置缓存 54
2.2.5. 调试模式 54
2.2.6. 维护模式 55
2.3. 目录结构 55
2.3.1. 简介 55
2.3.2. 根目录 55
2.3.2.1. App 目录 55
2.3.2.2. 启动目录 55
2.3.2.3. 配置目录 56
2.3.2.4. 数据库目录 56
2.3.2.5. 语言目录 56
2.3.2.6. Public 公开目录 56
2.3.2.7. 资源目录 56
2.3.2.8. 路由目录 56
2.3.2.9. 文件存储目录 56
2.3.2.10. 测试目录 56
2.3.2.11. Vendor 目录 56
2.3.3. App 目录 56

2.3.3.1. Broadcasting 目录 57
2.3.3.2. Console 目录 57
2.3.3.3. Events 目录 57
2.3.3.4. Exceptions 目录 57
2.3.3.5. Http 目录 57
2.3.3.6. Job 目录 57
2.3.3.7. Listeners 目录 57
2.3.3.8. Mail 目录 58
2.3.3.9. Notifications 目录 58
2.3.3.10. Policies 目录 58
2.3.3.11. Providers 目录 58
2.3.3.12. Rules 目录 58
2.4. 部署 58
2.4.1. 简介 58
2.4.2. 服务器要求 58
2.4.3. 服务器配置 59
2.4.3.1. Nginx 59
2.4.4. 优化 59
2.4.4.1. 优化自动加载 59
2.4.4.2. 优化配置加载 59
2.4.4.3. 优化路由加载器 59
2.4.4.4. 优化视图加载器 60
第三章 核心架构 60
3.1. 生命周期 60

3.1.1. 简介 60
3.1.2. 生命周期概览 60
3.1.2.1. 第一步 60
3.1.2.2. HTTP/Console 内核 60
3.1.2.3. 服务提供者 61
3.1.2.4. 路由 61
3.1.2.5. 最后 62
3.1.3. 聚焦服务提供者 62
3.2. 服务容器 63
3.2.1. 简介 63
3.2.1.1. 零配置解决方案 63
3.2.1.2. 何时使用容器 64
3.2.2. 绑定 64
3.2.2.1. 基础绑定 64
3.2.2.2. 绑定接口至实现 66
3.2.2.3. 上下文绑定 66
3.2.2.4. 标签 67
3.2.2.5. 继承绑定 68
3.2.3. 解析 68
3.2.3.1. Make 方法 68
3.2.3.2. 自动注入 69
3.2.4. 方法调用和注入 69
3.2.5. 容器事件 70
3.2.6. PSR-11 71

3.3. 服务提供者 71
3.3.1. 简介 71
3.3.2. 编写服务提供者 72
3.3.2.1. Register 方法 72
3.3.2.2. boot 方法 73
3.3.3. 注册服务提供者 75
3.3.4. 延迟加载服务提供者 75
3.4. 门面(Facades) 76
3.4.1. 简介 76
3.4.1.1. 辅助函数 76
3.4.2. 何时使用门面 76
3.4.2.1. 门面 vs.依赖注入 77
3.4.2.2. 门面 vs.辅助函数 77
3.4.3. 门面工作原理 78
3.4.4. 实时门面 78
3.4.5. 门面类参考 78
第四章 基本概念 79
4.1. 路由 79
4.1.1. 路由入门 79
4.1.1.1. 路由重定向 81
4.1.1.2. 路由视图 81
4.1.2. 路由参数 81
4.1.2.1. 必填参数 81
4.1.2.2. 可选参数 82

4.1.2.3. 正则约束 83
4.1.2.4. 命名路由 84
4.1.3. 路由分组 85
4.1.3.1. 路由中间件 85
4.1.3.2. 控制器 86
4.1.3.3. 子域路由 86
4.1.3.4. 路由前缀 87
4.1.3.5. 路由名称前缀 87
4.1.4. 路由模型绑定 88
4.1.4.1. 隐式绑定 88
4.1.4.2. 隐式枚举绑定 89
4.1.4.3. 显式绑定 90
4.1.5. fallback 路由 91
4.1.6. 速率限制 92
4.1.6.1. 定义速率限制器 92
4.1.6.2. 将速率限制器附加到路由 93
4.1.7. 伪造表单方法 93
4.1.8. 访问当前路由 94
4.1.9. 跨域资源共享(CORS) 94
4.1.10. 路由缓存 94
4.2. 中间件 95
4.2.1. 简介 95
4.2.2. 定义中间件 95
4.2.3. 注册中间件 96

4.2.3.1. 全局中间件 96
4.2.3.2. 指定路由中间件 96
4.2.3.3. 中间件组 98
4.2.3.4. 排序中间件 98
4.2.4. 中间件参数 98
4.2.5. 可终止的中间件 99
4.3. CSRF 保护 101
4.3.1. 简介 101
4.3.2. 阻止 CSRF 请求 102
4.3.3. X-CSRF-TOKEN 102
4.3.4. X-XSRF-TOKEN 102
4.4. 控制器 102
4.4.1. 简介 102
4.4.2. 编写控制器 103
4.4.2.1. 基本控制器 103
4.4.2.2. 单一动作控制器 104
4.4.3. 控制器中间件 105
4.4.4. 资源控制器 105
4.4.4.1. 部分资源路由 105
4.4.4.2. 嵌套资源 106
4.4.4.3. 命名资源路由 106
4.4.4.4. 命名资源路由参数 106
4.4.4.5. 限制范围的资源路由 107
4.4.4.6. 本地化资源 URIs 107

4.4.4.7. 补充资源控制器 108
4.4.5. 依赖注入&控制器 108
4.4.5.1. 构造函数注入 108
4.4.5.2. 方法注入 109
4.5. HTTP 请求 111
4.5.1. 简介 111
4.5.2. 与请求交互 111
4.5.2.1. 访问请求 111
4.5.2.2. 请求路径和方法 112
4.5.2.2.1. 检索请求路径 112
4.5.2.2.2. 检索请求路径/路由 112
4.5.2.2.3. 检索请求 URL 113
4.5.2.3. 请求头 113
4.5.2.4. 请求 IP 地址 113
4.5.2.5. 请求协商 113
4.5.2.6. PSR-7 Requests 114
4.5.3. 输入 114
4.5.3.1. 检索输入 114
4.5.3.1.1. 检索所有输入数据 114
4.5.3.2. 判断输入是否存在 114
4.5.3.3. 合并附加输入 115
4.5.3.4. 旧数据 115
4.5.4. 输入过滤&规范化 115
4.5.5. 文件 115

4.5.5.1. 获取上传的文件 115
4.5.5.2. 存储上传的文件 116
4.5.6. 配置受信任的代理 116
4.6. HTTP 响应 116
4.6.1. 创建响应 116
4.6.1.1. 在响应中附加 header 信息 117
4.6.1.2. 在响应中附加 Cookies 信息 117
4.6.1.3. Cookies & 加密 117
4.6.2. 重定向 118
4.6.2.1. 重定向到指定名称的路由 118
4.6.2.2. 重定向到控制器行为 119
4.6.2.3. 重定向到外部域名 119
4.6.2.4. 重定向并使用闪存的 Session 数据 119
4.6.3. 其他响应类型 119
4.6.3.1. 视图响应 119
4.6.3.2. JSON 响应 120
4.6.3.3. 文件下载 120
4.6.3.4. 文件响应 120
4.6.4. 响应宏 121
4.7. 视图 121
4.7.1. 介绍 121
4.7.2. 创建和渲染视图 121
4.7.2.1. 嵌套视图目录 121
4.7.2.2. 创建第一个可用视图 122

4.7.2.3. 判断视图文件是否存在 122
4.7.3. 向视图传递数据 122
4.7.3.1. 与所有视图共享数据 122
4.7.4. 视图 Composer 123
4.7.5. 优化视图 123
4.8. Blade 模板 123
4.8.1. 简介 123
4.8.2. 显示数据 124
4.8.2.1. HTML 实体编码 124
4.8.3. Blade 指令 125
4.8.3.1. If 语句 125
4.9. URL 生成 125
4.9.1. 简介 125
4.9.2. 快速入门 126
4.9.2.1. 生成基础 URL 126
4.9.2.2. 访问当前 URL 126
4.9.3. 命名路由 URL 126
4.9.3.1. 签名 URL 127
4.9.4. 控制器行为 URL 127
4.9.5. 默认值 127
4.10. Session 128
4.10.1. 简介 128
4.10.1.1. 配置 128
4.10.1.2. 驱动先决条件 128

4.10.1.2.1. 数据库 128
4.10.1.2.2. Redis 129
4.10.2. 使用 Session 129
4.10.2.1. 获取数据 129
4.10.2.2. 存储数据 129
4.10.2.3. 闪存数据 130
4.10.2.4. 删除数据 130
4.10.2.5. 重新生成 SessionID 130
4.10.3. Session 阻塞 131
4.10.4. 添加自定义 Session 驱动 131
4.11. 验证 132
4.11.1. 简介 132
4.11.2. 快速入门 132
4.11.2.1. 定义路由 132
4.11.2.2. 创建控制器 132
4.11.2.3. 编写验证逻辑 134
4.11.2.4. 显示验证错误信息 135
4.11.2.5. 回填表单 135
4.11.2.6. 可选字段注意事项 135
4.11.3. 验证表单请求 136
4.11.3.1. 创建表单请求验证 136
4.11.3.2. 表单请求认证授权 137
4.11.3.3. 自定义错误消息 137
4.11.3.4. 自定义验证属性 137

4.11.3.5. 准备验证接入 138
4.11.4. 手动创建验证器 139
4.11.4.1. 自动重定向 140
4.11.4.2. 命名错误包 141
4.11.4.3. 自定义错误消息 141
4.11.4.4. 验证后钩子 142
4.11.5. 处理验证字段 143
4.11.6. 错误信息处理 143
4.11.6.1. 自定义验证错误信息 143
4.11.7. 可用验证规则 143
4.11.8. 按条件添加条件规则 144
4.11.9. 验证数组 144
4.11.10. 自定义验证规则 144
4.11.10.1. 使用验证规则对象 144
4.11.10.2. 使用闭包 144
4.11.10.3. 隐式规则 144
4.12. 错误处理 145
4.12.1. 简介 145
4.12.2. 配置 145
4.12.3. 异常处理 145
4.12.3.1. 异常报告 145
4.12.3.2. 按类型忽略异常 146
4.12.3.3. 渲染异常 147
4.12.3.4. Reportable & Renderable 异常 148

4.12.4. HTTP 异常 150
4.12.4.1. 自定义 HTTP 错误页面 150
4.13. 日志 150
4.13.1. 简介 150
4.13.2. 配置 150
4.13.2.1. 可用的通道驱动 151
4.13.2.2. 通道先决条件 151
4.13.2.3. 记录弃用警告 151
4.13.3. 构建日志堆栈 152
4.13.4. 编写日志消息 153
4.13.4.1. 上下文信息 153
4.13.4.2. 写入指定的通道 154
4.13.5. Monolog 通道自正定义 154
4.13.5.1. 为通道自定义 Monolog 154
4.13.5.2. 创建 Monolog 处理器通道 155
4.13.5.3. 通过工厂创建通道 155
第五章 进阶系列 156
5.1. Artisan 控制台 156
5.1.1. 简介 156
5.1.1.1. Tinker 命令(REPL) 156
5.1.2. 编写命令 157
5.1.2.1. 生成命令 157
5.1.2.2. 命令结构 157
5.1.2.3. 闭包命令 159

5.1.3. 定义期望输入 160
5.1.3.1. 参数 160
5.1.3.2. 选项 161
5.1.3.3. 输入数组 161
5.1.3.4. 输入说明 162
5.1.4. 命令 I/O 162
5.1.4.1. 检索输入 162
5.1.4.2. 交互式输入 163
5.1.4.3. 文字输出 164
5.1.5. 注册命令 164
5.1.6. 通过代码执行命令 165
5.1.6.1. 从其他命令调用命令 166
5.2. 广播 167
5.2.1. 简介 167
5.2.2. 服务器安装 168
5.2.2.1. 配置 168
5.2.2.2. Pusher Channels 168
5.2.3. 客户端安装 168
5.2.3.1. Pusher Channels 168
5.2.4. 概念概览 168
5.2.4.1. 使用示例程序 169
5.2.5. 定义广播事件 169
5.2.5.1. 广播名称 169
5.2.5.2. 广播数据 170

5.2.5.3. 广播队列 171
5.2.5.4. 广播条件 172
5.2.6. 授权频道 173
5.2.6.1. 定义授权路由 173
5.2.6.2. 定义授权回调 173
5.2.6.3. 定义频道类 173
5.2.7. 广播事件 174
5.2.7.1. 只广播给他人 174
5.2.8. 接收广播 175
5.2.8.1. 事件监听 175
5.2.8.2. 退出频道 175
5.2.8.3. 命令空间 176
5.2.9. Presence 频道 176
5.2.9.1. 授权 Presence 频道 176
5.2.9.2. 加入 Presence 频道 176
5.2.9.3. 广播到 Presence 频道 176
5.2.10. 模型广播 177
5.2.10.1. 广播约定模型 177
5.2.10.2. 监听模型广播 178
5.2.11. 客户端事件 178
5.2.12. 消息通知 178
5.3. 缓存 179
5.3.1. 简介 179
5.3.2. 配置 179

5.3.2.1. 驱动的前提条件 179
5.3.3. 缓存使用 180
5.3.3.1. 获取缓存实例 180
5.3.3.2. 从缓存中获取数据 181
5.3.3.3. 在缓存中存储数据 181
5.3.3.4. 从缓存中移移除数据 182
5.3.3.5. 缓存辅助函数 182
5.3.4. 缓存标签 182
5.3.4.1. 存储缓存标签 182
5.3.4.2. 访问缓存标签 182
5.3.4.3. 移除缓存标签 183
5.3.5. 原子锁 183
5.3.5.1. 驱动程序先决条件 183
5.3.6. 添加自定义缓存驱动 183
5.3.6.1. 编写驱动 183
5.3.6.2. 注册驱动 184
5.3.6.3. 缓存事件 186
5.4. 集合 187
5.4.1. 简介 187
5.4.1.1. 创建集合 187
5.4.1.2. 扩展集合 188
5.4.2. 集合方法 188
5.4.3. 方法列表 189
5.4.4. 高阶消息传递 189

5.4.5. 懒性集合 189
5.4.5.1. 介绍 189
5.5. 契约 189
5.5.1. 简介 189
5.5.2. 何时使用契约 189
5.5.3. 如何使用契约 190
5.6. 事件 190
5.6.1. 简介 190
5.6.2. 注册事件/监听器 191
5.6.2.1. 生成事件和监听器 191
5.6.2.2. 手动注册事件 192
5.6.2.3. 事件发现 193
5.6.3. 定义事件 193
5.6.4. 定义监听器 194
5.6.5. 事件监听器队列 194
5.6.5.1. 手动访问队列 194
5.6.5.2. 处理失败任务 194
5.6.6. 调度事件 195
5.6.7. 事件订阅者 195
5.6.7.1. 构建事件订阅者 195
5.6.7.2. 注册事件订阅者 195
5.7. 文件存储 195
5.7.1. 简介 195
5.7.2. 配置 195

5.7.2.1. 本地驱动 196
5.7.2.2. 公共磁盘 196
5.7.2.3. 驱动要求 196
5.7.3. 获取磁盘实例 196
5.7.3.1. 按需配置磁盘 197
5.7.4. 检索文件 197
5.7.4.1. 下载文件 197
5.7.4.2. 文件 URLs 197
5.7.4.3. 文件元数据 198
5.7.5. 存储文件 198
5.7.5.1. 文件上传 198
5.7.5.2. 文件可见性 199
5.7.6. 删除文件 200
5.7.7. 目录 200
5.7.8. 自定义文件系统 200
5.8. 辅助函数 200
5.8.1. 简介 200
5.8.2. 方法列表 201
5.9. HTTP 客户端 201
5.9.1. 简介 201
5.9.2. 创建请求 201
5.9.2.1. 请求数据 202
5.9.2.2. 请求头 202
5.9.2.3. 认证 203

5.9.2.4. 超时 203
5.9.2.5. 重试 203
5.9.2.6. 错误处理 204
5.9.3. 测试 204
5.9.3.1. 伪造响应 204
5.9.3.2. 检查请求 205
5.9.4. 事件 205
5.10. 本地化 205
5.10.1. 简介 205
5.10.1.1. 配置语言环境 205
5.10.2. 定义翻译字符串 205
5.10.2.1. 使用短键 205
5.10.3. 检索翻译字符串 206
5.10.3.1. 替换翻译字符串中的参数 206
5.10.4. 覆盖扩展包的语言文件 206
5.11. 邮件 206
5.11.1. 简介 206
5.11.1.1. 配置 206
5.11.1.2. 驱动前提 207
5.11.2. 生成 Mailables 207
5.11.3. 编写 Mailables 207
5.11.3.1. 配置发件人 207
5.11.3.2. 配置视图 207
5.11.3.3. 视图数据 207

5.11.3.4. 附件 207
5.11.3.5. 内部附件 207
5.11.3.6. 自定义 SwiftMailer 消息 208
5.11.4. Markdonw 格式邮件 208
5.11.5. 发送邮件 208
5.11.6. 渲染可邮寄类 208
5.11.7. 本地化邮件 208
5.11.8. 邮件&本地开发 209
5.11.9. 事件 209
5.12. 通知 210
5.12.1. 简介 210
5.12.2. 创建通知 210
5.12.3. 发送通知 210
5.12.3.1. 使用 Notifiable Trait 210
5.12.3.2. 使用 Notification Facade 211
5.12.4. 邮件通知 211
5.12.4.1. 格式化邮件消息 211
5.12.4.2. 自定义发送人 212
5.12.4.3. 自定义收件人 213
5.12.4.4. 自定义主题 214
5.12.5. Markdown 邮件通知 214
5.12.5.1. 生成消息 214
5.12.5.2. 编写消息 215
5.12.5.3. 自定义组件 215

5.12.6. 数据库通知 216
5.12.6.1. 先决条件 216
5.12.6.2. 格式化数据库通知 216
5.12.6.3. 访问通知 217
5.12.6.4. 标记通知已读 217
5.12.7. 广播通知 218
5.12.7.1. 先决条件 218
5.12.7.2. 格式化广播通知 218
5.12.7.3. 监听通知 219
5.12.8. 短信 219
5.12.8.1. 先决条件 219
5.12.8.2. 格式化短信通知 220
5.12.8.3. 自定义发件人号码 220
5.12.9. Slack 通知 221
5.12.9.1. 先决条件 221
5.12.9.2. 格式化 Slack 通知 221
5.12.9.3. Slack 附件 221
5.12.10. 本地化通知 222
5.12.10.1. 用户首选语言环境 222
5.12.11. 通知事件 223
5.12.12. 自定义通道 223
5.13. 扩展包开发 225
5.13.1. 简介 225
5.13.1.1. 关于 Facades 225

5.13.2. 包自动发现 225
5.13.2.1. 选择退出包发现 226
5.13.3. 服务提供者 227
5.13.4. 资源 227
5.13.4.1. 配置 227
5.13.4.2. 路由 228
5.13.4.3. 迁移 228
5.13.4.4. 语言包 229
5.13.4.5. 视图 230
5.13.4.6. 视图组件 230
5.13.5. 命令 231
5.13.6. 发布前端资源 232
5.13.7. 发布文件组 233
5.14. 队列 234
5.14.1. 简介 234
5.14.1.1. 连接 Vs 队列 234
5.14.1.2. 驱动程序说明和先决条件 235
5.14.2. 创建任务 236
5.14.2.1. 生成任务类 236
5.14.2.2. 任务类结构 236
5.14.2.3. Unique Jobs 238
5.14.3. 任务中间件 239
5.14.3.1. 访问限制 239
5.14.4. 调度任务 240

5.14.4.1. 延迟调度 241
5.14.4.2. 同步调度 242
5.14.4.3. 任务链 243
5.14.4.4. 自定义队列和连接 244
5.14.4.5. 指定任务最大尝试次数/超时值 245
5.14.4.6. 错误处理 246
5.14.5. 任务批处理 246
5.14.5.1. 定义可批处理任务 247
5.14.5.2. 调度批次 247
5.14.6. 队列闭包 247
5.14.7. 运行队列工作者 248
5.14.7.1. queue:work 命令 248
5.14.7.2. 队列优先级 248
5.14.8. Supervisor 配置 249
5.14.9. 处理失败任务 249
5.14.9.1. 任务失败后清理 250
5.15. 限流 250
5.15.1. 简介 250
5.15.1.1. 缓存配置 250
5.15.2. 基础用法 250
5.15.2.1. 手动配置尝试次数 250
5.15.2.2. 清除尝试次数 251
5.16. 任务调度 252
5.16.1. 简介 252

5.16.2. 定义调度 252
5.16.2.1. 调度 Artisan 命令 253
5.16.2.2. 调度队列任务 254
5.16.2.3. 调度 Shell 命令 254
5.16.2.4. 调度频率选项 254
5.16.2.5. 时区 255
5.16.2.6. 避免任务重复 255
5.16.2.7. 任务只运行在单台服务器上 256
5.16.2.8. 后台任务 256
5.16.2.9. 维护模式 256
5.16.3. 运行调度程序 257
5.16.4. 本地运行调度程序 257
5.16.5. 任务输出 257
5.16.6. 任务钩子 258
5.16.7. 事件 259
第六章 安全相关 259
6.1. 用户认证 259
6.1.1. 简介 259
6.1.1.1. 入门套件 260
6.1.1.2. 数据库注意事项 260
6.1.2. 快速入门 260
6.1.2.1. 安装入门套件 260
6.1.2.2. 获取已验证用户 261
6.1.2.3. 路由保护 261

6.1.2.4. 登录限流 262
6.1.3. 手动验证用户 262
6.1.3.1. 记住用户 262
6.1.3.2. 其他认证方法 262
6.1.4. HTTP 基本认证 263
6.1.4.1. 无状态 HTTP 基本认证 263
6.1.5. 退出登录 263
6.1.5.1. 其他设备的会话无效 263
6.1.6. 添加自定义保护 263
6.1.6.1. 关闭请求保护 263
6.1.7. 添加自定义用户提供者 264
6.1.7.1. 用户提供程序契约 264
6.1.7.2. 可认证的契约 264
6.1.8. 事件 264
6.2. 用户授权 264
6.2.1. 简介 264
6.2.2. Gates 265
6.2.2.1. 编写 Gates 265
6.2.2.2. 授权动作 266
6.2.2.3. Gate 响应 266
6.2.2.4. Gate 拦截检查 267
6.2.3. 创建策略 267
6.2.3.1. 生成策略 267
6.2.3.2. 注册策略 268

6.2.4. 编写策略 268
6.2.4.1. 策略方法 268
6.2.4.2. 策略响应 268
6.2.4.3. 不包含模型方法 268
6.2.4.4. Guest 用户 268
6.2.4.5. 策略过滤器 268
6.2.5. 使用策略授权动作 268
6.2.5.1. 通过用户模型 268
6.3. 邮箱验证 269
6.3.1. 简介 269
6.3.2. 模型准备 269
6.3.3. 数据库准备 269
6.3.4. 路由 269
6.4. 加密解密 270
6.4.1. 简介 270
6.4.2. 设置 270
6.4.3. 基本用法 270
6.5. 哈希 272
6.5.1. 简介 272
6.5.2. 配置 272
6.5.3. 基本用法 272
6.6. 密码重置 274
6.6.1. 简介 274
6.6.1.1. 数据库准备 274

6.6.2. 路由 274
6.6.2.1. 请求密码重置链接 274
6.6.2.2. 重置密码 274
6.6.3. 自定义 274
第七章 数据库 275
7.1. 快速入门 275
7.1.1. 简介 275
7.1.1.1. 配置 275
7.1.1.1.1. SQLite 配置 275
7.1.1.1.2. URL 形式配置 276
7.1.1.2. 读写分离 276
7.1.2. 运行原生 SQL 查询 277
7.1.2.1. 使用多个数据库连接 278
7.1.2.2. 监听查询事件 279
7.1.3. 数据库事务 280
7.1.3.1. 处理死锁 281
7.1.3.2. 手动处理事务 281
7.1.4. 连接到数据库 CLI 282
7.2. 查询构建器 282
7.2.1. 简介 282
7.2.2. 获取结果集 282
7.2.2.1. 分块结果 283
7.2.2.2. 聚合 284
7.2.3. 查询（Select） 285

7.2.4. 原生表达式 285
7.2.5. 连接（Joins） 285
7.2.5.1. 内连接 285
7.2.5.2. 左连接/右连接 286
7.2.5.3. 交叉连接 286
7.2.5.4. 高级连接语句 286
7.2.5.5. 子查询连接 287
7.2.6. 联合（Union） 287
7.2.7. Where 子句 288
7.2.7.1. 参数分组 288
7.2.7.2. Where exists 子句 289
7.2.7.3. 子查询 Where 子句 290
7.2.7.4. JSON Where 子句 290
7.2.8. 排序、分组 291
7.2.8.1. orderBy 291
7.2.9. 条件子句 291
7.2.10. 插入 292
7.2.11. 更新 293
7.2.11.1. 更新或新增 293
7.2.11.2. 更新 JSON 294
7.2.11.3. 自增自减 294
7.2.12. 删除 295
7.2.13. 悲观锁 295
7.2.14. 调试 295

7.3. 分页 296
7.3.1. 简介 296
7.3.2. 基本用法 296
7.3.2.1. 查询构造器分页 296
7.3.2.2. Eloquent 分页 297
7.3.2.3. 游标分页 298
7.3.2.4. 手动创建分页 298
7.3.3. 显示分页视图 298
7.3.3.1. 调整分页链接接口 298
7.3.3.2. 将结果转换为 JSON 299
7.3.4. 自定义分页视图 299
7.3.4.1. 使用 Bootstrap 299
7.3.5. 分页器实例方法 300
7.4. 迁移 300
7.4.1. 简介 300
7.4.2. 生成迁移 301
7.4.2.1. 整合迁移 301
7.4.3. 迁移结构 301
7.4.4. 运行迁移 303
7.4.4.1. 回滚迁移 303
7.4.5. 数据表 303
7.4.5.1. 创建数据表 303
7.4.5.2. 更新数据表 304
7.4.5.3. 重命名/删除数据表 304

7.4.6. 字段 304
7.4.6.1. 创建字段 304
7.4.6.2. 可用的字段类型 305
7.4.6.3. 字段修饰符 305
7.4.6.4. 修改字段 306
7.4.6.5. 删除字段 306
7.4.7. 索引 306
7.4.7.1. 创建索引 306
7.4.7.2. 重命名索引 307
7.4.7.3. 删除索引 307
7.4.7.4. 外键索引 307
7.4.8. 事件 308
7.5. 数据填充 308
7.5.1. 简介 308
7.5.2. 编写 Seeders 308
7.5.2.1. 使用模型工厂 308
7.5.2.2. 调用其他 Seeders 309
7.5.3. 运行 Seeders 310
7.6. Redis 310
7.6.1. 简介 310
7.6.2. 配置 310
7.6.2.1. 集群 311
7.6.2.2. Predis 312
7.6.2.3. PhpRedis 313

7.6.3. 与 Redis 交互 313
7.6.3.1. 事务 313
7.6.3.2. 管道命令 314
7.6.4. 发布/订阅 314
第八章 Eloquent ORM 316
8.1. 快速入门 316
8.1.1. 简介 316
8.1.2. 生成模型类 317
8.1.3. Eloquent 模型约定 317
8.1.3.1. 数据表名称 317
8.1.3.2. 主键 318
8.1.3.3. 时间戳 320
8.1.3.4. 默认属性值 320
8.1.4. 检索模型 321
8.1.4.1. 集合 322
8.1.4.2. 结果分块 322
8.1.4.3. 使用惰性集合进行分块 323
8.1.4.4. 游标 324
8.1.5. 高级子查询 324
8.1.5.1. selects 子查询 324
8.1.5.2. 子查询排序 325
8.1.6. 检索单个模型/聚合结果 325
8.1.6.1. 检索或创建模型 326
8.1.6.2. 检索单条唯一的模型 327

8.1.6.3. 检索聚合 327
8.1.7. 插入/更新模型 327
8.1.7.1. 插入 327
8.1.7.2. 更新 329
8.1.7.3. 批量赋值 329
8.1.7.4. 其他创建方法 330
8.1.8. 删除模型 330
8.1.8.1. 软删除 330
8.1.8.2. 查询软删除模型 331
8.1.9. 修剪模型 331
8.1.10. 复制模型 331
8.1.11. 查询作用域 332
8.1.11.1. 全局作用域 332
8.1.11.2. 本地作用域 332
8.1.12. 模型比较 333
8.1.13. 事件 333
8.1.13.1. 使用闭包 333
8.1.13.2. 观察者 334
8.1.13.3. 静默事件 335
8.2. 关联关系 335
8.2.1. 简介 335
8.2.2. 定义关联关系 335
8.2.2.1. 一对一 336
8.2.2.2. 一对多 336

8.2.2.3. 一对多(反向) / 属于 337
8.2.3. 多对多关联 338
8.2.4. 多态关系 338
8.2.4.1. 一对一(多态) 339
8.2.4.2. 一对多(多态) 339
8.2.4.3. 多对多(多态) 339
8.2.5. 关联查询 339
8.2.5.1. 关联方法 Vs 动态属性 339
8.2.5.2. 查询已存在的关联 340
8.2.5.3. 查询不存在的关联 340
8.2.5.4. 多态关联查询 340
8.2.6. 聚合关联模型 341
8.2.7. 预加载 341
8.2.8. 插入&更新关联模型 341
8.2.9. 更新父级时间戳 341
8.3. Eloquent 集合 341
8.3.1. 简介 341
8.3.2. 可用的方法 341
8.3.3. 自定义集合 342
8.4. 访问器和修改器 343
8.4.1. 简介 343
8.4.2. 访问器&修改器 343
8.4.2.1. 定义一个访问器 343
8.4.2.2. 定义一个修改器 343

8.4.3. 日期修改器 344
8.4.4. 属性转换 345
8.4.4.1. 数组 & JSON 转换 345
8.4.5. 自定义类型转换 346
8.4.5.1. 值对象转换 346
8.4.5.2. 数组/ JSON 序列化 346
8.4.5.3. 入站转换 346
8.5. API 资源类 346
8.5.1. 简介 346
8.5.2. 生成资源 347
8.5.3. 核心概念 347
8.5.3.1. 资源集合 347
8.5.4. 编写资源 347
8.5.4.1. 数据包裹 347
8.5.4.2. 分页 348
8.5.4.3. 条件属性 349
8.5.4.4. 条件关联 350
8.5.4.5. 添加元数据 350
8.5.5. 资源响应 350
8.6. 序列化 350
8.6.1. 简介 350
8.6.2. 序列化模型&集合 351
8.6.2.1. 序列化为数组 351
8.6.2.2. 序列化为 JSON 351

8.6.3. 隐藏 JSON 属性 351
8.6.4. 追加 JSON 值 352
8.6.5. 日期序列化 353
第九章 测试系列 354
9.1. 快速入门 354
9.1.1. 简介 354
9.1.2. 环境 354
9.1.3. 创建测试 355
9.1.4. 运行测试 355
9.1.4.1. 并行运行测试 355
9.1.4.2. 报告测试覆盖率 355
9.2. HTTP 测试 355
9.2.1. 简介 355
9.2.2. 创建请求 357
9.2.2.1. 自定义请求头 357
9.2.2.2. 测试响应 358
9.2.3. Session/认证 359
9.2.4. 测试 JSON APIs 360
9.2.4.1. 验证 JSON 完全匹配 361
9.2.4.2. 验证 JSON 路径 362
9.2.5. 测试文件上传 363
9.3. 命令行测试 364
9.3.1. 简介 364
9.3.2. 期望输入/输出 364

9.4. 数据库测试 365
9.4.1. 简介 365
9.4.2. 生成模型工厂 366
9.4.3. 每次测试后重置数据库 366
9.4.4. 编写模型工厂 366
9.4.4.1. 工厂扩展 367
9.4.4.2. 工厂状态 367
9.4.4.3. 工厂回调 368
9.5. 模拟器 368
9.5.1. 简介 368
9.5.2. 模拟对象 368
9.5.3. 任务模拟 369
9.5.4. 事件模拟 369
9.5.5. 邮件模拟 369
9.5.6. 通知模拟 369
第十章 附录：官方扩展包 370
10.1. 交易工具包（Stripe） 370
10.1.1. 简介 370
10.1.2. 安装 370
10.1.2.1. 数据库迁移 370
10.2. 交易工具包（Paddle） 370
10.2.1. 简介 370
10.2.2. 安装 371
10.3. Envoy 部署工具 371

10.3.1. 简介 371

10.3.2. 安装 371

10.4. Horizon 队列管理工具 371

10.4.1. 简介 371

10.4.2. 安装 371

第十一章 补充说明 372

第一章 序言

1.1. 发行说明

原文地址：https://laravel.com/docs/9.x/releases

1.1.1. 版本化方案

Laravel 及官方发布的包皆遵循语义化版本。主要的框架版本每年「2 月」发布，而次要的和补丁版本可能每周发布一次。次要版本和修补程序版本应从不 包含非兼容性的更改。

你从应用或包中引用 Laravel 框架或其组件时，应始终使用版本约束，如 ^9.0，因为 Laravel 的主要版本确实包含非兼容性更改。但是，我们努力确保您可以在一天或更短的时间内更新到新的主要版本。

命名参数

目前，PHP 的 命名参数 功能还没有被 Laravel 的向后兼容性指南所涵盖。我们可以在必要时选择重命名函数参数，以改进 Laravel 代码库。因此，在调用 Laravel 方法时使用命名参数应该谨慎，并且要理解参数名将来可能会改变。

1.1.2. 支持策略

对于所有 Laravel 发行版本，BUG 修复的期限为 18 个月，安全修复的期限为 2 年。对于包括 Lumen 在内的所有额外的库，只有最新的版本才会得到 BUG 修复。此外，请查阅 Laravel 支持的 数据库版本。

版本	PHP (*)	发布时间	Bug 修复截止时间	安全修复截止时间
10	8.1-8.3	2023 年 2 月 14 日	2024 年 8 月 6 日	2025 年 2 月 4 日
9 (LTS)	8.0-8.2	2022 年 2 月 8 日	2023 年 8 月 8 日	2024 年 2 月 6 日

8	7.3-8.1	2020 年 9 月 8 日	2022 年 7 月 26 日	2023 年 1 月 24 日
7	7.2-8.0	2020 年 3 月 3 日	2020 年 10 月 6 日	2021 年 3 月 3 日
6 (LTS)	7.2-8.0	2019 年 9 月 3 日	2022 年 1 月 25 日	2022 年 9 月 6 日
5.8		2019 年 2 月 26 日	2019 年 8 月 26 日	2020 年 2 月 26 日
5.7		2018 年 9 月 4 日	2019 年 3 月 4 日	2019 年 9 月 4 日
5.6		2018 年 2 月 7 日	2018 年 8 月 7 日	2019 年 2 月 7 日
5.5 (LTS)		2017 年 8 月 30 日	2019 年 8 月 30 日	2020 年 8 月 30 日

生命周期结束 仅安全修改

(*) 支持的 PHP 版本

1.1.3. Laravel 9

正如你所知，随着 Laravel 8 的发布，Laravel 已经过渡到了年度发布。以前，主要版本每 6 个月发布一次。这一转变旨在减轻社区的维护负担，并挑战我们的开发团队在不引入突破性更改的情况下提供惊人、强大的新功能。因此，我们在不破坏向后兼容性的情况下，向 Laravel 8 提供了各种强大的功能，例如并行测试支持、改进的 Breeze starter 工具包、HTTP 客户端改进，甚至还有新的 Eloquent 关联关系类型，例如 oFMany(一对多检索)。

因此，在当前版本中发布新功能的承诺可能会导致未来的「主要」版本主要用于「维护」任务，例如升级上游依赖项，这可以在这些发行说明中看到。

Laravel 9 延续了 Laravel 8.x 的改进通过引入对 Symfony 6.0 组件、Symfony Mailer、Flysystem 3.0、改进的 routes:list 输出、Laravel Scout 数据库驱动程序、新的 Eloquent 访问器 / 修改器语法、通过枚举的隐式路由绑定，以及其他各种错误修复和可用性改进。

1.1.3.1. PHP 8.0

Laravel 9.x 至少需要 PHP8.0.2。

1.1.3.2. Symfony Mailer

Symfony Mailer 的支持是由 Dries Vints, James Brooks, 和 Julius Kiekbusch.

Laravel 以前的版本使用了 Swift Mailer 库发送外发邮件。然而，该库已不再维护，由 Symfony Mailer 继承。

1.1.3.3. Flysystem 3.x

Flysystem 3.x 的支持由 Dries Vints 提供。

Laravel 9.x 将我们上游的 Flysystem 依赖升级到 Flysystem 3.x。 Flysystem 为 Storage 门面提供的所有文件系统交互提供支持。

1.1.3.4. Eloquent 访问器/修改器改进

改进的 Eloquent 访问器 / 修改器由 Taylor Otwell 贡献。

Laravel 9.x 提供了一种新的方式来定义 Eloquent 访问器和修改器。在以前的 Laravel 版本中，定义访问器和修改器的唯一方法是在模型上定义前缀方法，如下所示：

```
public function getNameAttribute($value)
{
    return strtoupper($value);
}
```

```
public function setNameAttribute($value)
{
    $this->attributes['name'] = $value;
}
```

1.1.3.5. Enum Eloquent 属性转换

枚举转换由 Mohamed Said 贡献。

1.1.3.6. 使用枚举的隐式路由绑定

隐式路由绑定由 Nuno Maduro 贡献。

PHP 8.1 引入了对 Enums 的支持。Laravel 9.x 引入了在路由定义中键入提示 Enum 的能力，并且 Laravel 只会在该路由段是 URI 中的有效 Enum 值时调用该路由。 否则，将自动返回 HTTP 404 响应。

1.1.3.7. 路由绑定的强制作用域名

路由绑定的强制作用域由 Claudio Dekker 贡献.

在之前的 Laravel 版本中，您可能希望在路由定义中限定第二个 Eloquent 模型，使其必须是之前 Eloquent 模型的子模型。

1.1.3.8. 控制器路由组

控制器路由组改进由 Luke Downing 贡献.

您现在可以使用 controller 方法为组内的所有路由定义公共控制器。

1.1.3.9. 全文索引/Where 子名

全文索引和”where” 子句由 Taylor Otwell 和 Dries Vints 贡献。

使用 MySQL 或 PostgreSQL 时，现在可以将 fullText 方法添加到列定义中以生成全文索引：

```
$table->text('bio')->fullText();
```

1.1.3.10.Laravel Scout 数据库引擎

Laravel Scout 数据库引擎由 Taylor Otwell and Dries Vints 贡献。

如果您的应用程序与中小型数据库交互或工作量较小，您现在可以使用 Scout 的 “数据库” 引擎，而不是 Algolia 或 MeiliSearch 等专用搜索服务。数据库引擎将在过滤现有数据库的结果时使用 “where like” 子句和全文索引，以确定查询的适用搜索结果。

1.2. 升级说明

1.2.1. 升级指南

1.2.1.1. 从 8.x 升级到 9.0

预计升级时间：30 分钟

1.2.2. 高影响变化

1.2.2.1. 更新依赖

要求 PHP 8.0.2

Laravel 现在 需要 PHP 8.0.2 or 更高。

1.2.2.2. Flysystem 3.x

影响的可能性：高

Laravel 9.x 已从 Flysystem 1.x 迁移到 3.x。 在底层，Flysystem 支持 Storage 门面提供的所有文件操作方法。 鉴于此，您的应用程序中可能需要进行一些更改； 但是，我们已尝试使这种过渡尽可能无缝。

1.2.2.3. Symfony Mailer

影响的可能性：高

Laravel 9.x 中最大的变化之一是从 SwiftMailer（截至 2021 年 12 月不再维护）到 Symfony Mailer 的过渡。 但是，我们已尝试使您的应用程序尽可能无缝地过渡。 话虽如此，请仔细查看下面的更改列表，以确保您的应用程序完全兼容。

1.2.3. 中影响变化

1.2.3.1. Belongs To Many

影响程度：中

belongsToMany 关系的 firstOrNew、firstOrCreate 和 updateOrCreate 方法都接受一个属性数组作为其第一个参数。在 Laravel 的先前版本中，这个属性数组相当于现有记录的 pivot 中间表。

1.3. 贡献导引

1.3.1. Bug 报告

为了鼓励积极的合作，Laravel 强烈鼓励提交 pull request，而不仅仅是提交「Bug 报告」。「Bug 报告」也可以包含失败测试的 pull request 的形式发送。只有在标记为「准备好审核」（而不是处于「草稿」状态）并且新功能的所有测试都通过时，才会审核拉取请求。停留在「草稿」状态的、非活动的拉取请求将在几天后关闭。

然而，如果你提交了一个 Bug 报告，你的问题（issue）应该包含一个标题和对问题的清晰描述。你还应该包含尽可能多的相关信息以及演示该问题的代码示例。错误报告的目标是使您自己（以及其他人）能够轻松地复现错误并开发修复程序。

请记住，创建 Bug 报告是希望有相同问题的其他人能够与您协作解决它。不要期望 Bug 报告能够自动查看任何活动或者其他人跳转自此来修复它。创建 Bug 报告有助于你自己和其他人开始修复问题。如果你想加入，你可以通过修复 issues 列表 来提供帮助。你必须通过 GitHub 认证才能查看 Laravel 的所有问题。

Laravel 源码托管在 GitHub 上，每个项目都有一些仓库：

Laravel Application

Laravel Art

Laravel Documentation

Laravel Dusk

Laravel Cashier Stripe

Laravel Cashier Paddle

Laravel Echo

Laravel Envoy

Laravel Framework

Laravel Homestead

Laravel Homestead Build Scripts

Laravel Horizon

Laravel Jetstream

Laravel Passport

Laravel Sail

Laravel Sanctum

Laravel Scout

Laravel Socialite

Laravel Telescope

Laravel Website

1.3.2. 帮助支持

Laravel 的 GitHub 的 issue 功能不打算提供 Laravel 的帮助或支持。相反，使用以下渠道之一。

GitHub 讨论

Laracasts 论坛

Laravel.io 论坛

StackOverflow

Discord

Larachat

IRC

1.3.3. 核心发展讨论

你可以在 Laravel 框架存储库的 GitHub 讨论区 中提出新功能或对现有 Laravel 行为的改进。 如果您提出一项新功能，请愿意至少实现一些完成该功能所需的代码。

在 Laravel Discord 服务器 的 #internals 频道中进行了有关错误、新功能和现有功能实现的非正式讨论。 Laravel 的维护者 Taylor Otwell 通常在工作日的上午 8 点至下午 5 点（UTC-06:00 或美国 / 芝加哥）出现在频道中，并在其他时间偶尔出现在频道中。

1.3.4. 哪个分支

所有错误修复应发送到最新的稳定分支。 错误修复不应该被发送到 master 分支，除非它们修复了仅存在于即将发布的版本中的功能。

与当前版本完全向后兼容的次要 ** 功能可能会发送到最新的稳定分支。

主要 新功能应始终发送到包含即将发布的版本的 master 分支。

1.3.5. 编译文件

如果您提交的更改会影响已编译文件，例如 laravel/laravel 存储库的 resources/css 或 resources/js 中的大部分文件，请不要提交已编译文件。 由于它们的体积很大，维护者实际上无法对其进行审查。 这可能被用作将恶意代码注

入 Laravel 的一种方式。 为了防御性地防止这种情况发生，所有编译的文件都将由 Laravel 维护者生成和提交。

1.3.6. 安全漏洞

如果您在 Laravel 中发现一个安全漏洞，请发送电子邮件至 Taylor Otwell 。所有安全漏洞将得到及时解决。

1.3.7. 编码风格

Laravel 遵循 PSR-2 编码标准和 PSR-4 自动加载标准。

1.3.7.1. PHP 文档

下面是一个有效的 Laravel 文档块的示例。 请注意，@param 属性后跟两个空格、参数类型、另外两个空格，最后是变量名：

```
/**
 * 向容器注册绑定。
 *
 * @param  string|array  $abstract
 * @param  \Closure|string|null  $concrete
 * @param  bool  $shared
 * @return void
 *
 * @throws \Exception
 */
public function bind($abstract, $concrete = null, $shared = false)
{
    //
}
```

1.3.7.2. StyleCI

如果您的代码样式不完美，请不要担心！ StyleCI 将在合并拉取请求后自动将任何样式修复合并到 Laravel 存储库中。 这使我们能够专注于贡献的内容，而不是代码风格。

1.3.8. 行为准则

Laravel 行为准则源自 Ruby 行为准则。任何违反行为准则的行为都可以报告给 Taylor Otwell（taylor@laravel.com）：

参与者容忍反对意见。

参加者必须确保他们的语言和行为没有人身攻击和贬低个人言论。

在解释他人的言行时，参与者应始终保持良好的意图。

不能容忍可合理视为骚扰的行为。

第二章 快速入门

2.1. 安装

2.1.1. 认识 Laravel

Laravel 是一个 Web 应用框架， 有着表现力强、语法优雅的特点。Web 框架为创建应用提供了一个结构和起点，你只需要专注于创造，我们来为你处理细节。

Laravel 致力于提供出色的开发体验，同时提供强大的特性，例如完全的依赖注入，富有表现力的数据库抽象层，队列和计划任务，单元和集成测试等等。

无论你是刚刚接触 PHP 和 Web 框架的新人，亦或是有着多年经验的老手，Laravel 都是一个可以与你一同成长的框架。我们将帮助你迈出成为 Web 开发者的第一步，或是将你的经验提高到下一个等级。我们迫不及待的想看看你的作品。

2.1.1.1. 为什么选择 Laravel?

有非常多的工具和框架可以被用于构建一个 Web 应用。但我们相信 Laravel 是构建现代化、全栈 Web 应用的最佳选择。

2.1.2. 你的第一个 Laravel 项目

我们希望尽可能轻松地开始使用 Laravel。在你自己的计算机上开发和运行 Laravel 项目有多种选择。虽然你可能希望以后再研究这些选项，但 Laravel 提供了 Sail，使用 Docker 运行 Laravel 项目的内置解决方案

Docker 是一种在小型、轻量级的「容器」中运行应用程序和服务的工具，它不会干扰本地计算机安装的软件或配置。这意味着你不必担心在个人计算机上配置或设置复杂的开发工具，例如 web 服务器和数据库。要开始，你只需安装 Docker Desktop。

Laravel Sail 是一个轻量级的命令行界面，用于与 Laravel 的默认 Docker 配置进行交互。Sail 为使用 PHP、MySQL 和 Redis 构建 Laravel 应用程序提供了一个很好的起点，而无需之前的 Docker 经验。

2.1.2.1. macOS 入门

如果你在 Mac 上开发并且已经安装了 Docker Desktop，你可以使用一个简单的终端命令来创建一个新的 Laravel 项目。

2.1.2.2. Windows 入门

在新建 Laravel 应用前，请确保你的 Windows 电脑已经安装了 Docker Desktop。之后，请确保已经安装并启用了适用于 Linux 的 Windows 子系统 2（WSL2）。

2.1.2.3. 大 Linux 使用 Laravel Sail

如果在 Linux 开发，并且已经安装了 Docker Compose，你可以使用简单的终端命令来创建一个新的 Laravel 项目。

2.1.3. 初始配置

Laravel 框架的所有配置文件都存储在 config 目录中。每个选项都有文档记

录，所以请随意浏览这些文件并熟悉可用的选项。

Laravel 几乎不需要额外的配置。你可以自由地开始开发！然而，你可能希望查看 config/app.php 文件及其文档。它包含几个选项，比如 timezone 和 locale，你可能希望根据你的应用程序进行更改。

2.1.3.1. 基于环境的配置

由于 Laravel 的许多配置选项值可能会根据你的应用程序是在本地计算机上还是在生产 Web 服务器上运行而有所不同，因此许多重要的配置值是使用 .env 存在于应用程序根目录中的文件来定义的。

你的 .env 文件不应提交到应用程序的源代码管理，因为每个使用你的应用程序的开发人员 / 服务器可能需要不同的环境配置。此外，如果入侵者获得对你的源代码控制存储库的访问权限，这将是一个安全风险，因为任何敏感凭据都会被暴露。

2.1.3.2. 目录配置

Laravel 应该始终从为你的 Web 服务器配置的「Web 目录」的根目录中提供服务。你不应该尝试从「Web 目录」的子目录中提供 Laravel 应用程序。尝试这样做可能会暴露应用程序中存在的敏感文件。

2.2. 配置

2.2.1. 介绍

Laravel 框架的所有配置文件都存储在 config 目录中。 每个选项都记录在案，因此请随意查看文件并熟悉可用的选项。

这些配置文件允许您配置诸如数据库连接信息、邮件服务器信息以及各种其他核心配置值（例如应用程序时区和加密密钥）之类的内容。

2.2.2. 环境配置

根据应用程序运行的环境设置不同的配置值通常很有帮助。 例如，您可能希望在本地使用与在生产服务器上不同的缓存驱动程序。

为了让这件事变得轻而易举，Laravel 使用了 DotEnv PHP 库。 在全新的 Laravel 安装中，应用程序的根目录将包含一个 .env.example 文件，该文件定义了许多常见的环境变量。 在 Laravel 安装过程中，该文件会自动复制到 .env。

Laravel 的默认 .env 文件包含一些常见的配置值，这些值可能会根据您的应用程序是在本地运行还是在生产 Web 服务器上运行而有所不同。 然后使用 Laravel 的 env 函数从 config 目录中的各种 Laravel 配置文件中检索这些值。

如果您正在与团队一起开发，您可能希望继续在您的应用程序中包含一个 .env.example 文件。 通过将占位符值放入示例配置文件中，您团队中的其他开发人员可以清楚地看到运行您的应用程序需要哪些环境变量。

2.2.2.1. 环境变量类型

.env 文件中的所有变量都被解析为字符串，因此创建了一些保留值以允许你从 env() 函数中返回更多类型的变量。

.evn 值	env() 值
true	(bool) true
(true)	(bool) true
false	(bool) false
(false)	(bool) false
empty	(sting) ''
(empty)	(sting) ''
null	(null) null
(null)	(null) null

2.2.2.2. 获取环境配置

当应用程序收到请求时，.env 文件中列出的所有变量将被加载到 PHP 的超级全局变量 $_ENV 中。你可以使用 env 函数检索这些变量的值。事实上，如果你查看 Laravel 的配置文件，你就能注意到有数个选项已经使用了这个函数。

```
'debug' => env('APP_DEBUG', false),
```

2.2.2.3. 获取当前环境配置

应用程序当前所处环境是通过 .env 文件中的 APP_ENV 变量确定的。你可以通过 App façade 中的 environment 方法来访问此值。

```
use Illuminate\Support\Facades\App;

$environment = App::environment();
```

2.2.3. 访问配置值

您可以在应用程序的任何位置使用全局 config 辅助函数轻松访问配置值。可以使用 “点” 语法访问配置值，其中包括您希望访问的文件名和选项。 也可以指定默认值，如果配置选项不存在，将返回：

```
$value = config('app.timezone');
```

2.2.4. 配置缓存

为了提高应用程序的速度，您应该使用 config:cache Artisan 命令将所有配置文件缓存到一个文件中。这会将应用程序的所有配置选项组合到一个文件中，框架可以快速加载该文件。

您通常应该在生产部署过程中运行 php artisan config:cache 命令。 该命令不应在本地开发期间运行，因为在应用程序开发过程中经常需要更改配置选项。

2.2.5. 调试模式

config/app.php 配置文件中的 debug 选项决定了实际向用户显示的错误信

息量。 默认情况下，此选项设置为尊重 APP_DEBUG 环境变量的值，该变量存储在您的 .env 文件中。

对于本地开发，您应该将 APP_DEBUG 环境变量设置为 true。 在您的生产环境中，此值应始终为 false。 如果在生产环境中将该变量设置为 true，您可能会将敏感的配置值暴露给应用程序的最终用户。

2.2.6. 维护模式

当您的应用程序处于维护模式时，将为您的应用程序的所有请求显示一个自定义视图。 这使得在更新或执行维护时可以轻松 “禁用” 您的应用程序。 维护模式检查包含在应用程序的默认中间件堆栈中。 如果应用程序处于维护模式，则会抛出一个 Symfony\Component\HttpKernel\Exception\HttpException 实例，状态码为 503。

要启用维护模式，请执行 down Artisan 命令：

```
php artisan down
```

2.3. 目录结构

2.3.1. 简介

默认的 Laravel 应用程序结构旨在为大型和小型应用程序提供一个很好的起点。 但是您可以随意组织您的应用程序。 Laravel 几乎不限制任何给定类的位置，只要 Composer 可以自动加载该类。

2.3.2. 根目录

2.3.2.1. App 目录

app 目录包含了应的核心代码。

2.3.2.2. 启动目录

bootstrap 目录包含启动框架的 app.php 文件。 该目录还包含一个「cache」目录，其中包含用于性能优化的框架生成文件，例如路由和服务缓存文件。 您通常不需要修改此目录中的任何文件。

2.3.2.3. 配置目录

config 目录包含了应用所有的配置文件。

2.3.2.4. 数据库目录

database 目录包含数据库迁移，模型工厂和种子生成器文件。

2.3.2.5. 语言目录

lang 目录包含应用程序的所有语言文件。

2.3.2.6. Public 公开目录

public 目录包含了应用入口文件 index.php 和前端资源文件。

2.3.2.7. 资源目录

resources 目录包含了 views 以及未编译的资源文件(如 CSS 或 JavaScript)。此目录还包含所有的语言文件。

2.3.2.8. 路由目录

routes 目录包含了应用定义的所有路由定义。

2.3.2.9. 文件存储目录

storage 目录包含了编译后的 blade 模板、基于文件的 Session、文件缓存，以及其他由框架生成的文件，该目录细分为 app、frameworkd 和 logs 子目录。

2.3.2.10.测试目录

tests 目录包含自动化测试文件，其中默认已经提供了一个开箱即用的 PHPUnit 参考。

2.3.2.11.Vendor 目录

vendor 目录包含了应用所有通过 Composer 加载的依赖。

2.3.3. App 目录

您的大部分应用程序都位于 app 目录中。默认情况下，此目录在 App 下命名，并由 Composer 使用 [PSR-4 自动加载标准](www.php-fig.org/psr/psr-4/) 自动加载。

app 目录包含各种附加目录，例如 Console、Http 和 Providers。将 Console 和 Http 目录视为为应用程序核心提供 API。 HTTP 协议和 CLI 都是与应用程序交互的机制，但实际上并不包含应用程序逻辑。换句话说，它们是向您的应用程序发出命令的两种方式。 Console 目录包含您的所有 Artisan 命令，而 Http 目录包含您的控制器、中间件和请求。

当您使用 make Artisan 命令生成类时，会在 app 目录中生成各种其他目录。因此，例如，在您执行 make:job Artisan 命令生成作业类之前，app/Jobs 目录将不存在。

2.3.3.1. Broadcasting 目录

Broadcating 目录包含应用程序的所有广播通道类。这些类使用 make:channel 命令生成。此目录默认是不存在的，但是当你创建第一个通道时它将被创建。

2.3.3.2. Console 目录

Console 目录包含应用程序的所有自定义 Artisan 命令。这些命令可以使用 make:command 命令生成。

2.3.3.3. Events 目录

默认情况下这个目录是不存在的。

2.3.3.4. Exceptions 目录

Exceptions 目录包含应用程序的异常处理，并且也是一个放置应用程序抛出任何异常的好地方。

2.3.3.5. Http 目录

Http 目录包含你的控制器，中间件和表单请求。处理进入应用程序请求的所有逻辑几乎都放置在些目录。

2.3.3.6. Job 目录

默认情况下这个目录是不存在的。

2.3.3.7. Listeners 目录

默认情况下这个目录是不存在的。

2.3.3.8. Mail 目录

默认情况下这个目录是不存在的。

2.3.3.9. Notifications 目录

默认情况下这个目录是不存在的。

2.3.3.10.Policies 目录

默认情况下这个目录是不存在的。

2.3.3.11.Providers 目录

Providers 目录包含应用程序的服务提供者。服务提供者通过在服务容器中绑定服务引导应用程序，注册事件或者准备为应用程序即将到来的请求执行其它任何任务。

2.3.3.12.Rules 目录

默认情况下这个目录是不存在的，但如果执行 make:rule Artisan 命令，它将被创建出来。

2.4. 部署

2.4.1. 简介

当你准备在生产环境部署 Laravel 应用时，请注意以下几点以使你的应用尽可能高效地运行。本文将会提供几个范本以使你的 Laravel 应用部署妥当。

2.4.2. 服务器要求

Laravel 框架对系统有一些要求，请确保你的 Web 服务器至少满足以下 PHP 版本及扩展需求。

PHP >= 8.0

BCMath PHP 扩展

Ctype PHP 扩展

DOM PHP 扩展

Fileinfo PHP 扩展

JSON PHP 扩展

Mbstring PHP 扩展

OpenSSL PHP 扩展

PCRE PHP 扩展

PDO PHP 扩展

Tokenizer PHP 扩展

XML PHP 扩展

2.4.3. 服务器配置

2.4.3.1. Nginx

如果你想要部署你的应用到 Nginx 服务器上，你可能会用到下面这个配置文件作为一个开始来配置你的 Web 服务器。

2.4.4. 优化

2.4.4.1. 优化自动加载

将项目部署到生产环境时，请确保您正在优化 Composer 的类自动加载器映射，以便 Composer 可以快速找到要为给定类加载的正确文件：

```
composer install --optimize-autoloader --no-dev
```

2.4.4.2. 优化配置加载

将应用程序部署到生产环境时，应确保在部署过程中运行 config:cache Artisan 命令 来提前缓存一些 配置 等信息。

2.4.4.3. 优化路由加载器

如果您正在构建具有许多路由的大型应用程序，则应确保在部署过程中运行

route:cache Artisan 命令。

2.4.4.4. 优化视图加载器

将应用程序部署到生产环境时，应确保在部署过程中运行 view:cache Artisan 命令。

第三章 核心架构

3.1. 生命周期

3.1.1. 简介

在「真实世界」中使用任何工具时，如果你了解该工具的工作原理，你会更加自信。应用程序开发也不例外。当您了解开发工具的功能时，你会觉得使用它们更舒服、更自信。

本文的目的是让您对 Laravel 框架的工作原理有一个良好的、高层次的理解。通过更好地了解整个框架，一切感觉都不那么「神奇」，你将更有信心构建你的应用程序。如果你不明白所有的规则，不要灰心！只要试着对正在发生的事情有一个基本的掌握，你的知识就会随着你探索文档的其他部分而增长。

3.1.2. 生命周期概览

3.1.2.1. 第一步

Laravel 应用程序的所有请求的入口点都是 public/index.php 文件。所有请求都由你的 web 服务器（Apache/Nginx）配置定向到此文件。那个 index.php 文件不包含太多代码。相反，它是加载框架其余部分的起点。

该 index.php 文件将加载 Composer 生成的自动加载器定义，然后从 bootstrap/app.php 中检索 Laravel 应用程序的实例。Laravel 本身采取的第一个操作是创建应用 / 服务容器 的实例。

3.1.2.2. HTTP/Console 内核

接下来，根据进入应用的请求类型，传入的请求将被发送到 HTTP 内核或

者 Console 内核。这两个内核充当所有请求流经的中心位置。现在，我们只关注位于 app/Http/Kernel.php 中的 HTTP 内核。

HTTP 内核继承了 Illuminate\Foundation\Http\Kernel 类，该类定义了一个将在执行请求之前运行的 bootstrappers 数组。这些引导程序用来配置异常处理、配置日志、检测应用程序环境，并执行在实际处理请求之前需要完成的其他任务。通常情况下，你不需要在意这些配置。

HTTP 内核还定义了一个 HTTP 中间件 列表，所有请求在被应用程序处理之前都必须通过该列表。这些中间件处理读写 HTTP 会话，确定应用程序是否处于维护模式，校验 CSRF 令牌 等等。我们接下来会做详细的讨论。

HTTP 内核的 handle 方法的签名非常简单：它接收 Request 接口并返回 Response 接口。把内核想象成一个代表整个应用程序的大黑匣子。向它提供 HTTP 请求，它将返回 HTTP 响应。

3.1.2.3. 服务提供者

最重要的内核引导操作之一是为应用程序加载 服务提供者 。应用程序的所有服务提供程序都在 config/app.php 文件中的 providers 数组。

Laravel 将遍历这个提供者列表并实例化它们中的每一个。实例化提供程序后，将在所有提供程序上调用 register 方法。然后，一旦所有的提供者都被注册了，就会对每个提供程序调用 boot 方法。服务提供者可能依赖于在执行 boot 方法时注册并可用的每个容器绑定。

服务提供者负责引导框架的所有不同组件，如数据库、队列、验证和路由组件。基本上，Laravel 提供的每个主要功能都是由服务提供商引导和配置的。由于它们引导和配置框架提供的许多特性，服务提供者是整个 Laravel 引导过程中最重要的部分。

3.1.2.4. 路由

应用程序中最重要的服务提供者之一是 App\Providers\RouteServiceProvider。此服务提供者加载应用程序的 routes 目录中包含的路由文件。继续，打开 RouteServiceProvider 代码，看看它是如何工作的！

一旦应用程序被引导并且所有服务提供者都被注册，「请求」将被传递给路由器进行调度。路由器将请求发送到路由或控制器，并运行任何路由特定的中间件。

中间件为过滤或检查进入应用程序的 HTTP 请求提供了一种方便的机制。例如，Laravel 包含一个这样的中间件，用于验证应用程序的用户是否经过身份验证。如果用户未通过身份验证，中间件将用户重定向到登录页。但是，如果用户经过身份验证，中间件将允许请求进一步进入应用程序。一些中间件被分配给应用程序中的所有路由，比如那些在 HTTP 内核的 $middleware 属性中定义的路由，而一些只被分配给特定的路由或路由组。您可以通过阅读完整的 中间件文档 来了解关于中间件的信息。

如果请求通过了所有匹配路由分配的中间件，则执行路由或控制器方法，并通过路由的中间件链路返回路由或控制器方法的响应。

3.1.2.5. 最后

一旦路由或控制器方法返回一个响应，该响应将通过路由的中间件返回，从而使应用程序有机会修改或检查传出的响应。

最后，一旦响应通过中间件返回，HTTP 内核的 handle 方法将返回响应对象，并且 index.php 文件在返回的响应上调用 send 方法。 send 方法将响应内容发送到用户的 web 浏览器。至此，我们已经完成了整个 Laravel 请求生命周期的旅程！

3.1.3. 聚焦服务提供者

服务提供者确实是引导 Laravel 应用程序的关键。创建应用程序实例，注册服务提供者，并将请求传递给引导应用程序。就这么简单！

牢牢掌握服务提供者的构建和其对 Laravel 应用处理机制的原理是非常有价值的。你的应用的默认服务提供会存放在 app/Providers 目录下面。

默认情况下，AppServiceProvider 是空白的。这里是用于你添加应用自身的引导处理和服务容器绑定的一个非常棒的地方。在大型项目中，您可能希望创建多个服务提供者，每个服务提供者都为应用程序使用的特定服务提供更细粒度的

引导。

3.2. 服务容器

3.2.1. 简介

Laravel 服务容器是一个用于管理类依赖以及实现依赖注入的强有力工具。依赖注入这个名词表面看起来花哨，实质上是指：通过构造函数，或者某些情况下通过「setter」方法将类依赖「注入」到类中。

3.2.1.1. 零配置解决方案

如果一个类没有依赖项或只依赖于其他具体类（而不是接口），则不需要指定容器如何解析该类。例如，您可以将以下代码放在 routes/web.php 文件中：

```
<?php

class Service
{
    //
}

Route::get('/', function (Service $service) {
    die(get_class($service));
});
```

在这个例子中，点击应用程序的 / 路由将自动解析 Service 类并将其注入到路由的处理程序中。 这是一个有趣的改变。 这意味着您可以开发应用程序并利用依赖注入，而不必担心臃肿的配置文件。

很荣幸的通知您，在构建 Laravel 应用程序时，您将要编写的许多类都可以通过容器自动接收它们的依赖关系，包括 控制器、事件监听器、 中间件 等等。此外，您可以在 队列系统 的 handle 方法中键入提示依赖项。 一旦你尝到了自动和零配置依赖注入的力量，你就会觉得没有它是不可以开发的。

3.2.1.2. 何时使用容器

得益于零配置解决方案，通常情况下，你只需要在路由、控制器、事件侦听器和其他地方键入提示依赖项，而不必手动与容器打交道。例如，可以在路由定义中键入 Illuminate\Http\Request 对象，以便轻松访问当前请求的 Request 类。尽管我们不必与容器交互来编写此代码，但它在幕后管理着这些依赖项的注入：

```
use Illuminate\Http\Request;

Route::get('/', function (Request $request) {
    // ...
});
```

在许多情况下，由于自动依赖注入和 facades，你在构建 Laravel 应用程序的时候无需手动绑定或解析容器中的任何内容。那么，你将在什么时候手动与容器打交道呢？让我们来看看下面两种情况。

首先，如果您编写了一个实现接口的类，并希望在路由或类的构造函数上键入该接口的提示，则必须 告诉容器如何解析该接口。第二，如果您正在 编写一个 Laravel 包 计划与其他 Laravel 开发人员共享，那么您可能需要将包的服务绑定到容器中。

3.2.2. 绑定

3.2.2.1. 基础绑定

简单绑定

几乎所有的服务容器绑定都会在 服务提供者 中注册，下面示例中的大多数将演示如何在该上下文（服务提供者）中使用容器。

在服务提供者中，你总是可以通过 $this->app 属性访问容器。我们可以通过容器的 bind 方法注册绑定，bind 方法的第一个参数为要绑定的类或接口名，第二个参数是一个返回类实例的闭包：

```
use App\Services\Transistor;
```

```
use App\Services\PodcastParser;
$this->app->bind(Transistor::class, function ($app) {
    return new Transistor($app->make(PodcastParser::class));
});
```

注意，我们接受容器本身作为解析器的参数。然后，我们可以使用容器来解析正在构建的对象的子依赖。

单例的绑定

Singleton 方法绑定一个只会解析一次的类或接口到容器，然后接下来对容器的调用将会返回一个对象实例：

```
use App\Services\Transistor;
use App\Services\PodcastParser;
$this->app->singleton(Transistor::class, function ($app) {
    return new Transistor($app->make(PodcastParser::class));
});
```

绑定作用域单例

该 scoped 方法将一个类或接口绑定到容器中，该容器只应在给定的 Laravel 请求 / 作业生命周期内解析一次。虽然该方法与 singleton 方法类似，但是当 Laravel 应用程序开始一个新的 “生命周期” 时， 使用 scoped 方法注册的实例 将被刷新，例如当 Laravel Octane 工作者处理新请求或 Laravel 队列工作者处理新作业时：

```
use App\Services\Transistor;
use App\Services\PodcastParser;
$this->app->scoped(Transistor::class, function ($app) {
    return new Transistor($app->make(PodcastParser::class));
});
```

绑定实例

还可以使用 instance 方法将现有对象实例绑定到容器中。给定实例将始终在后续调用容器时返回：

```
use App\Services\Transistor;

use App\Services\PodcastParser;

$service = new Transistor(new PodcastParser);

$this->app->instance(Transistor::class, $service);
```

3.2.2.2. 绑定接口至实现

服务容器的一个非常强大的特性是它能够将接口绑定到给定的实现。例如，假设我们有一个 EventPusher 接口和一个 RedisEventPusher 实现。一旦我们对这个接口的 RedisEventPusher 实现进行了编码，我们就可以像这样在服务容器中注册它：

```
use App\Contracts\EventPusher;

use App\Services\RedisEventPusher;

$this->app->bind(EventPusher::class, RedisEventPusher::class);
```

此语句告诉容器，当类需要 EventPusher 的实现时，它应该注入 RedisEventPusher。

3.2.2.3. 上下文绑定

有时你可能有两个类使用相同的接口，但是你希望将不同的实现分别注入到各自的类中。例如，两个控制器可能依赖于 Illuminate\Contracts\Filesystem\Filesystem 契约 的不同实现。Laravel 提供了一个简单流畅的方式来定义这种行为：

```
use App\Http\Controllers\PhotoController;

use App\Http\Controllers\UploadController;
```

```
use App\Http\Controllers\VideoController;
use Illuminate\Contracts\Filesystem\Filesystem;
use Illuminate\Support\Facades\Storage;

$this->app->when(PhotoController::class)
          ->needs(Filesystem::class)
          ->give(function () {
              return Storage::disk('local');
          });

$this->app->when([VideoController::class, UploadController::class])
          ->needs(Filesystem::class)
          ->give(function () {
              return Storage::disk('s3');
          });
```

3.2.2.4. 标签

少数情况下，我们需要解析特定分类下的所有绑定，例如，你下在构建一个接收多个不同 Report 接口实现的报告聚合器，在注册完 Report 实现之后，可以通过 tag 方法给它们分配一个标签。

```
$this->app->bind('SpeedReport', function () {
    //
});
$this->app->bind('MemoryReport', function () {
```

```
    //
});
$this->app->tag(['SpeedReport', 'MemoryReport'], 'reports');
```

这些服务被打上标签后，可以通过 tagged 方法来轻松解析它们。

```
$this->app->bind('ReportAggregator', function ($app)
{
    return new ReportAggregator($app->tagged('reports'));
});
```

3.2.2.5. 继承绑定

extend 方法允许对解析服务进行修改。例如，当服务被解析后，可以运行额外代码装饰或配置该服务。extend 方法接收一个闭包来返回修改后的服务。

```
$this->app->extend(Service::class, function($service)
{
    return new DecoratedService($service);
});
```

3.2.3. 解析

3.2.3.1. Make 方法

您可以使用 make 方法从容器中解析类实例。make 方法接受您希望解析的类或接口的名称：

```
use App\Services\Transistor;

$transistor = $this->app->make(Transistor::class);
```

如果您的某些类的依赖项无法通过容器解析，您可以通过将它们作为关联数组传递给 makeWith 方法来注入它们。例如，我们可以手动传递 Transistor 服务所需的 $id 构造函数参数：

```
use App\Services\Transistor;

$transistor = $this->app->makeWith(Transistor::class, ['id' => 1]);
```

3.2.3.2. 自动注入

另外，并且更重要的是，你可以简单地使用「类型提示」的方式在类的构造函数中注入那些需要容器解析的依赖项，包括 控制器，事件监听，中间件 等 。此外，你也可以在 队列任务 的 handle 方法中使用「类型提示」注入依赖。实际上，这才是大多数对象应该被容器解析的方式。

3.2.4. 方法调用和注入

有时您可能希望调用对象实例上的方法，同时允许容器自动注入该方法的依赖项。例如，给定以下类：

```
<?php


namespace App;
use App\Repositories\UserRepository;


class UserReport
{
    /**
     * 生成新的用户报告
     *
     * @param  \App\Repositories\UserRepository  $repository
```

```
     * @return array
     */
    public function generate(UserRepository $repository)
    {
        // ...
    }
}
```

您可以通过容器调用 generate 方法，如下所示：

```
use App\UserReport;
use Illuminate\Support\Facades\App;

$report = App::call([new UserReport, 'generate']);
```

3.2.5. 容器事件

服务容器在每次解析对象时都会触发一个事件，您可以使用 resolving 方法监听该事件。

```
use App\Services\Transistor;
$this->app->resolving(Transistor::class, function ($transistor, $app)
{
    // 当容器解析 "Transistor" 类型的对象时调用...
});

$this->app->resolving(function ($object, $app) {
    // 当容器解析任何类型的对象时调用...
```

```
});
```

如您所见，正在解析的对象将被传递给回调，从而允许您在对象提供给其使用者之前设置对象的任何其他属性。

3.2.6. PSR-11

Laravel 的服务容器实现了 PSR-11 接口。因此，您可以添加 PSR-11 容器接口的类型提示来获取 Laravel 容器的实例：

```
use App\Services\Transistor;
use Psr\Container\ContainerInterface;
Route::get('/', function (ContainerInterface $container) {
    $service = $container->get(Transistor::class);
    //
});
```

如果无法解析给定的标识符，将引发异常。如果标识符从未绑定，则异常将是 Psr\Container\NotFoundExceptionInterface 的实例。如果标识符已绑定但无法解析，则将抛出 Psr\Container\ContainerExceptionInterface 的实例。

3.3. 服务提供者

3.3.1. 简介

服务提供者是所有 Laravel 应用程序的引导中心。你的应用程序，以及通过服务器引导的 Laravel 核心服务都是通过服务提供器引导。

但是，「引导」是什么意思呢？通常，我们可以理解为 注册，比如注册服务容器绑定，事件监听器，中间件，甚至是路由。服务提供者是配置应用程序的中心。

当你打开 Laravel 的 config/app.php 文件时，你会看到 providers 数组。数

组中的内容是应用程序要加载的所有服务提供者的类。当然，其中有很多「延迟」提供者，他们并不会在每次请求的时候都加载，只有他们的服务实际被需要时才会加载。

3.3.2. 编写服务提供者

所有的服务提供者都继承自 Illuminate\Support\ServiceProvider 类。大多服务提供者都包含一个 register 和一个 boot 方法。在 register 方法中，你只需要将服务绑定到 服务容器。而不要尝试在 register 方法中注册任何监听器，路由，或者其他任何功能。

使用 Artisan 命令行工具，通过 make:provider 命令可以生成一个新的提供者：

```
php artisan make:provider RiakServiceProvider
```

3.3.2.1. Register 方法

如上所述，在 register 方法中，你只需要将服务绑定到 服务容器 中。而不要尝试在 register 方法中注册任何监听器，路由，或者其他任何功能。否则，你可能会意外地使用到尚未加载的服务提供者提供的服务。

让我们来看一个基础的服务提供者。在任何服务提供者方法中，你总是通过 $app 属性来访问服务容器：

```
<?php

namespace App\Providers;

use App\Services\Riak\Connection;

use Illuminate\Support\ServiceProvider;


class RiakServiceProvider extends ServiceProvider

{
```

```
    /**
     * 注册应用服务
     *
     * @return void
     */
    public function register()
    {
        $this->app->singleton(Connection::class, function ($app) {
            return new Connection(config('riak'));
        });
    }
}
```

这个服务提供者只是定义了一个 register 方法，并且使用这个方法在服务容器中定义了一个 Riak\Connection 接口。如果你不理解服务容器的工作原理，请查看其文档。

3.3.2.2. boot 方法

如果我们想要在服务提供者中注册视图 composer 该怎么做？这就要用到 boot 方法了。该方法在所有服务提供者被注册以后才会被调用，这就是说我们可以在其中访问框架已注册的所有其他服务。

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\View;

use Illuminate\Support\ServiceProvider;
```

```
class ComposerServiceProvider extends ServiceProvider
{
    /**
     * 启动所有的应用服务
     *
     * @return void
     */
    public function boot()
    {
        View::composer('view', function () {
            //
        });
    }
}
```

Boot 方法的依赖注入

你可以为服务提供者的 boot 方法设置类型提示。服务容器 会自动注入你所需要的依赖：

```
use Illuminate\Contracts\Routing\ResponseFactory;
/**
 * 引导所有的应用服务
 *
 * @param  \Illuminate\Contracts\Routing\ResponseFactory  $response
 * @return void
 */
```

```
public function boot(ResponseFactory $response)
{
    $response->macro('serialized', function ($value) {
        //
    });
}
```

3.3.3. 注册服务提供者

所有的服务提供者都通过配置文件 config/app.php 中进行注册，该文件包含了一个列出所有服务提供者名字的 providers 数组，默认情况下，其中列出了所有核心服务提供者，这些服务提供者启动框架核心组件，比如邮件、队列、缓存等等。

要注册提供器，只需要将其添加到数组。

```
'providers' => [
    // 其它服务提供者
    App\Providers\ComposerServiceProvider::class,
],
```

3.3.4. 延迟加载服务提供者

如果你的服务提供者 只 在 服务容器 中注册，可以选择延迟加载该绑定直到注册绑定的服务真的需要时再加载，延迟加载这样的一个提供者将会提升应用的性能，因为它不会在每次请求时都从文件系统加载。

Laravel 编译并保存延迟服务提供者提供的所有服务的列表，以及其服务提供者类的名称。因此，只有当你在尝试解析其中一项服务时，Laravel 才会加载服务提供者。

3.4. 门面(Facades)

3.4.1. 简介

在整个 Laravel 文档中，您将看到通过 Facades 与 Laravel 特性交互的代码示例。Facades 为应用程序的 服务容器 中可用的类提供了「静态代理」。在 Laravel 这艘船上有许多 Facades，提供了几乎所有 Laravel 的特征。

Laravel Facades 充当服务容器中底层类的「静态代理」，提供简洁、富有表现力的好处，同时保持比传统静态方法更多的可测试性和灵活性。如果你不完全理解引擎盖下的 Facades 是如何工作的，那也没问题，跟着流程走，继续学习 Laravel。

Laravel 的所有 Facades 都在 Illuminate\Support\Facades 命名空间中定义。因此，我们可以很容易地访问这样一个 Facades：

```
use Illuminate\Support\Facades\Cache;

use Illuminate\Support\Facades\Route;

Route::get('/cache', function () {

    return Cache::get('key');

});
```

在整个框架中，很多示例使用了门面来演示框架的各种特性。

3.4.1.1. 辅助函数

为了补充 Facades，Laravel 提供了各种全局 「助手函数」，使它更容易与常见的 Laravel 功能进行交互。可以与之交互的一些常用助手函数有 view，response， url， config，等。Laravel 提供的每个助手函数都有相应的特性；但是，在专用的 辅助函数文档 中有一个完整的列表。

3.4.2. 何时使用门面

Facade 有很多好处。它们提供了一种简洁、令人难忘的语法，允许您使用

Laravel 的特性，而无需记住必须手动注入或配置的长类名。此外，由于它们对 PHP 的动态方法的独特用法，它们很容易测试：

但是，在使用 Facade 时必须小心。Facade 的主要危险是「范围溢出」。由于 Facade 非常容易使用并且不需要注入，所以让类继续增长并在单个类中使用多个 Facade 是很容易的。通过使用依赖注入，一个大型构造函数给您的视觉反馈减轻了这种可能性，即类增长过大。所以，在使用 Facade 的时候，要特别注意你的类规模，这样它的职责范围就不会太窄。如果你的类太大了，可以考虑把它分成多个较小的类。

3.4.2.1. 门面 vs.依赖注入

依赖注入的主要好处之一是能够交换注入类的实现。这在测试期间非常有用，因为您可以注入一个 mock 或 stub 并断言在 stub 上调用了各种方法。

通常，真正的静态方法是不可能 mock 或 stub 的。但是由于 Facades 使用动态方法对服务容器中解析出来的对象方法的调用进行了代理， 我们也可以像测试注入类实例一样测试 Facades。比如，像下面的路由：

```
use Illuminate\Support\Facades\Cache;

Route::get('/cache', function () {

    return Cache::get('key');

});
```

3.4.2.2. 门面 vs.辅助函数

除了门面外，Laravel 还包含各种「辅助函数」来实现这些常用功能，比如生成视图、触发事件、任务调度或者发送 HTTP 响应。许多辅助函数都有与之对应的 Facade。例如，下面这个 Facades 和辅助函数的作用是一样的：

```
Illuminate\Support\Facades\View::make('profile');

return view('profile');
```

门面和辅助函数之间没有实际的区别。 当你使用辅助函数时，你可以像测试

相应的 Facade 那样进行测试。

3.4.3. 门面工作原理

在 Laravel 应用程序中，Facades 是一个提供从容器访问对象的类。完成这项工作的部分属于 Facade 类。Laravel 的 Facade、以及您创建的任何自定义 Facade，都继承自 Illuminate\Support\Facades\Facade 类。

Facade 基类使用 __callStatic() 魔术方法将来自 Facade 的调用推迟到从容器解析出对象后。在下面的示例中，调用了 Laravel 缓存系统。

3.4.4. 实时门面

使用实时门面，你可以将应用程序中的任何类视为门面。

3.4.5. 门面类参考

序号	门面	类	服务容器绑定
1	App	Illuminate\Foundation\Application	app
2	Artisan	Illuminate\Contracts\Console\Kernel	artisan
3	Auth	Illuminate\Auth\Auth\Manager	auth
4	Auth(实例)	Illuminate\Contracts\Guard	Auth.driver
5	Blade	Illuminate\View\Compilers\BladeCompiler	Blade.compile
6	Broadcast	Illuminate\Contracts\Broadcasting\Factory	
7	Broadcast(实例)	Illuinate\Contracts\Broadcasting\Broadcaster	

8	Bus	Illuminate\Contracts\Bus\Dispatcher	
9	Cache	Illuminate\Cache\CacheManager	cache
10	Cache(实例)	Illuminate\Cache\Repository	cache.store
11	Config	Illuminate\Config\Repository	config
12	Cookie	Illuminate\Cookie\CookieJar	cookie
13	Crypt	Illuminate\Encryption\Encrypter	encrypter
14	DB	Illuminate\Database\DatabaseManager	db

第四章 基本概念

4.1. 路由

4.1.1. 路由入门

最基本的 Laravel 路由接受一个 URI 和一个闭包，提供了一个简单优雅的方法来定义路由和行为，而不需要复杂的路由配置文件：

```
use Illuminate\Support\Facades\Route;


Route::get('/greeting', function () {
    return 'Hello World';
```

```
});
```

默认路由文件

所有 Laravel 路由都定义在你的路由文件中，它位于 routes 目录。这些文件会被你的应用程序中的 App\Providers\RouteServiceProvider 自动加载。routes/web.php 文件用于定义 web 界面的路由。这些路由被分配给 web 中间件组，它提供了 SESSION 状态和 CSRF 保护等功能。定义在 routes/api.php 中的路由都是无状态的，并且被分配了 api 中间件组。

对于大多数应用程序，都是以在 routes/web.php 文件定义路由开始的。可以通过在浏览器中输入定义的路由 URL 来访问 routes/web.php 中定义的路由。例如，你可以在浏览器中输入 http://example.com/user 来访问以下路由：

```
use App\Http\Controllers\UserController;

Route::get('/user', [UserController::class, 'index']);
```

定义在 routes/api.php 文件中的路由是被 RouteServiceProvider 嵌套在一个路由组内。 在这个路由组内，将自动应用 /api URI 前缀，所以你无需手动将其应用于文件中的每个路由。你可以通过修改 RouteServiceProvider 类来修改前缀和其他路由组选项。

可用的路由方法

路由器允许你注册能响应任何 HTTP 请求的路由：

```
Route::get($uri, $callback);

Route::post($uri, $callback);

Route::put($uri, $callback);

Route::patch($uri, $callback);

Route::delete($uri, $callback);

Route::options($uri, $callback);
```

4.1.1.1. 路由重定向

如果要定义重定向到另一个 URI 的路由，可以使用 Route::redirect 方法。这个方法可以快速的实现重定向，而不再需要去定义完整的路由或者控制器：

```
Route::redirect('/here', '/there');
```

默认情况下，Route::redirect 返回 302 状态码。你可以使用可选的第三个参数来定制状态码：

```
Route::redirect('/here', '/there', 301);
```

您还可以使用 Route::permanentRedirect 方法来返回 301 状态码。

```
Route::permanentRedirect('/here', '/there');
```

4.1.1.2. 路由视图

如果你的路由只需要返回一个 视图，可以使用 Route::view 方法。它和 redirect 一样方便，不需要定义完整的路由或控制器。view 方法有三个参数，其中前两个是必填参数，分别是 URI 和视图名称。第三个参数选填，可以传入一个数组，数组中的数据会被传递给视图：

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => '欢迎']);
```

4.1.2. 路由参数

4.1.2.1. 必填参数

有时你将需要捕获路由内的 URI 段。例如，你可能需要从 URL 中捕获用户的 ID。你可以通过定义路由参数来做到这一点：

```
Route::get('user/{id}', function ($id) {

return 'User ' . $id;

});
```

也可以根据你的需要在路由中定义多个参数：

```
Route::get('/posts/{post}/comments/{comment}', function ($postId,
$commentId) {
    //
});
```

路由的参数通常都会被放在 {} ，并且参数名只能为字母。下划线 (_) 也可以用于路由参数名中。路由参数会按路由定义的顺序依次注入到路由回调或者控制器中，而不受回调或者控制器的参数名称的影响。

参数和依赖注入

如果你的路由具有依赖关系，而你希望 Laravel 服务容器自动注入到路由的回调中，则应在依赖关系之后列出路由参数：

```
use Illuminate\Http\Request;
Route::get('/user/{id}', function (Request $request, $id) {
    return 'User '.$id;
});
```

4.1.2.2. 可选参数

有时，你可能需要指定一个路由参数，但你希望这个参数是可选的。你可以在参数后面加上 ？ 标记来实现，但前提是要确保路由的相应变量有默认值：

```
Route::get('user/{name?}', function ($name = null) {
    return $name;
});
Route::get('user/{name?}', function ($name = 'John')
{
    return $name;
```

```
});
```

4.1.2.3. 正则约束

你可以使用路由实例上的 where 方法来限制路由参数的格式。 where 方法接受参数的名称和定义如何约束参数的正则表达式：

```
Route::get('user/{name}', function ($name) {
    //
})->where('name', '[A-Za-z]+');

Route::get('user/{id}', function ($id) {
    //
})->where('id', '[0-9]+');

Route::get('user/{id}/{name}', function ($id, $name) {
    //
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

为方便起见，一些常用的正则表达式模式具有帮助方法，可让你快速将模式约束添加到路由：

```
Route::get('/user/{id}/{name}', function ($id, $name) {
    //
})->whereNumber('id')->whereAlpha('name');

Route::get('/user/{name}', function ($name) {
    //
```

```
})->whereAlphaNumeric('name');


Route::get('/user/{id}', function ($id) {
    //
})->whereUuid('id');
```

如果传入的请求与路由模式约束不匹配，将返回 404 HTTP 响应。

4.1.2.4. 命名路由

命名路由允许为特定路由方便地生成 URL 或重定向。通过将 name 方法链接到路由定义上，可以指定路由的名称：

```
Route::get('/user/profile', function () {
    //
})->name('profile');
```

你还可以为控制器操作指定路由名称：

```
Route::get(
    '/user/profile',
    [UserProfileController::class, 'show']
)->name('profile');
```

生成命名路由的 URL

一旦你为给定的路由分配了一个名字，你可以在通过 Laravel 的 route 和 redirect 辅助函数生成 URL 或重定向时使用该路由的名称：

```
// 正在生成 URL...
$url = route('profile');


// 正在生成重定向...
```

```
return redirect()->route('profile');
```

如果命名路由定义了参数，你可以将参数作为第二个参数传递给 route 函数。给定的参数将自动插入到生成的 URL 的正确位置：

```
Route::get('/user/{id}/profile', function ($id) {
    //
})->name('profile');
$url = route('profile', ['id' => 1]);
```

如果你在数组中传递其他参数，这些键/值对将自动添加到生成的 URL 的查询字符串中：

```
Route::get('/user/{id}/profile', function ($id) {
    //
})->name('profile');
$url = route('profile', ['id' => 1, 'photos' => 'yes']);
// /user/1/profile?photos=yes
```

4.1.3. 路由分组

路由组允许你共享路由属性，例如中间件，而无需在每个单独的路由上定义这些属性。

嵌套组尝试智能地将属性与其父组「合并」。中间件和 where 条件合并，同时附加名称和前缀。 URI 前缀中的命名空间分隔符和斜杠会在适当的地方自动添加。

4.1.3.1. 路由中间件

要将 middleware 分配给组内的所有路由，你可以在定义组之前使用 middleware 方法。 中间件按照它们在数组中列出的顺序执行：

```
Route::middleware(['first', 'second'])->group(functio
```

```
n () {
Route::get('/', function () {
// Uses first & second Middleware
});
Route::get('user/profile', function () {
// Uses first & second Middleware
});
});
```

4.1.3.2. 控制器

如果一组路由都使用相同的控制器，你可以使用 controller 方法为组内的所有路由定义公共控制器。然后，在定义路由时，你只需要提供它们调用的控制器方法：

```
use App\Http\Controllers\OrderController;
Route::controller(OrderController::class)->group(function () {
    Route::get('/orders/{id}', 'show');
    Route::post('/orders', 'store');
});
```

4.1.3.3. 子域路由

路由组也可以用来处理子域路由。子域可以像路由 uri 一样被分配路由参数，允许你捕获子域的一部分以便在路由或控制器中使用。子域可以在定义组之前调用 domain 方法来指定:

```
Route::domain('{account}.example.com')->group(function () {
```

```
    Route::get('user/{id}', function ($account, $id) {
        //
    });
});
```

4.1.3.4. 路由前缀

prefix 方法可以可以用给定的 URI 为组中的每个路由做前缀。例如，你可能想要在组内的所有路由 uri 前面加上 admin 前缀：

```
Route::prefix('admin')->group(function () {
    Route::get('users', function () {
        // Matches The "/admin/users" URL
    });
});
```

4.1.3.5. 路由名称前缀

name 方法可以用给定字符串作为组中的每个路由名的前缀。例如，你可能想要用 admin 作为所有分组路由的前缀。因为给定字符串的前缀与指定的路由名完全一致，所以我们一定要提供末尾 . 字符在前缀中：

```
Route::name('admin.')->group(function () {
    Route::get('/users', function () {
        // Route assigned name "admin.users"...
    })->name('users');
});
```

4.1.4. 路由模型绑定

将模型 ID 注入到路由或控制器操作时，你通常会查询数据库以检索与该 ID 对应的模型。Laravel 路由模型绑定提供了一种方便的方法来自动将模型实例直接注入到你的路由中。例如，你可以注入与给定 ID 匹配的整个 User 模型实例，而不是注入用户的 ID。

4.1.4.1. 隐式绑定

Laravel 自动解析定义在路由或控制器操作中的 Eloquent 模型，其类型提示的变量名称与路由段名称匹配。例如：

```
use App\Models\User;

Route::get('/users/{user}', function (User $user) {
    return $user->email;
});
```

由于 $user 变量被类型提示为 App\Models\User Eloquent 模型，并且变量名称与 {user} URI 段匹配，Laravel 将自动注入 ID 匹配相应的模型实例 来自请求 URI 的值。如果在数据库中没有找到匹配的模型实例，将自动生成 404 HTTP 响应。

当然，使用控制器方法时也可以使用隐式绑定。同样，请注意 {user} URI 段与控制器中的 $user 变量匹配，该变量包含 App\Models\User 类型提示：

```
use App\Http\Controllers\UserController;
use App\Models\User;


// Route definition...
Route::get('/users/{user}', [UserController::class, 'show']);


// Controller method definition...
```

```
public function show(User $user)
{
    return view('user.profile', ['user' => $user]);
}
```

软删除模型

通常，隐式模型绑定不会检索已 软删除 的模型。但是，你可以通过将 withTrashed 方法链接到你的路由定义来指示隐式绑定来检索这些模型：

```
use App\Models\User;
Route::get('/users/{user}', function (User $user) {
    return $user->email;
})->withTrashed();
```

4.1.4.2. 隐式枚举绑定

PHP 8.1 引入了对 Enums 的支持。为了补充这个特性，Laravel 允许你在你的路由定义中键入一个 Enum，并且 Laravel 只会在该路由段对应于一个有效的 Enum 值时调用该路由。否则，将自动返回 404 HTTP 响应。例如，给定以下枚举：

```
<?php
namespace App\Enums;

enum Category: string
{
    case Fruits = 'fruits';
    case People = 'people';
```

```
}
```

你可以定义一个只有在 {category} 路由段是 fruits 或 people 时才会被调用的路由。 否则，Laravel 将返回 404 HTTP 响应：

```
use App\Enums\Category;
use Illuminate\Support\Facades\Route;

Route::get('/categories/{category}', function (Category $category) {
    return $category->value;
});
```

4.1.4.3. 显式绑定

你不需要使用 Laravel 隐式的、基于约定的模型解析来使用模型绑定。你还可以显式定义路由参数与模型的对应方式。要注册显式绑定，请使用路由器的“模型”方法为给定参数指定类。你应该在 RouteServiceProvider 类的 boot 方法的开头定义显式模型绑定：

```
use App\Models\User;
use Illuminate\Support\Facades\Route;

/**
 * Define your route model bindings, pattern filters, etc.
 *
 * @return void
 */
public function boot()
```

```
{
    Route::model('user', User::class);

    // ...
}
```

接下来，定义一个包含 {user} 参数的路由：

```
use App\Models\User;

Route::get('/users/{user}', function (User $user) {
    //
});
```

由于我们已将所有 {user} 参数绑定到 App\Models\User 模型，该类的一个实例将被注入到路由中。 因此，例如，对 users/1 的请求将从 ID 为 1 的数据库中注入 User 实例。

如果在数据库中没有找到匹配的模型实例，则会自动生成 404 HTTP 响应。

4.1.5. fallback 路由

使用 Route::fallback 方法，你可以定义一个在没有其他路由匹配传入请求时将执行的路由。 通常，未处理的请求将通过应用程序的异常处理程序自动呈现「404」页面。但是，由于你通常会在 routes/web.php 文件中定义 fallback 路由，因此 web 中间件组中的所有中间件都将应用于该路由。你可以根据需要随意向此路由添加额外的中间件：

```
Route::fallback(function () {
    //
});
```

4.1.6. 速率限制

4.1.6.1. 定义速率限制器

Laravel 包含强大且可自定义的速率限制服务，你可以利用这些服务来限制给定路由或一组路由的流量。首先，你应该定义满足应用程序需求的速率限制器配置。通常，这应该在应用程序的 App\Providers\RouteServiceProvider 类的 configureRateLimiting 方法中完成。

速率限制器是使用 RateLimiter 外观的 for 方法定义的。 for 方法接受一个速率限制器名称和一个闭包，该闭包返回应该应用于分配给速率限制器的路由的限制配置。限制配置是 Illuminate\Cache\RateLimiting\Limit 类的实例。此类包含有用的「构建器」方法，以便你可以快速定义限制。速率限制器名称可以是你希望的任何字符串：

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;


/**
 * Configure the rate limiters for the application.
 *
 * @return void
 */
protected function configureRateLimiting()
{
    RateLimiter::for('global', function (Request $request) {
        return Limit::perMinute(1000);
    });
}
```

如果传入的请求超过指定的速率限制，Laravel 将自动返回一个带有 429 HTTP 状态码的响应。如果你想定义自己的响应，应该由速率限制返回，你可以使用 response 方法：

```
RateLimiter::for('global', function (Request $request) {
    return Limit::perMinute(1000)->response(function () {
        return response('Custom response...', 429);
    });
});
```

4.1.6.2. 将速率限制器附加到路由

可以使用 throttle middleware 将速率限制器附加到路由或路由组。路由中间件接受你希望分配给路由的速率限制器的名称：

```
Route::middleware(['throttle:uploads'])->group(function () {
    Route::post('/audio', function () {
        //
    });

    Route::post('/video', function () {
        //
    });
});
```

4.1.7. 伪造表单方法

HTML 表单不支持「PUT」，「PATCH」或「DELETE」请求。所以，当定

义「PUT」，「PATCH」或「DELETE」路由用在 HTML 表单时，你将需要一个隐藏的加「_method」字段在表单中。该「_method」字段的值将会与 HTTP 请求一起发送。

4.1.8. 访问当前路由

您可以使用 Route Facade 的 current 、currentRouteName 和 currentRouteAction 方法来访问有关处理传入请求的路由的信息：

```
use Illuminate\Support\Facades\Route;



$route = Route::current(); // Illuminate\Routing\Route

$name = Route::currentRouteName(); // string

$action = Route::currentRouteAction(); // string
```

4.1.9. 跨域资源共享(CORS)

Laravel 可以使用你配置的值自动响应 CORS OPTIONS HTTP 请求。所有 CORS 设置都可以在应用程序的 config/cors.php 配置文件中进行配置。OPTIONS 请求将由默认包含在全局中间件堆栈中的 HandleCors 中间件 自动处理。

4.1.10. 路由缓存

在将应用程序部署到生产环境时，你应该利用 Laravel 的路由缓存。使用路由缓存将大大减少注册所有应用程序路由所需的时间。要生成路由缓存，请执行 route:cache Artisan 命令：

```
php artisan route:cache
```

4.2. 中间件

4.2.1. 简介

中间件提供了一种方便的机制来检查和过滤进入应用程序的 HTTP 请求。例如，Laravel 包含一个中间件，用于验证您的应用程序的用户是否经过身份验证。 如果用户未通过身份验证，中间件会将用户重定向到应用程序的登录屏幕。但是，如果用户通过了身份验证，中间件将允许请求进一步进入应用程序。

除了身份验证之外，还可以编写其他中间件来执行各种任务。例如，日志中间件可能会将所有传入请求记录到您的应用程序。Laravel 框架中包含了几个中间件，包括用于身份验证和 CSRF 保护的中间件。 所有这些中间件都位于 app/Http/Middleware 目录中。

4.2.2. 定义中间件

要创建一个中间件，请使用 make:middleware Artisan 命令：

```
php artisan make:middleware EnsureTokenIsValid
```

这个命令会在 app/Http/Middleware 目录下创建一个新的中间件类。在这个中间件中，如果提供的 token 输入匹配指定的值，我们将只允许访问路由。否则，我们会将用户重定向回 home URI：

```
<?php

namespace App\Http\Middleware;


use Closure;


class EnsureTokenIsValid
{
    /**
```

```
     * Handle an incoming request.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @return mixed
     */
    public function handle($request, Closure $next)
    {
        if ($request->input('token') !== 'my-secret-token') {
            return redirect('home');
        }

        return $next($request);
    }
}
```

如您所见，如果给定的 token 与我们的秘密令牌不匹配，中间件将向客户端返回 HTTP 重定向； 否则，请求将被进一步传递到应用程序中。要将请求更深入地传递到应用程序中（允许中间件「通过」），您应该使用 $request 调用 $next 回调。

4.2.3. 注册中间件

4.2.3.1. 全局中间件

如果您希望在对应用程序的每个 HTTP 请求期间运行中间件，请在 app/Http/Kernel.php 类的 $middleware 属性中列出中间件类。

4.2.3.2. 指定路由中间件

如果您想将中间件分配给特定的路由，您应该首先在应用程序的 app/Http/Kernel.php 文件中为中间件分配一个键。默认情况下，该类的 $routeMiddleware 属性包含 Laravel 中包含的中间件的条目。 您可以将自己的中间件添加到此列表中，并为其分配您选择的键：

```
// 在 App\Http\Kernel 类中...

protected $routeMiddleware = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'auth.basic' =>
\Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'bindings' =>
\Illuminate\Routing\Middleware\SubstituteBindings::class,
    'cache.headers' =>
\Illuminate\Http\Middleware\SetCacheHeaders::class,
    'can' => \Illuminate\Auth\Middleware\Authorize::class,
    'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,
    'signed' =>
\Illuminate\Routing\Middleware\ValidateSignature::class,
    'throttle' =>
\Illuminate\Routing\Middleware\ThrottleRequests::class,
    'verified' =>
\Illuminate\Auth\Middleware\EnsureEmailIsVerified::class,
];
```

一旦在 HTTP 内核中定义了中间件，您就可以使用 middleware 方法将中间件分配给路由：

```
Route::get('/profile', function () {
    //
```

```
})->middleware('auth');
```

4.2.3.3. 中间件组

有时您可能希望将多个中间件组合在一个键下，以使它们更容易分配给路由。您可以使用 HTTP 内核的 $middlewareGroups 属性来完成此操作。

4.2.3.4. 排序中间件

很少，您可能需要您的中间件按特定顺序执行，但在将它们分配给路由时无法控制它们的顺序。 在这种情况下，您可以使用 app/Http/Kernel.php 文件的 $middlewarePriority 属性指定中间件优先级。

4.2.4. 中间件参数

中间件也可以接收额外的参数。例如，如果您的应用程序需要在执行给定操作之前验证经过身份验证的用户是否具有给定的「角色」，您可以创建一个 EnsureUserHasRole 中间件，该中间件接收角色名称作为附加参数。

额外的中间件参数将在 $next 参数之后传递给中间件：

```
use Closure;

class EnsureUserHasRole
{
    /**
     * 处理传入的请求。
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @param  string  $role
     * @return mixed
```

```
     */
    public function handle($request, Closure $next, $role)
    {
        if (! $request->user()->hasRole($role)) {
            // 重定向...
        }

        return $next($request);
    }

}
```

在定义路由时，可以指定中间件参数，方法是使用 : 分隔中间件名称和参数。多个参数应以逗号分隔：

```
Route::put('/post/{id}', function ($id) {
    //
})->middleware('role:editor');
```

4.2.5. 可终止的中间件

有时，在将 HTTP 响应发送到浏览器之后，中间件可能需要做一些工作。如果您在中间件上定义了一个 terminate 方法，并且您的 Web 服务器使用 FastCGI，则在将响应发送到浏览器后会自动调用 terminate 方法：

```
<?php

namespace Illuminate\Session\Middleware;

use Closure;
```

```
class TerminatingMiddleware
{
    /**
     * 处理传入的请求。
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @return mixed
     */
    public function handle($request, Closure $next)
    {
        return $next($request);
    }

    /**
     * 在响应发送到浏览器后处理任务。
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Illuminate\Http\Response  $response
     * @return void
     */
    public function terminate($request, $response)
    {
```

```
        // ...
    }
}
```

terminate 方法应该同时接收请求和响应。一旦你定义了一个可终止的中间件，你应该将它添加到 app/Http/Kernel.php 文件中的路由或全局中间件列表中。

当在中间件上调用 terminate 方法时，Laravel 会从 服务容器 解析一个新的中间件实例。如果您想在调用 handle 和 terminate 方法时使用相同的中间件实例，请使用容器的 singleton 方法向容器注册中间件。 通常这应该在你的 AppServiceProvider 的 register 方法中完成：

```
use App\Http\Middleware\TerminatingMiddleware;


/**
 * 注册任何应用程序服务。
 *
 * @return void
 */
public function register()
{
    $this->app->singleton(TerminatingMiddleware::class);
}
```

4.3. CSRF 保护

4.3.1. 简介

跨站点请求伪造是一种恶意利用，利用这种手段，代表经过身份验证的用户

执行未经授权的命令。值得庆幸的是，Laravel 可以轻松保护您的应用程序免受跨站点请求伪造（CSRF）攻击。

4.3.2. 阻止 CSRF 请求

Laravel 为应用程序管理的每个活动 用户会话 自动生成 CSRF 「令牌」。此令牌用于验证经过身份验证的用户是实际向应用程序发出请求的人。由于此令牌存储在用户的会话中，并且每次重新生成会话时都会更改，因此恶意应用程序将无法访问它。

4.3.3. X-CSRF-TOKEN

除了检查 CSRF 令牌作为 POST 参数外，App\Http\Middleware\VerifyCsrfToken 中间件还将检查 X-CSRF-TOKEN 请求标头。

4.3.4. X-XSRF-TOKEN

Laravel 将当前的 CSRF 令牌存储在一个加密的 XSRF-TOKEN cookie 中，该 cookie 包含在框架生成的每个响应中。您可以使用 cookie 值来设置 X-XSRF-TOKEN 请求标头。

这个 cookie 主要是为了方便开发人员发送的，因为一些 JavaScript 框架和库，如 Angular 和 Axios，会自动将其值放在同源请求的 X-XSRF-TOKEN 标头中。

4.4. 控制器

4.4.1. 简介

您可能希望使用 “控制器” 类来组织此行为，而不是将所有请求处理逻辑定义为路由文件中的闭包。控制器可以将相关的请求处理逻辑分组到一个类中。例如，一个 UserController 类可能会处理所有与用户相关的传入请求，包括显示、创建、更新和删除用户。 默认情况下，控制器存储在 app/Http/Controllers 目录中。

4.4.2. 编写控制器

4.4.2.1. 基本控制器

让我们看一个基本控制器的例子。 请注意，控制器扩展了 Laravel 中包含的基本控制器类 App\Http\Controllers\Controller：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use App\Models\User;


class UserController extends Controller
{
    /**
     * 显示给定用户的个人资料。
     *
     * @param  int  $id
     * @return \Illuminate\View\View
     */
    public function show($id)
    {
        return view('user.profile', [
            'user' => User::findOrFail($id)
        ]);
    }
}
```

您可以像这样定义此控制器方法的路由：

```
use App\Http\Controllers\UserController;


Route::get('/user/{id}', [UserController::class, 'show']);
```

当传入的请求与指定的路由 URI 匹配时，将调用 App\Http\Controllers\UserController 类的 show 方法，并将路由参数传递给该方法。

4.4.2.2. 单一动作控制器

如果控制器动作特别复杂，您可能会发现将整个控制器类专用于该单个动作很方便。为此，您可以在控制器中定义一个 __invoke 方法：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use App\Models\User;


class ProvisionServer extends Controller
{
    /**
     * 配置新的 Web 服务器。
     *
     * @return \Illuminate\Http\Response
     */
    public function __invoke()
    {
```

```
        // ...
    }
}
```

为单动作控制器注册路由时，不需要指定控制器方法。相反，您可以简单地将控制器的名称传递给路由器：

```
use App\Http\Controllers\ProvisionServer;
erver', ProvisionServer::class);
```

4.4.3. 控制器中间件

中间件 可以在您的路由文件中分配给控制器的路由：

```
Route::get('profile', [UserController::class, 'show'])-
>middleware('auth');
```

4.4.4. 资源控制器

如果你将应用程序中的每个 Eloquent 模型都视为资源，那么通常对应用程序中的每个资源都执行相同的操作。例如，假设你的应用程序中包含一个 Photo 模型和一个 Movie 模型。用户可能可以创建，读取，更新或者删除这些资源。

4.4.4.1. 部分资源路由

当声明资源路由时，你可以指定控制器处理的部分行为，而不是所有默认的行为。

```
Route::resource('photos', 'PhotoController')->only([
    'index', 'show'
]);


Route::resource('photos', 'PhotoController')->except([
```

```
    'create', 'store', 'update', 'destroy'
]);
```

4.4.4.2. 嵌套资源

有时可能需要定义一个嵌套的资源型路由。例如，照片资源可能被添加了多个评论。那么可以在路由中使用 . 符号来声明资源型控制器：

```
use App\Http\Controllers\PhotoCommentController;

Route::resource('photos.comments', PhotoCommentController::class);
```

4.4.4.3. 命名资源路由

默认情况下，所有的资源控制器行为都有一个路由名称。你可以传入 names 数组来覆盖这些名称：

```
use App\Http\Controllers\PhotoController;

Route::resource('photos', PhotoController::class)->names([
    'create' => 'photos.build'
]);
```

4.4.4.4. 命名资源路由参数

默认情况下，Route::resource 会根据资源名称的「单数」形式创建资源路由的路由参数。你可以使用 parameters 方法来轻松地覆盖资源路由名称。传入 parameters 方法应该是资源名称和参数名称的关联数组：

```
use App\Http\Controllers\AdminUserController;

Route::resource('users', AdminUserController::class)->parameters([
    'users' => 'admin_user'
```

```
]);
```

上面的示例将会为资源的 show 路由生成以下的 URL：

```
/users/{admin_user}
```

4.4.4.5. 限制范围的资源路由

Laravel 的 作用域隐式模型绑定 功能可以自动确定嵌套绑定的范围，以便确认已解析的子模型属于父模型。通过在定义嵌套资源时使用 scoped 方法，你可以启用自动范围界定，并指示 Laravel 应该通过以下方式来检索子资源的哪个字段：

```
use App\Http\Controllers\PhotoCommentController;

Route::resource('photos.comments', PhotoCommentController::class)->scoped([
    'comment' => 'slug',
]);
```

4.4.4.6. 本地化资源 URIs

默认情况下，Route::resource 将会用英文动词创建资源 URIs。如果需要自定义 create 和 edit 行为的动名词，你可以在 App\Providers\RouteServiceProvider 的 boot 方法中使用 Route::resourceVerbs 方法实现：

```
/**
 * 定义你的路由模型绑定，模式过滤器等
 *
 * @return void
 */
public function boot()
```

```
{
    Route::resourceVerbs([
        'create' => 'crear',
        'edit' => 'editar',
    ]);

    // ...
}
```

4.4.4.7. 补充资源控制器

如果您需要向资源控制器添加超出默认资源路由集的其他路由，则应在调用 Route::resource 方法之前定义这些路由；否则，由 resource 方法定义的路由可能会无意中优先于您的补充路由：

```
use App\Http\Controller\PhotoController;
Route::get('/photos/popular', [PhotoController::class, 'popular']);
Route::resource('photos', PhotoController::class);
```

4.4.5. 依赖注入&控制器

4.4.5.1. 构造函数注入

Laravel 服务容器 用于解析所有 Laravel 控制器。因此，您可以在其构造函数中对控制器可能需要的任何依赖项进行类型提示。声明的依赖项将自动解析并注入到控制器实例中：

```
namespace App\Http\Controllers;
```

```
use App\Repositories\UserRepository;


class UserController extends Controller
{
    /**
     * 用户存储库实例。
     */
    protected $users;


    /**
     * 创建一个新的控制器实例。
     *
     * @param  \App\Repositories\UserRepository  $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }
}
```

4.4.5.2. 方法注入

除了构造函数注入之外，您还可以类型提示依赖于控制器的方法。方法注入的一个常见用例是将 Illuminate\Http\Request 实例注入到控制器方法中：

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;


class UserController extends Controller
{
    /**
     * 存储一个新用户。
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        $name = $request->name;


        //
    }
}
```

如果你的控制器方法要从路由参数中获取输入内容，请在你的依赖项之后列出你的路由参数。例如，你可以像下方这样定义路由：

```
use App\Http\Controllers\UserController;

Route::put('/user/{id}', [UserController::class, 'update']);
```

4.5. HTTP 请求

4.5.1. 简介

Laravel 的 Illuminate\Http\Request 类提供了一种面向对象的方法，可以与应用程序处理的当前 HTTP 请求进行交互，以及检索与请求一起提交的输入内容，cookies 和文件。

4.5.2. 与请求交互

4.5.2.1. 访问请求

要通过依赖注入获得当前 HTTP 请求的实例，您应该在路由闭包或控制器方法上导入 Illuminate\Http\Request 类。 传入的请求实例将由 Laravel 服务容器 自动注入：

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;


class UserController extends Controller
{
    /**
     * 存储一个新用户.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
```

```
        $name = $request->input('name');


        //
    }
}
```

如上所述，你也可以在路由闭包上导入 Illuminate\Http\Request 类。服务容器在执行时将自动传入请求注入到闭包中：

```
use Illuminate\Http\Request;

Route::get('/', function (Request $request) {
    //
});
```

4.5.2.2. 请求路径和方法

Illuminate\Http\Request 实例提供了多种方法来检查传入的 Http 请求，并扩展了 Symfony\Component\HttpFoundation\Request 类。下面我们将讨论几个最重要的方法。

4.5.2.2.1. 检索请求路径

path 方法返回请求的路径信息。因此，如果传入请求的目标是 http://example.com/foo/bar，则 path 方法将返回 foo/bar：

```
$uri = $request->path();
```

4.5.2.2.2. 检索请求路径/路由

is 方法允许您验证传入的请求路径是否与给定的模式匹配。使用此方法时，可以使用 * 字符作为通配符：

```
if ($request->is('admin/*')) {
```

```
    //
}
```

4.5.2.2.3. 检索请求 URL

要检索传入请求的完整 URL，你可以使用 url 或 fullUrl 方法。url 方法将返回不包含查询字符串的 URL，而 fullUrl 方法包含查询字符串：

```
$url = $request->url();

$urlWithQueryString = $request->fullUrl();
```

4.5.2.3. 请求头

你可以使用 header 方法从 Illuminate\Http\Request 实例中检索一个请求头。如果请求中不存在该头，将返回 null。然而，header 方法接受一个可选的第二个参数，如果请求中不存在该头，将返回该参数：

```
$value = $request->header('X-Header-Name');

$value = $request->header('X-Header-Name', 'default');
```

4.5.2.4. 请求 IP 地址

ip 方法可用来检索向你的应用程序发出请求的客户机的 IP 地址：

```
$ipAddress = $request->ip();
```

4.5.2.5. 请求协商

Laravel 提供一些通过 Accept 头检查传入请求的请求内容类型的方法。首先，getAcceptableContentTypes 方法将返回一个包含通过请求接受的所有内容类型的数组：

```
$contentTypes = $request->getAcceptableContentTypes();
```

4.5.2.6. PSR-7 Requests

PSR-7 标准 指定 HTTP 消息的接口，包括请求和响应。如果要获取 PSR-7 请求的实例而不是 Laravel 请求，则首先需要安装一些库。Laravel 使用 Symfony HTTP Message Bridge 组件将典型的 Laravel 请求和响应转换为 PSR-7 兼容的实现：

```
composer require symfony/psr-http-message-bridge

composer require nyholm/psr7
```

4.5.3. 输入

4.5.3.1. 检索输入

4.5.3.1.1. 检索所有输入数据

可以使用 all 方法以 array 的形式检索所有传入请求的输入数据。无论传入的请求是来自 HTML 表单还是 XHR 请求，都可以使用此方法。

```
$input = $request->all();
```

使用 collect 方法，您可以将所有传入请求的输入数据检索为 collection：

```
$input = $request->collect();
```

4.5.3.2. 判断输入是否存在

你可以使用 has 来判断当前请求中是否含有指定的值。如果请求中存在该值则 has 方法将会返回 true：

```
if ($request->has('name')) {
    //
}
```

4.5.3.3. 合并附加输入

有时您可能需要手动将其他输入合并到请求的现有输入数据中。为此，您可以使用 merge 方法：

```
$request->merge(['votes' => 0]);
```

4.5.3.4. 旧数据

Laravel 允许你在两次请求之间保持数据。这个特性在有效性校验出错后重新填充表单时非常有用。不过，如果你使用 Laravel 自带的 表单验证，不需要自己手动调用这些方法，因为一些 Laravel 内置的验证功能会自动调用它们。

4.5.4. 输入过滤&规范化

默认情况下，Laravel 在应用程序的全局中间件堆栈中包含 App\Http\Middleware\TrimStrings 和 App\Http\Middleware\ConvertEmptyStringsToNull 中间件。 这些中间件在 App\Http\Kernel 类的全局中间件堆栈中列出。 这些中间件将自动修剪请求中的所有传入字符串字段，并将所有空字符串字段转换为 null。 这使您不必担心路由和控制器中的这些规范化问题。

如果您想禁用此行为，则可以通过从 App\Http\Kernel 类的 $middleware 属性中删除这两个中间件，将它们从应用程序的中间件堆栈中删除。

4.5.5. 文件

4.5.5.1. 获取上传的文件

您可以使用 file 方法或使用动态属性从 Illuminate\Http\Request 实例中访问上传的文件。 该 file 方法返回 Illuminate\Http\UploadedFile 的实例，该类继承了 PHP 的 SplFileInfo 类的同时也提供了各种与文件交互的方法：

```
$file = $request->file('photo');

$file = $request->photo;
```

4.5.5.2. 存储上传的文件

要存储上传的文件，通常会使用已配置的 文件系统。你可以使用 UploadedFile 的 store 方法把上传文件移动到你的某个磁盘上，该文件可能是本地文件系统中的一个位置，甚至像 Amazon S3 这样的云存储位置。

store 方法接受相对于文件系统配置的根目录应存储文件的路径。 该路径不能包含文件名，因为将自动生成一个唯一的 ID 作为文件名。

store 方法还接受一个可选的第二个参数作为应用于存储文件的磁盘的名称。该方法将返回文件相对于磁盘根目录的路径：

```
$path = $request->photo->store('images');

$path = $request->photo->store('images', 's3');
```

4.5.6. 配置受信任的代理

在终止 TLS / SSL 证书的负载平衡器后面运行应用程序时，您可能会注意到，使用 url 帮助程序时，应用程序有时不会生成 HTTPS 链接。通常，这是因为正在从端口 80 上的负载平衡器转发应用程序的流量，并且不知道它应该生成安全链接。

4.6. HTTP 响应

4.6.1. 创建响应

所有路由和控制器处理完业务逻辑之后都会返回响应到用户的浏览器，Laravel 提供了多种不同的响应方式，其中最基本就是从路由或控制器返回一个简单的字符串，框架会自动将这个字符串转化为一个完整的 HTTP 响应：

```
Route::get('/', function () {

    return 'Hello World';
```

```
});
```

除了从路由或控制器返回字符串之外，还可以返回数组。框架会自动将数组转化为一个 JSON 响应。

```
Route::get('/', function () {
    return [1, 2, 3];
});
```

4.6.1.1. 在响应中附加 header 信息

请记住，大多数响应方法都是可以链式调用的，它允许你流畅地构建响应实例。例如，在将响应发送回用户之前，可以使用 header 方法将一系列头添加到响应中：

```
return response($content)
            ->header('Content-Type', $type)
            ->header('X-Header-One', 'Header Value')
            ->header('X-Header-Two', 'Header Value');
```

4.6.1.2. 在响应中附加 Cookies 信息

可以使用 cookie 方法将 cookie 附加到传出的 illumize\Http\Response 实例。你应将 cookie 的名称、值和有效分钟数传递给此方法：

```
return response('Hello World')->cookie(
    'name', 'value', $minutes
);
```

4.6.1.3. Cookies & 加密

默认情况下，由 Laravel 生成的所有 cookie 都经过了加密和签名，因此客户端无法篡改或读取它们。如果要对应用程序生成的部分 cookie 禁用加密，可以使用 App\Http\Middleware\EncryptCookies 中间件的 $except 属性，该属性位于 app/Http/Middleware 目录中：

```
/**
 * 这个名字的 Cookie 将不会加密
 *
 * @var array
 */
protected $except = [
    'cookie_name',
];
```

4.6.2. 重定向

重定向响应是 Illuminate\Http\RedirectResponse 类的实例，包含将用户重定向到另一个 URL 所需的适当 HTTP 头。Laravel 有几种方法可以生成 RedirectResponse 实例。最简单的方法是使用全局 redirect 助手函数：

```
Route::get('dashboard', function () {
    return redirect('home/dashboard');
});
```

4.6.2.1. 重定向到指定名称的路由

当你在没有传递参数的情况下调用 redirect 助手函数时，将返回 Illuminate\Routing\Redirector 的实例，允许你调用 Redirector 实例上的任何方法。例如，要对命名路由生成 RedirectResponse ，可以使用 route 方法：

```
return redirect()->route('login');
```

4.6.2.2. 重定向到控制器行为

也可以生成重定向到 controller actions。只要把控制器和 action 的名称传递给 action 方法：

```
use App\Http\Controllers\UserController;

return redirect()->action([UserController::class, 'index']);
```

4.6.2.3. 重定向到外部域名

有有时候你需要重定向到应用外的域名。可以通过调用 away 方法，它会创建一个不带有任何额外的 URL 编码、有效性校验和检查 RedirectResponse 实例：

```
return redirect()->away('https://www.google.com');
```

4.6.2.4. 重定向并使用闪存的 Session 数据

重定向到新的 URL 的同时 传送数据给 seesion 是很常见的。 通常这是在你将消息发送到 session 后成功执行操作后完成的。

4.6.3. 其他响应类型

response 助手可以用于生成其他类型的响应实例。

4.6.3.1. 视图响应

如果您需要控制响应的状态和标头，但还需要返回 view 作为响应的内容，您应该使用 view 方法：

```
return response()
            ->view('hello', $data, 200)
            ->header('Content-Type', $type);
```

当然，如果您不需要传递自定义 HTTP 状态代码或自定义标头，则可以使用全局 view 辅助函数。

4.6.3.2. JSON 响应

json 自动将 Content-Type 头信息设置为 application/json，同时使用 PHP 的 json_encode 函数将给定的数组转换为 JSON：

```
return response()->json([
    'name' => 'Abigail',
    'state' => 'CA',
]);
```

4.6.3.3. 文件下载

download 方法可用于生成强制用户浏览器在给定路径下载文件的响应。download 方法接受文件名作为该方法的第二个参数，这将确定下载文件的用户看到的文件名。最后，您可以将一组 HTTP 标头作为该方法的第三个参数传递：

```
return response()->download($pathToFile);

return response()->download($pathToFile, $name, $headers);
```

4.6.3.4. 文件响应

file 方法可用于直接在用户的浏览器中显示文件，例如图像或 PDF，而不是启动下载。这个方法接受文件的路径作为它的第一个参数和一个头数组作为它的第二个参数：

```
return response()->file($pathToFile);



return response()->file($pathToFile, $headers);
```

4.6.4. 响应宏

如果你想定义一个可以在各种路由和控制器中重复使用的自定义响应，你可以使用 Response Facade 上的 macro 方法。

4.7. 视图

4.7.1. 介绍

当然，直接从路由和控制器返回整个 HTML 文档字符串是不切实际的。值得庆幸的是，视图提供了一种方便的方式来将我们所有的 HTML 放在单独的文件中。视图将您的控制器 / 应用程序逻辑与您的表示逻辑分开并存储在 resources/views 目录中。一个简单的视图可能看起来像这样：

```
<!-- View stored in resources/views/greeting.blade.php -->

<html>

    <body>

        <h1>Hello, {{ $name }}</h1>

    </body>

</html>
```

4.7.2. 创建和渲染视图

您可以通过在应用程序 resources/views 目录中放置具有 .blade.php 扩展名的文件来创建视图。 该 .blade.php 扩展通知框架该文件包含一个 Blade 模板。Blade 模板包含 HTML 和 Blade 指令，允许您轻松地回显值、创建「if」语句、迭代数据等。

4.7.2.1. 嵌套视图目录

视图也可以嵌套在目录 resources/views 的子目录中。「.」符号可用于引用嵌套视图。例如，如果您的视图存储在 resources/views/admin/profile.blade.php，

您可以从应用程序的路由 / 控制器之一返回它，如下所示：

```
return view('admin.profile', $data);
```

4.7.2.2. 创建第一个可用视图

使用 View 门面的 first 方法，你可以创建给定数组视图中第一个存在的视图。如果你的应用程序或开发的第三方包允许定制或覆盖视图，这非常有用：

```
use Illuminate\Support\Facades\View;

return View::first(['custom.admin', 'admin'], $data);
```

4.7.2.3. 判断视图文件是否存在

如果需要判断视图文件是否存在，可以使用 View 门面。如果视图存在，exists 方法会返回 true：

```
use Illuminate\Support\Facades\View;

if (View::exists('emails.customer')) {
    //
}
```

4.7.3. 向视图传递数据

正如您在前面的示例中看到的，您可以将数据数组传递给视图，以使该数据可用于视图：

```
return view('greetings', ['name' => 'Victoria']);
```

4.7.3.1. 与所有视图共享数据

有时，您可能需要与应用程序呈现的所有视图共享数据。

4.7.4. 视图 Composer

视图 Composer 是在渲染视图时调用的回调或类方法。如果您希望在每次呈现视图时将数据绑定到该视图，则视图编辑器可以帮助您将该逻辑组织到一个位置。如果应用程序中的多个路由或控制器返回相同的视图并且总是需要特定的数据，则视图合成器可能会特别有用。

通常，视图合成器将在您的应用程序的 服务提供者之中注册。在这个例子中，我们假设我们已经创建了一个新 App\Providers\ViewServiceProvider 的来构建这个逻辑。

4.7.5. 优化视图

默认情况下，Blade 模板视图是按需编译的。当执行渲染视图的请求时，Laravel 将确定视图的编译版本是否存在。如果文件存在，Laravel 将比较未编译的视图和已编译的视图是否有修改。如果编译后的视图不存在，或者未编译的视图已被修改，Laravel 将重新编译该视图。

4.8. Blade 模板

4.8.1. 简介

Blade 是 Laravel 提供的一个简单而又强大的模板引擎。 和其他流行的 PHP 模板引擎不同，Blade 并不限制你在视图中使用原生 PHP 代码。实际上，所有 Blade 视图文件都将被编译成原生的 PHP 代码并缓存起来，除非它被修改，否则不会重新编译，这就意味着 Blade 基本上不会给你的应用增加任何负担。Blade 模板文件使用 .blade.php 作为文件扩展名，被存放在 resources/views 目录。

Blade 视图可以使用全局 view 函数从 Route 或控制器返回。当然，正如有关 views 的文档中所描述的，可以使用 view 函数的第二个参数将数据传递到 Blade 视图：

```
Route::get('/', function () {
    return view('greeting', ['name' => 'Finn']);
});
```

4.8.2. 显示数据

你可以把变量置于花括号中以在视图中显示数据。

4.8.2.1. HTML 实体编码

默认情况下，Blade（和 Laravel e 助手）将对 HTML 实体进行双重编码。如果您想禁用双重编码，请从 AppServiceProvider 的 boot 方法调用 Blade::withoutDoubleEncoding 方法：

```
<?php
namespace App\Providers;
use Illuminate\Support\Facades\Blade;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * 在此启动任意服务
     *
     * @return void
     */
    public function boot()
    {
```

```
        Blade::withoutDoubleEncoding();
    }
}
```

4.8.3. Blade 指令

除了模板继承和显示数据以外， Blade 还为常见的 PHP 控制结构提供了便捷的快捷方式，例如条件语句和循环。这些快捷方式为 PHP 控制结构提供了一个非常清晰、简洁的书写方式，同时，还与 PHP 中的控制结构保持了相似的语法特性。

4.8.3.1. If 语句

您可以使用 @if ， @elseif ， @else 和 @endif 指令构造 if 语句。这些指令功能与它们所对应的 PHP 语句完全一致：

```
@if (count($records) === 1)
    // 有一条记录
@elseif (count($records) > 1)
    // 有多条记录
@else
    // 没有记录
@endif
```

4.9. URL 生成

4.9.1. 简介

Laravel 提供了几个辅助函数来为应用程序生成 URL。主要用于在模板和 API 响应中构建 URL 或者在应用程序的其它部分生成重定向响应。

4.9.2. 快速入门

4.9.2.1. 生成基础 URL

Url 辅助函数可用于为应用生成任意 URL，生成的 URL 将自动使用当前请求中的方案 (HTTP or HTTPS) 和主机。

```
$post = App\Post::find(1);

echo url("/posts/{$post->id}");
```

4.9.2.2. 访问当前 URL

如果没有给辅助函数 url 提供路径，则会返回一个 Illuminate\Routing\UrlGenerator 实例，来允许您访问当前 UR 的信息。

```
// 获取当前 URL 没有 query string...

echo url()->current();

// 获取当前 URL 包括 query string...

echo url()->full();

// 获取上个请求 URL

echo url()->previous();
```

4.9.3. 命名路由 URL

辅助函数 route 可以用于生成指定 命名路由 的 URL。命名路由生成的 URL 不与路由上定义的 URL 相耦合。因此，就算路由的 URL 有任何改变，都不需要对 route 函数调用进行任何更改。例如，假设你的应用程序包含以下路由：

```
Route::get('/post/{post}', function (Post $post) {

    //

})->name('post.show');
```

4.9.3.1. 签名 URL

Laravel 允许你轻松地为命名路径创建「签名」URL，这些 URL 在查询字符串后附加了「签名」哈希，允许 Laravel 验证 URL 自创建以来未被修改过。签名 URL 对于可公开访问但需要一层防止 URL 操作的路由特别有用。

例如，你可以使用签名 URL 来实现通过电子邮件发送给客户的公共「取消订阅」链接。要创建指向路径的签名 URL ，请使用 URL facade 的 signedRoute 方法：

```
use Illuminate\Support\Facades\URL;

return URL::signedRoute('unsubscribe', ['user' => 1]);
```

4.9.4. 控制器行为 URL

action 功能可以为给定的控制器行为生成 URL。

```
use App\Http\Controllers\HomeController;

$url = action([HomeController::class, 'index']);
```

如果控制器方法接收路由参数，你可以通过第二个参数传递：

```
url = action([UserController::class, 'profile'], ['id' => 1]);
```

4.9.5. 默认值

对于某些应用程序，你可能希望为某些 URL 参数的请求范围指定默认值。例如，假设有些路由定义了 {locale} 参数。

```
Route::get('/{locale}/posts', function () {

    //

})->name('post.index');
```

4.10. Session

4.10.1. 简介

由于 HTTP 驱动的应用程序是无状态的，Session 提供了一种在多个请求之间存储有关用户信息的方法，这类信息一般都存储在后续请求可以访问的持久存储 / 后端中。

Laravel 通过同一个可读性强的 API 处理各种自带的后台驱动程序。支持诸如比较热门的 Memcached，Redis 和数据库。

4.10.1.1.配置

Session 的配置文件存储在 config/session.php 文件中。请务必查看此文件中对于你而言可用的选项。默认情况下，Laravel 为绝大多数应用程序配置的 Session 驱动为 file 驱动，它适用于大多数程序。如果你的应用程序需要在多个 Web 服务器之间进行负载平衡，你应该选择一个所有服务器都可以访问的集中式存储，例如 Redis 或数据库。

在生产环境中，你可以考虑使用 memcached 或 redis 驱动，让 Session 的性能更加出色。

Session driver 的配置预设了每个请求存储 Session 数据的位置。Laravel 自带了几个不错而且开箱即用的驱动：

file - Sessions 存储在 storage/framework/sessions 中。

cookie - Sessions 被存储在安全加密的 cookie 中。

database - Sessions 被存储在关系型数据库中。

memcached / redis - Sessions 被存储在基于高速缓存的存储系统中。

dynamodb - Sessions 被存储在 AWS DynamoDB 中.

array - Sessions 存储在 PHP 数组中，但不会被持久化。

4.10.1.2.驱动先决条件

4.10.1.2.1. 数据库

使用 database Session 驱动时，你需要创建一个记录 Session 的表。下面是 Schema 的声明示例：

```
Schema::create('sessions', function ($table) {
    $table->string('id')->primary();
    $table->foreignId('user_id')->nullable()->index();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->text('payload');
    $table->integer('last_activity')->index();
});
```

4.10.1.2.2. Redis

在 Laravel 使用 Redis Session 驱动前，你需要安装 PhpRedis PHP 扩展，可以通过 PECL 或者 通过 Composer 安装这个 predis/predis 包 (~1.0)。

4.10.2. 使用 Session

4.10.2.1.获取数据

在 Laravel 中有两种基本的 Session 使用方式：全局 session 助手函数和通过 Request 实例。首先看下通过 Request 实例访问 Session，它可以隐式绑定路由闭包或者控制器方法。记住，Laravel 会自动注入控制器方法的依赖。

4.10.2.2.存储数据

Session 里存储数据， 你通常将使用 Request 实例中的 put 方法 或者 session 助手函数：

```
//通过调用请求实例的 put 方法
```

```
$request->session()->put('key', 'value');

// 通过全局辅助函数 session

session(['key' => 'value']);
```

4.10.2.3.闪存数据

有时候你可能想要在 Session 中存储有效的数据用于下个请求，这可以通过 flash 方法来实现。使用该方法存储的 Session 数据只在随后的 HTTP 请求中有效，然后将会被删除。闪存数据主要用于短期的状态消息。

```
$request->session()->flash('status', 'Task was successful!');
```

4.10.2.4.删除数据

Forget 方法从 Session 中移除指定数据，如果您要从 Session 中移除所有数据，可以使用 flush 方法。

```
// Forget a single key...

$request->session()->forget('key');

// Forget multiple keys...

$request->session()->forget(['key1', 'key2']);

$request->session()->flush();
```

4.10.2.5.重新生成 SessionID

重新生成 Session ID 经常被用来阻止恶意用户使用 Session Fixation 攻击你的应用。

如果你正在使用 入门套件 或者 Laravel Fortify 中的任意一种， Laravel 会在认证阶段自动生成 Session ID；然而如果你需要手动重新生成 Session ID ，可以使用 regenerate 方法：

```
$request->session()->regenerate();
```

4.10.3. Session 阻塞

默认情况下，Laravel 允许使用同一 Session 的请求并发地执行，举例来说，如果你使用一个 JavaScript HTTP 库向你的应用执行两次 HTTP 请求，它们将同时执行。对多数应用这不是问题，然而 在一小部分应用中可能出现 Session 数据丢失，这些应用会向两个不同的应用端并发请求，并同时写入数据到 Session。

4.10.4. 添加自定义 Session 驱动

如果现存的 Session 驱动不能满足你的需求，Laravel 允许你自定义 Session Handler。你的自定义驱动应实现 PHP 内置的 SessionHandlerInterface ，这个接口仅包含几个方法。以下是 MongoDB 驱动实现的代码片段：

```
<?php

namespace App\Extensions;


class MongoSessionHandler implements \SessionHandlerInterface
{
    public function open($savePath, $sessionName) {}

    public function close() {}

    public function read($sessionId) {}

    public function write($sessionId, $data) {}

    public function destroy($sessionId) {}

    public function gc($lifetime) {}
}
```

4.11. 验证

4.11.1. 简介

Laravel 提供了几种不同的方法来验证传入应用程序的数据。最常见的做法是在所有传入的 HTTP 请求中使用 validate 方法。但是，我们还将讨论其他验证方法。

Laravel 包含了各种方便的验证规则，你可以将它们应用于数据，甚至可以验证给定数据库表中的值是否唯一。我们将详细介绍每个验证规则，以便你熟悉 Laravel 的所有验证功能。

4.11.2. 快速入门

为了了解 Laravel 强大的验证功能，我们来看一个表单验证并将错误消息展示给用户的完整示例。通过阅读概述，这将会对你如何使用 Laravel 验证传入的请求数据有一个很好的理解：

4.11.2.1.定义路由

首先，假设我们在 routes/web.php 路由文件中定义了下面这些路由：

```
use App\Http\Controllers\PostController;

Route::get('/post/create', [PostController::class, 'create']);

Route::post('/post', [PostController::class, 'store']);
```

GET 路由会用户显示了一个创建新的文章的表单，POST 路由将新的文章存储到数据库。

4.11.2.2.创建控制器

下面让我们一起来看看处理这些路由的控制器 store 方法暂时留空。

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use Illuminate\Http\Request;
```

```
class PostController extends Controller
{
    /**
     * 显示创建博客文章的表单.
     *
     * @return Response
     */
    public function create()
    {
        return view('post.create');
    }

    /**
     * 保存一篇新的博客文章
     *
     * @param  Request  $request
     * @return Response
     */
    public function store(Request $request)
    {
        // 验证并存储博客文章...
    }
}
```

4.11.2.3.编写验证逻辑

现在我们开始在 store 方法中编写用来验证新的博客文章的逻辑代码。为此，我们将使用 Illuminate\Http\Request 类提供的 validate 方法。如果验证通过，你的代码会继续正常运行。如果验证失败，则会抛出 Illuminate\Validation\ValidationException 异常，并自动将对应的错误响应返回给用户。

如果在传统 HTTP 请求期间验证失败，则会生成对先前 URL 的重定向响应。如果传入的请求是 XHR，将将返回包含验证错误信息的 JSON 响应。

为了深入理解 validate 方法，让我们接着回到 store 方法中：

```
/**
 * 保存一篇新的博客文章
 *
 * @param  Request  $request
 * @return Response
 */
public function store(Request $request)
{
    $validatedData = $request->validate([
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);

    // 博客文章验证通过...
}
```

如您所见，验证规则被传递到 validate 方法中。不用担心 - 所有可用的验证规则均已记录在案。另外再提醒一次，如果验证失败，会自动生成一个对应的响应。如果验证通过，那我们的控制器会继续正常运行。

4.11.2.4.显示验证错误信息

那么，如果传入的请求参数未通过给定的验证规则呢？正如前面所提到的，Laravel 会自动将用户重定向到之前的位置。另外，所有的验证错误信息和 请求输入 都将自动存储到 闪存 session 中。

Illuminate\View\Middleware\ShareErrorsFromSession 中间件与应用程序的所有视图共享一个 $errors 变量，该变量由 web 中间件组提供。当应用该中间件时，$errors 变量在你的视图中始终可用，从而使你可以方便地假定 $errors 变量始终被定义并且可以安全地使用。$errors 变量是 Illuminate\Support\MessageBag 的一个实例。有关使用此对象的更多信息，请 参阅这篇文档。

4.11.2.5.回填表单

当 Laravel 由于验证错误而生成重定向响应时，框架将自动 将所有请求的输入闪存到 session 中。这样做是为了方便你在下一个请求期间访问输入，并重新填充用户尝试提交的表单。

4.11.2.6.可选字段注意事项

默认情况下， 在您的 Laravel 应用的全局中间件堆栈 App\Http\Kernel 类中包含了 TrimStrings 和 ConvertEmptyStringsToNull 中间件。因此，如果你不想让 null 被验证器标识为非法的话，你需要将「可选」字段标志为 nullable。例如：

```
$request->validate([
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
    'publish_at' => 'nullable|date',
]);
```

4.11.3. 验证表单请求

4.11.3.1.创建表单请求验证

面对更复杂的验证场景，您可以创建一个「表单请求」。表单请求是一个包含了验证逻辑的自定义请求类。要创建一个表单请求类，请使用 make:request Artisan CLI 命令：

```
php artisan make:request StorePostRequest
```

该命令生成的表单请求类将被置于 app/Http/Requests 目录中。如果这个目录不存在，在您运行 make:request 命令后将会创建这个目录。Laravel 生成的每个表单请求都有两个方法：authorize 和 rules。

正如您可能已经猜到的那样，authorize 方法负责确定当前经过身份验证的用户是否可以执行请求操作，而 rules 方法则返回适用于请求数据的验证规则：

```
/**
 * 获取应用于请求的验证规则
 *
 * @return array
 */
public function rules()
{
    return [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ];
}
```

所以，验证规则是如何运行的呢？你所需要做的就是在控制器方法中类型提示传入的请求。在调用控制器方法之前验证传入的表单请求，这意味着你不需要

在控制器中写任何验证逻辑。

4.11.3.2.表单请求认证授权

表单请求类内也包含了 authorize 方法。在这个方法中，你可以检查经过身份验证的用户确定其是否具有更新给定资源的权限。比方说，你可以判断用户是否拥有更新文章评论的权限。

```
/**
 * 判断用户是否有请求权限
 *
 * @return bool
 */
public function authorize()
{
    $comment = Comment::find($this->route('comment'));

    return $comment && $this->user()->can('update', $comment);
}
```

由于所有的表单请求都是继承了框架中的请求基类，所以我们可以使用 user 方法去获取当前认证登录的用户。

4.11.3.3.自定义错误消息

你可以通过重写表单请求的 messages 方法来自定义错误消息。

4.11.3.4.自定义验证属性

如果你希望将验证消息中的 :attribute 部分替换为自定义的属性名，可以通过重写 attributes 方法来指定自定义的名称。该方法会返回属性名及对应自定义名称键值对数组。

```
/**
```

```
* Get custom attributes for validator errors.
*
* @return array
*/
public function attributes()
{
    return [
        'email' => 'email address',
    ];
}
```

4.11.3.5.准备验证接入

如果需要在应用验证规则之前清除请求中的任何数据，可以使用 prepareForValidation 方法。

```
use Illuminate\Support\Str;


/**
 * 准备验证数据
 *
 * @return void
 */
protected function prepareForValidation()
{
    $this->merge([
```

```
        'slug' => Str::slug($this->slug),
    ]);
}
```

4.11.4. 手动创建验证器

如果您不想在请求中使用 validate 方法,您可以使用 Validator 门面 手动创建一个验证器实例。门面中的 make 方法将会生成一个新的验证器实例：

```
<?php

namespace App\Http\Controllers;


use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;


class PostController extends Controller
{
    /**
     * 存储一篇博客文章。
     *
     * @param  Request  $request
     * @return Response
     */
    public function store(Request $request)
    {
```

```
        $validator = Validator::make($request->all(), [
            'title' => 'required|unique:posts|max:255',
            'body' => 'required',
        ]);

        if ($validator->fails()) {
            return redirect('post/create')
                        ->withErrors($validator)
                        ->withInput();
        }

        // 获取通过验证的数据...
        $validated = $validator->validated();

        // 获取部分通过验证的数据...
        $validated = $validator->safe()->only(['name', 'email']);
        $validated = $validator->safe()->except(['name', 'email']);

        //存储博客文章...
    }
}
```

make 方法中的第一个参数是期望校验的数据。第二个参数是应用到数据上的校验规则。

4.11.4.1.自动重定向

如果您想要在手动创建验证器实例的同时，使用请求的 validate 方法的自动重定向功能，那么您可以在现有的验证器实例上调用 validate 方法。如果校验失败，用户将被自动重定向，或在 XHR 请求下，将会返回一个 JSON 响应：

```
Validator::make($request->all(), [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
])->validate();
```

如果验证失败，您可以使用 validateWithBag 方法将错误消息存储在命名错误包中。

```
Validator::make($request->all(), [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
])->validateWithBag('post');
```

4.11.4.2.命名错误包

如果您的一个页面中存在多个表单，您可以为错误的 MessageBag 命名，用于检索指定表单的错误信息。只需将名称作为 withErrors 的第二个参数传递给它：

```
return redirect('register')
            ->withErrors($validator, 'login');
```

然您可以通过 $errors 变量访问命名后的 MessageBag 实例：

```
{{ $errors->login->first('email') }}
```

4.11.4.3.自定义错误消息

如果需要，您可以提供验证程序实例使用的自定义错误消息，而不是 Laravel

提供的默认错误消息。有几种指定自定义消息的方法。首先，您可以将自定义消息作为第三个参数传递给 Validator::make 方法：

```
$validator = Validator::make($input, $rules, $messages = [
    'required' => 'The :attribute field is required.',
]);
```

4.11.4.4.验证后钩子

验证器允许您在完成验证操作后执行附加的回调。以便您处理下一步的验证，甚至是往信息集合中添加更多的错误信息。您可以在验证器实例上使用 after 方法实现：

```
$validator = Validator::make(...);

$validator->after(function ($validator) {
    if ($this->somethingElseIsInvalid()) {
        $validator->errors()->add('field', 'Something is wrong with this field!');
    }
});

if ($validator->fails()) {
    //
}
```

4.11.5. 处理验证字段

在使用表单请求或手动创建的验证器实例验证传入请求数据后，您可能希望检索经过验证的请求数据。 这可以通过多种方式实现。 首先，您可以在表单请求或验证器实例上调用 validated 方法。 此方法返回已验证的数据数组：

```
$validated = $request->validated();

$validated = $validator->validated();
```

4.11.6. 错误信息处理

通过 Validator 实例调用 errors 方法，它会返回一个 Illuminate\Support\MessageBag 实例，该实例包含了各种可以很方便地处理错误信息的方法。并自动给所有视图提供 $errors 变量，也是 MessageBag 类的一个实例。

您可以使用 first 方法检索给定字段的第一个错误信息：

```
$errors = $validator->errors();

echo $errors->first('email');
```

4.11.6.1.自定义验证错误信息

Laravel 的内置验证规则每个都对应一个错误消息，位于应用程序的 lang/en/validation.php 文件中。在此文件中，您将找到每个验证规则的翻译条目。您可以根据应用程序的需求随意更改或修改这些消息。

此外，您可以将此文件复制到另一个翻译语言目录中，以翻译应用程序语言的消息。要了解有关 Laravel 本地化的更多信息，请查看完整的本地化文档。

4.11.7. 可用验证规则

以下是所有可用验证规则。

accepted

验证字段必须是 yes, on, 1, 或 true。

4.11.8. 按条件添加条件规则

字段包含特定值跳过验证

如果另一个字段具有给定的值，你可能偶尔希望不验证给定字段。

4.11.9. 验证数组

验证表单的输入为数组的字段也不难。你可以使用 [点] 方法来验证数组中的属性。例如，如果传入的 HTTP 请求中包含 photos[profile] 字段，可以如相验证。

```
$validator = Validator::make($request->all(), [
    'photos.profile' => 'required|image',
]);
```

4.11.10. 自定义验证规则

4.11.10.1. 使用验证规则对象

Laravel 提供了许多有用的验证规则；同时也支持自定义规则。注册自定义验证规则的方法之一，就是使用规则对象。可以使用 Artisan 命令 make:rule 来生成新的规则对象。接下来，让我们用这个命令生成一个验证字符串是否是大写的规则。Laravel 会将新的规则存放在 app/Rules 目录中。

```
php artisan make:rule Uppercase
```

4.11.10.2. 使用闭包

如果你在应用程序中只需要一次自定义规则的功能，则可以使用闭包代替规则对象。

4.11.10.3. 隐式规则

默认情况下，当正在验证的属性不存在或包含空字符串时，不会运行正常的验证规则，包括自定义规则。例如，unique 规则不会针对空字符串运行：

```
use Illuminate\Support\Facades\Validator;

$rules = ['name' => 'unique:users,name'];

$input = ['name' => ''];

Validator::make($input, $rules)->passes(); // true
```

4.12. 错误处理

4.12.1. 简介

当你开始一个新的 Laravel 项目时，它已经为您配置了错误和异常处理。App\Exceptions\Handler 类用于记录应用程序触发的所有异常，然后将其呈现回用户。我们将在本文中深入讨论这个类。

4.12.2. 配置

你的 config/app.php 配置文件中的 debug 选项决定了对于一个错误实际上将显示多少信息给用户。默认情况下，该选项的设置将遵照存储在 .env 文件中的 APP_DEBUG 环境变量的值。

对于本地开发，你应该将 APP_DEBUG 环境变量的值设置为 true 。在生产环境中，该值应始终为 false 。如果在生产中将该值设置为 true ，则可能会将敏感配置值暴露给应用程序的终端用户。

4.12.3. 异常处理

4.12.3.1.异常报告

所有异常都是由 App\Exceptions\Handler 类处理。此类包含一个 register 方法，可以在其中注册自定义异常报告程序和渲染器回调。我们将详细研究每个概念。异常报告用于记录异常或将其发送到如 Flare、Bugsnag 或 Sentry 等外部服务。默认情况下，将根据你的 日志 配置来记录异常。不过，你可以用任何自己喜欢的方式来记录异常。

例如，如果您需要以不同的方式报告不同类型的异常，您可以使用 reportable 方法注册一个闭包，当需要报告给定的异常的时候便会执行它。 Laravel 将通过检查闭包的类型提示来判断闭包报告的异常类型：

```
use App\Exceptions\InvalidOrderException;

/**
 * 为应用程序注册异常处理回调
 *
 * @return void
 */
public function register()
{
    $this->reportable(function (InvalidOrderException $e) {
        //
    });
}
```

4.12.3.2.按类型忽略异常

在构建应用时，会有一些类型的异常您只想忽略并且永远不会报告。应用的异常处理程序包含一个 $dontReport 属性，该属性被初始化为一个空数组。您添加到此属性的任何类都不会被报告；但是，它们可能仍然具有自定义呈现逻辑：

```
use App\Exceptions\InvalidOrderException;

/**
 * 不应上报的异常类型列表。
 *
```

```
 * @var array
 */
protected $dontReport = [
    InvalidOrderException::class,
];
```

4.12.3.3.渲染异常

默认情况下，Laravel 异常处理器会为你把异常转换为 HTTP 响应。然而，你可以自由地为特定类型的异常注册一个自定义的渲染闭包来实现。

传递给 renderable 方法的闭包函数应返回一个 Illuminate\Http\Response 的实例，它可以通过 response 助手函数生成。 Laravel 将会根据闭包的类型提示来推断闭包渲染的异常类型：

```
use App\Exceptions\InvalidOrderException;
/**
 * 注册异常处理回调
 *
 * @return void
 */
public function register()
{
    $this->renderable(function (InvalidOrderException $e, $request) {
        return response()->view('errors.invalid-order', [], 500);
    });
}
```

4.12.3.4.Reportable & Renderable 异常

除了在异常控制器的 register 方法中检查异常类型外，你可以直接地在自定义异常里定义 report 和 render 。当这些方法存在时，它们将被框架自动调用：

```
<?php

namespace App\Exceptions;

use Exception;


class InvalidOrderException extends Exception
{
    /**
     * 报告异常。
     *
     * @return bool|null
     */
    public function report()
    {
        //
    }


    /**
     *渲染异常为 HTTP 响应。
     *
     * @param  \Illuminate\Http\Request  $request
```

```
     * @return \Illuminate\Http\Response
     */
    public function render($request)
    {
        return response(...);
    }
}
```

如果你的异常扩展了一个已经可以渲染的异常，比如内置的 Laravel 或 Symfony 异常，你可以从异常的 render 方法返回 false 来渲染异常的默认 HTTP 响应：

```
/**
 * 将异常渲染为 HTTP 响应。
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\Response
 */
public function render($request)
{
    // 判断异常是否需要自定义报告...

    return false;
}
```

4.12.4. HTTP 异常

某些异常描述了服务器的 HTTP 错误代码。例如，可能是「 页面未找到 」错误 （404），「 未经授权的错误 」（401）或者甚至是开发者造成的 500 错误。要在应用的任意地方生成此类响应，你可以使用 abort 辅助函数：

```
abort(404);
```

4.12.4.1. 自定义 HTTP 错误页面

Laravel 创建了可以轻松显示各种 HTTP 状态码的自定义错误页面。例如，如果你想自定义 404 HTTP 状态码页面，只需创建一个 resources/views/errors/404.blade.php 文件。它会用来处理所有应用程序产生的 404 错误。视图目录下所有文件命名都应和它们所响应的 HTTP 状态码一一对应。

4.13. 日志

4.13.1. 简介

为了帮助你了解程序中正在发生什么，Laravel 提供了健壮的日志服务，允许你记录日志信息到文件，到系统错误日志，甚至到 Slack 通知你的整个团队。

Laravel 日志基于「 通道 」。每个通道代表一个具体的记录日志消息的方式。举例来说，single 通道会把日志记录到一个单独的文件里， slack 通道会发送日志信息到 Slack。基于它们的重要程度，日志可能会被写入到多个通道中去。

Laravel 使用了 Monolog 库，它为各种强大的日志处理提供支持。Laravel 使配置这些处理器变得小菜一碟，它允许以混合和匹配的方式，自定义你的程序日志处理。

4.13.2. 配置

所有的应用程序日志系统配置都位于 config/logging.php 配置文件中。这个文件允许你配置程序的日志通道，因此务必查看每个可用通道和它们的选项。我

们将在下面回顾一些常用的选项。

Laravel 默认使用 stack 通道记录日志消息。 stack 通道被用于将多个日志通道集成到一个单独的通道中去。获得更多构建堆栈信息，请查看 以下文档 。

配置通道名

默认情况下，Monolog 用与当前环境匹配的 channel name 实例化，例如 production 或 local。要更改此值，请在通道配置中添加 name 选项：

```
'stack' => [
    'driver' => 'stack',
    'name' => 'channel-name',
    'channels' => ['single', 'slack'],
],
```

4.13.2.1.可用的通道驱动

每个日志通道都由一个驱动程序驱动。驱动程序确定日志消息的实际记录方式和位置。以下日志通道驱动程序在每个 Laravel 应用程序中都可用。大多数驱动程序的条目已经存在于应用程序的 config/logging.php 配置文件中，因此请务必查看此文件以熟悉其内容：

名称	说明
custom	调用指定工厂来创建通道的程序
daily	一个基于 RotatingFileHandler 的每日循环的 Monolog 驱动程序

4.13.2.2.通道先决条件

single 和 daily 通道有三个可选配置选项：bubble 、permission 和 locking。

4.13.2.3.记录弃用警告

PHP、Laravel 和其他库经常通知他们的用户，他们的一些功能已被弃用，并将在未来的版本中删除。如果你想记录这些弃用警告，你可以在应用程序的 config/logging.php 配置文件中指定 deprecations 的日志通道：

```
'deprecations' => env('LOG_DEPRECATIONS_CHANNEL', 'null'),

'channels' => [

    ...

]
```

4.13.3. 构建日志堆栈

如前所述，stack 为方便起见，该驱动程序允许你将多个通道合并为一个日志通道。为了说明如何使用日志堆栈，让我们看一下你可能在生产应用程序中看到的示例配置：

```
'channels' => [

    'stack' => [

        'driver' => 'stack',

        'channels' => ['syslog', 'slack'],

    ],


    'syslog' => [

        'driver' => 'syslog',

        'level' => 'debug',

    ],


    'slack' => [
```

```
        'driver' => 'slack',
        'url' => env('LOG_SLACK_WEBHOOK_URL'),
        'username' => 'Laravel Log',
        'emoji' => ':boom:',
        'level' => 'critical',
    ],
],
```

4.13.4. 编写日志消息

你可以使用 Log facade 将信息写入日志。如前所述，记录器提供 RFC 5424 规范 中定义的八个记录级别： emergency、 alert、 critical、 error、 warning、 notice、 info 和 debug：

```
use Illuminate\Support\Facades\Log;
Log::emergency($message);
Log::alert($message);
Log::critical($message);
Log::error($message);
Log::warning($message);
Log::notice($message);
Log::info($message);
Log::debug($message);
```

4.13.4.1.上下文信息

可以将一组上下文数据传递给日志方法。此上下文数据将被格式化并与日志

消息一起显示：

```
use Illuminate\Support\Facades\Log;

Log::info('User failed to login.', ['id' => $user->id]);
```

4.13.4.2.写入指定的通道

有时你可能希望将消息记录到应用程序默认通道以外的通道。你可以使用 Log 门面的 channel 方法来检索并记录到配置文件中定义的任何通道：

```
use Illuminate\Support\Facades\Log;

Log::channel('slack')->info('Something happened!');
```

4.13.5. Monolog 通道自正定义

4.13.5.1.为通道自定义 Monolog

有时你可能需要完全控制如何为现有通道配置 Monolog。例如，你可能想要为 Laravel 的内置 single 通道配置自定义 Monolog FormatterInterface 实现。

首先，在通道的配置上定义一个 tap 数组。tap 数组应该包含一个类列表，这些类应该有机会在创建 Monolog 实例后进行自定义（或「tap」）。没有应该放置这些类的常规位置，因此你可以在应用程序中自由创建一个目录来包含这些类：

```
'single' => [
    'driver' => 'single',
    'tap' => [App\Logging\CustomizeFormatter::class],
    'path' => storage_path('logs/laravel.log'),
    'level' => 'debug',
],
```

4.13.5.2.创建 Monolog 处理器通道

Monolog 有多种 可用的处理程序，并且 Laravel 没有为每个处理程序包含一个内置通道。 在某些情况下，你可能希望创建一个自定义通道，它只是一个特定的 Monolog 处理程序的实例，它没有相应的 Laravel 日志驱动程序。这些通道可以使用 monolog 驱动轻松创建。

使用 monolog 驱动程序时，handler 配置选项用于指定将实例化哪个处理程序。或者，可以使用 with 配置选项指定处理程序所需的任何构造函数参数：

```
'logentries' => [
    'driver'  => 'monolog',
    'handler' => Monolog\Handler\SyslogUdpHandler::class,
    'with' => [
        'host' => 'my.logentries.internal.datahubhost.company.com',
        'port' => '10000',
    ],
],
```

4.13.5.3.通过工厂创建通道

如果你想定义一个完全自定义的通道，你可以在其中完全控制 Monolog 的实例化和配置，你可以在 config/logging.php 配置文件中指定 custom 驱动程序类型。你的配置应该包括一个 via 选项,其中包含将被调用以创建 Monolog 实例的工厂类的名称：

```
'channels' => [
    'example-custom-channel' => [
        'driver' => 'custom',
```

```
        'via' => App\Logging\CreateCustomLogger::class,
    ],
],
```

第五章 进阶系列

5.1. Artisan 控制台

5.1.1. 简介

Artisan 是 Laravel 附带的命令行接口。Artisan 以 artisan 脚本的形式存在于应用的根目录，并提供了许多有用的命令，这些命令可以在构建应用时为你提供帮助。你可以通过 list 命令查看所有可用的 Artisan 命令：

```
php artisan list
```

每个命令都包含了「help」界面，它会显示和概述命令的可用参数及选项。只需要在命令前加上 help 即可查看命令帮助界面：

```
php artisan help migrate
```

5.1.1.1. Tinker 命令(REPL)

Laravel Tinker 是为 Laravel 提供的强大的 REPL（交互式解释器），由 PsySH 提供支持。

安装

所有 Laravel 应用都默认包含了 Tinker。如果你之前已经将 Tinker 从应用中删除，可以使用 Composer 进行手动安装：

```
composer require laravel/tinker
```

用法

Tinker 让你可以在命令行与你的整个 Laravel 应用进行交互。包括但不限于 Eloquent 模型、任务、事件等。通过运行 Artisan 命令 tinker 进入 Tinker 环境。

```
php artisan tinker
```

5.1.2. 编写命令

除了 Artisan 提供的命令之外，你也可以编写自己的自定义命令。命令在多数情况下位于 app/Console/Commands 目录中；不过，只要的命令可以由 Composer 加载，你就可以自己选择自己的存储位置。

5.1.2.1. 生成命令

要创建新命令，可以使用 make:command Artisan 命令。该命令将在 app/Console/Commands 目录中创建一个新的命令类。如果你的应用程序中不存在此目录，请不要担心，它将在你第一次运行 make:command 命令时自动创建：

```
php artisan make:command SendEmails
```

5.1.2.2. 命令结构

生成命令后，应为该类的 signature 和 description 属性定义适当的值。当在 list 屏幕上显示命令时，将使用这些属性。 signature 属性还允许你定义 命令的输入期望值。 handle 执行命令时将调用该方法。你可以将命令逻辑放在此方法中。

让我们看一个示例命令。请注意，我们能够通过命令的 handle 方法请求我们需要的任何依赖项。Laravel 服务容器 将自动注入此方法签名中带有类型提示的所有依赖项：

```
<?php

namespace App\Console\Commands;
```

```
use App\Models\User;
use App\Support\DripEmailer;
use Illuminate\Console\Command;

class SendEmails extends Command
{
    /**
     * 命令名称及签名.
     *
     * @var string
     */
    protected $signature = 'mail:send {user}';

    /**
     * 命令描述.
     *
     * @var string
     */
    protected $description = 'Send a marketing email to a user';

    /**
     * 创建命令.
     *
     * @return void
```

```
     */
    public function __construct()
    {
        parent::__construct();
    }


    /**
     * 执行命令.
     *
     * @param  \App\Support\DripEmailer  $drip
     * @return mixed
     */
    public function handle(DripEmailer $drip)
    {
        $drip->send(User::find($this->argument('user')));
    }
}
```

5.1.2.3. 闭包命令

基于闭包的命令为将控制台命令定义为类提供了一种替代方法。与路由闭包可以替代控制器一样，可以将命令闭包视为命令类的替代。在 app/Console/Kernel.php 文件的 commands 方法中，Laravel 加载 routes/console.php 文件：

```
/**
```

```
 * 注册闭包命令
 *
 * @return void
 */
protected function commands()
{
    require base_path('routes/console.php');
}
```

尽管该文件没有定义 HTTP 路由，但它定义了进入应用程序的基于控制台的入口 (routes) 。在这个文件中，你可以使用 Artisan::command 方法定义所有的闭包路由。 command 方法接受两个参数： 命令名称 和可调用的闭包，闭包接收命令的参数和选项：

```
Artisan::command('mail:send {user}', function ($user) {
    $this->info("Sending email to: {$user}!");
});
```

该闭包绑定到基础命令实例，因此你可以完全访问通常可以在完整命令类上访问的所有辅助方法。

5.1.3. 定义期望输入

在编写控制台命令时，通常是通过参数和选项来收集用户输入的。Laravel 让你可以非常方便地在 signature 属性中定义你期望用户输入的内容。signature 属性允许使用单一且可读性高，类似路由的语法来定义命令的名称、参数和选项。

5.1.3.1. 参数

所有用户提供的参数及选项都被包含在花括号中。在下面的例子中，这个命令定义了一个必须的参数：user。

```
/**
```

```
 * 命令行的名称及签名
 *
 * @var string
 */
protected $signature = 'email:send {user}';
```

5.1.3.2. 选项

选项类似于参数，是用户输入的另一种格式。在命令行输入选项时，它们以两个连字符（--）作为前缀。有两种类型的选项：接收值和不接收值。不接收值的选项就像是一个布尔值的「开关」。让我们看一下这种类型选项的例子：

```
/**
 * 命令名称及签名
 *
 * @var string
 */
protected $signature = 'email:send {user} {--queue}';
```

在这个例子中，在调用 Artisan 命令时可以指定 --queue 的开关。如果传递了 --queue 选项，该选项的值将会是 true 。否则，其值将会是 false。

5.1.3.3. 输入数组

如果你想定义接收数组输入的参数或选项，你可以使用 * 符号。首先，我们先看一个数组参数的实例。

```
email:send {user*}
```

当调用这个方法的时候，user 参数的输入参数将按顺序传递给命令。例如，以下命令将会设置 user 的值为 foo 和 bar ：

```
php artisan mail:send foo bar
```

5.1.3.4. 输入说明

你可以通过使用冒号将参数名称与描述分隔来为输入参数和选项指定说明。如果你需要一些额外的空间来定义命令，可以将它自由的定义在多行中：

```
/**
 * 控制台命令的名称和签名。
 *
 * @var string
 */
protected $signature = 'mail:send
                        {user : The ID of the user}
                        {--queue : Whether the job should be
queued}';
```

5.1.4. 命令 I/O

5.1.4.1. 检索输入

当命令在执行时，你可能需要访问命令所接受的参数和选项的值。为此，你可以使用 argument 和 option 方法。如果选项或参数不存在，将会返回 null：

```
/**
 * 执行控制台命令。
 *
```

```
 * @return int
 */
public function handle()
{
    $userId = $this->argument('user');


    //
}
```

如果你需要检索所有的参数做为 array，请调用 arguments 方法：

```
$arguments = $this->arguments();
```

5.1.4.2. 交互式输入

除了显示输出以外，你还可以要求用户在执行命令期间提供输入。ask 方法将询问用户指定的问题来接收用户输入，然后用户输入将会传到你的命令中：

```
/**
 * 执行命令指令
 *
 * @return mixed
 */
public function handle()
{
    $name = $this->ask('What is your name?');
}
```

5.1.4.3. 文字输出

你可以使用 line，info，comment，question 和 error 方法，发送输出到控制台。这些方法中的每一个都会使用合适的 ANSI 颜色以展示不同的用途。例如，我们要为用户展示一些常规信息。通常，info 将会以绿色文本在控制台展示。

```
/**
 * 执行控制台命令
 *
 * @return mixed
 */
public function handle()
{
    // ...

    $this->info('The command was successful!');
}
```

输出错误信息，使用 error 方法。错误信息通常使用红色字体显示：

```
$this->error('Something went wrong!');
```

5.1.5. 注册命令

你的所有控制台命令都在你的应用程序的 App\Console\Kernel 类中注册，这是你的应用程序的「控制台内核」。在此类的 commands 方法中，你将看到对内核的 load 方法的调用。 load 方法将扫描 app/Console/Commands 目录并自动将其中包含的每个命令注册到 Artisan。你甚至可以自由地调用 load 方法来扫描其他目录以查找 Artisan 命令：

```
/**
```

```
 * 注册应用程序的命令。
 *
 * @return void
 */
protected function commands()
{
    $this->load(__DIR__.'/Commands');
    $this->load(__DIR__.'/../Domain/Orders/Commands');

    // ...
}
```

如有必要，你可以通过将命令的类名添加到 App\Console\Kernel 类中的 $commands 属性来手动注册命令。如果你的内核上尚未定义此属性，则应手动定义它。当 Artisan 启动时，此属性中列出的所有命令将由 服务容器 解析并注册到 Artisan：

```
protected $commands = [
    Commands\SendEmails::class
];
```

5.1.6. 通过代码执行命令

有时你可能希望在 CLI 之外执行 Artisan 命令。例如，你可能希望从路由或控制器执行 Artisan 命令。你可以使用 Artisan 外观上的 call 方法来完成此操作。 call 方法接受命令的签名名称或类名作为其第一个参数，以及一个命令参数数组作为第二个参数。将返回退出代码：

```
use Illuminate\Support\Facades\Artisan;
```

```
Route::post('/user/{user}/mail', function ($user) {
    $exitCode = Artisan::call('mail:send', [
        'user' => $user, '--queue' => 'default'
    ]);

    //
});
```

或者，你可以将整个 Artisan 命令作为字符串传递给 call 方法：

```
Artisan::call('mail:send 1 --queue=default');
```

5.1.6.1. 从其他命令调用命令

有时你可能希望从现有的 Artisan 命令调用其他命令。你可以使用 call 方法来执行此操作。这个 call 方法接受命令名称和命令参数 / 选项数组：

```
/**
 * 执行控制台命令。
 *
 * @return mixed
 */
public function handle()
{
    $this->call('mail:send', [
        'user' => 1, '--queue' => 'default'
    ]);
```

```
    //
}
```

5.2. 广播

5.2.1. 简介

在现代的 web 应用程序中，WebSockets 被用来实现实时、即时更新用户接口。当服务器上的数据更新后，更新信息会通过 WebSocket 连接发送到客户端等待处理。为了在 UI 中展示出这些数据的变化相比于不停地轮询应用程序，WebSockets 是一种更加可靠和高效的选择。

举一个例子，假设在应用程序中能够导出用户的 CSV 数据并且发送给他们。可是创建 CSV 文件要花费很长时间，因此你选择使用 队列任务去创建文件和发送邮件。当创建了 CSV 文件并将其发送给用户后，我们能够创建使用一个广播事件比如 App\Events\UserDataExported ，然后在应用中利用 JavaScript 来接收这个事件。一旦接收到事件，我们就可以向用户显示一条消息，表明他们的 CSV 已经通过电子邮件发送给他们，而不需要他们刷新页面。

为了帮助您构建这些类型的功能，Laravel 通过 WebSocket 连接使它很容易的去「广播」您的服务端 Laravel 事件。广播 Laravel 事件允许您在服务器端 Laravel 应用程序和客户端 JavaScript 应用程序之间共享相同的事件名称和数据。

广播背后的核心概念很简单：客户端连接到前端的命名频道，而您的 Laravel 应用程序将事件广播到后端的这些频道。这些事件可以包含您希望提供给前端的任何其他数据。

支持的驱动程序

默认情况下，Laravel 包含两个服务器端广播驱动程序供您选择：Pusher Channels 和 Ably。但是，社区驱动的软件包，例如 laravel-websockets 和 soketi 提供 不需要商业广播提供商的其他广播驱动程序。

5.2.2. 服务器安装

要开始使用 Laravel 的事件广播，我们需要在 Laravel 应用程序中进行一些配置以及安装一些包。

事件广播由服务器端广播驱动程序完成，该驱动程序广播您的 Laravel 事件，以便 Laravel Echo（一个 JavaScript 库）可以在浏览器客户端中接收它们。 不用担心 - 我们将逐步完成安装过程的每个部分。

5.2.2.1. 配置

您应用程序的所有事件广播配置都存储在 config/broadcasting.php 配置文件中。 Laravel 支持几个开箱即用的广播驱动程序：Pusher Channels、Redis 和一个用于本地的 log 驱动程序 开发和调试。 此外，还包含一个 null 驱动程序，允许您在测试期间完全禁用广播。 config/broadcasting.php 配置文件中包含每个驱动程序的配置示例。

5.2.2.2. Pusher Channels

如果您计划使用 Pusher Channels 广播您的事件，您应该使用 Composer 包管理器安装 Pusher Channels PHP SDK：

```
composer require pusher/pusher-php-server
```

5.2.3. 客户端安装

5.2.3.1. Pusher Channels

Laravel Echo 是一个 JavaScript 库，可以轻松订阅频道和收听服务器端广播驱动程序广播的事件。 你可以通过 NPM 包管理器安装 Echo。在此示例中，我们还将安装 pusher-js 包，因为我们将使用 Pusher Channels 广播器：

```
npm install --save-dev laravel-echo pusher-js
```

5.2.4. 概念概览

Laravel 的事件广播允许您使用基于驱动程序的 WebSockets 方法将服务器

端 Laravel 事件广播到客户端 JavaScript 应用程序。目前，Laravel 附带 Pusher Channels 和 Ably 驱动程序。 使用 Laravel Echo JavaScript 包可以在客户端轻松使用这些事件。

事件通过「频道」广播，可以指定为公共或私人。您的应用程序的任何访问者都可以订阅公共频道而无需任何身份验证或授权；但是，为了订阅私人频道，用户必须经过身份验证并获得授权才能在该频道上收听。

5.2.4.1. 使用示例程序

在深入了解事件广播的每个组件之前，让我们以电子商务商店为例进行高级概述。

在我们的应用程序中，假设我们有一个页面允许用户查看其订单的运输状态。我们还假设在应用程序处理运输状态更新时触发了一个 OrderShipmentStatusUpdated 事件：

```
use App\Events\OrderShipmentStatusUpdated;

OrderShipmentStatusUpdated::dispatch($order);
```

5.2.5. 定义广播事件

为了通知 Laravel 一个给定的事件应该被广播，你必须在事件类上实现 Illuminate\Contracts\Broadcasting\ShouldBroadcast 接口。该接口已经导入到框架生成的所有事件类中，因此您可以轻松地将其添加到您的任何事件中。

ShouldBroadcast 接口要求你实现一个方法：broadcastOn。 broadcastOn 方法应返回事件应在其上广播的频道或频道数组。 频道应该是 Channel、PrivateChannel 或 PresenceChannel 的实例。

5.2.5.1. 广播名称

默认情况下，Laravel 将使用事件的类名广播事件。但是，您可以通过在事件上定义一个 broadcastAs 方法来自定义广播名称：

```
/**
```

```
 * 事件的广播名称
 *
 * @return string
 */
public function broadcastAs()
{
    return 'server.created';
}
```

5.2.5.2. 广播数据

当一个事件被广播时，它的所有 public 属性都会自动序列化并作为事件的有效负载广播，允许您从 JavaScript 应用程序访问它的任何公共数据。因此，例如，如果您的事件有一个包含 Eloquent 模型的公共 $user 属性，则事件的广播负载将是：

```
{
    "user": {
        "id": 1,
        "name": "Patrick Stewart"
        ...
    }
}
```

但是，如果您希望对广播负载进行更细粒度的控制，您可以在事件中添加一个 broadcastWith 方法。此方法应返回您希望作为事件有效负载广播的数据数组：

```
/**
```

```
 * 获取要广播的数据。
 *
 * @return array
 */
public function broadcastWith()
{
    return ['id' => $this->user->id];
}
```

5.2.5.3. 广播队列

默认情况下，每个广播事件都放置在 queue.php 配置文件中指定的默认队列连接的默认队列中。 您可以通过在事件类上定义 connection 和 queue 属性来自定义广播者使用的队列连接和名称：

```
/**
* 事件被推送的队列名称.
*
* @var string
*/
public $broadcastQueue = 'your-queue-name';
```

如果您想要使用 sync 队列而不是默认的队列驱动来广播事件，可以实现 ShouldBroadcastNow 接口来取代 ShouldBroadcast：

```
/**
 * 广播事件时使用的队列连接的名称。
 *
```

```
 * @var string
 */
public $connection = 'redis';


/**
 * 放置广播作业的队列的名称。
 *
 * @var string
 */
public $queue = 'default';
```

5.2.5.4. 广播条件

有时候你需要在给定条件为 true 的时候才广播事件，你可以在事件类中增加一个 broadcastWhen 方法去定义这些条件：

```
/**
 * 确定事件是否要被广播
 *
 * @return bool
 */
public function broadcastWhen()
{
    return $this->value > 100;
}
```

5.2.6. 授权频道

私人频道要求您授权当前经过身份验证的用户可以实际收听频道。这是通过使用频道名称向您的 Laravel 应用程序发出 HTTP 请求并允许您的应用程序确定用户是否可以在该频道上收听来实现的。 使用 Laravel Echo 时，会自动发出授权订阅私人频道的 HTTP 请求；但是，您确实需要定义正确的路由来响应这些请求。

5.2.6.1. 定义授权路由

值得庆幸的是，Laravel 可以轻松定义响应通道授权请求的路由。在 Laravel 应用程序包含的 App\Providers\BroadcastServiceProvider 中，您将看到对 Broadcast::routes 方法的调用。此方法将注册 /broadcasting/auth 路由来处理授权请求：

```
Broadcast::routes();
```

5.2.6.2. 定义授权回调

接下来，我们需要定义实际确定当前经过身份验证的用户是否可以收听给定频道的逻辑。这是在应用程序中包含的 routes/channels.php 文件中完成的。在这个文件中，你可以使用 Broadcast::channel 方法注册频道授权回调：

```
Broadcast::channel('orders.{orderId}', function ($user, $orderId) {
    return $user->id === Order::findOrNew($orderId)->user_id;
});
```

channel 方法接受两个参数：频道名称和返回 true 或 false 指示用户是否有权收听频道的回调。

所有授权回调都接收当前经过身份验证的用户作为其第一个参数，并将任何其他通配符参数作为其后续参数。在此示例中，我们使用 {orderId} 占位符来指示频道名称的「ID」部分是通配符。

5.2.6.3. 定义频道类

如果您的应用程序使用许多不同的频道，您的 routes/channels.php 文件可能会变得庞大。因此，您可以使用通道类，而不是使用闭包来授权通道。要生成通道类，请使用 make:channel Artisan 命令。此命令将在 App/Broadcasting 目录中放置一个新的频道类。

```
php artisan make:channel
```

5.2.7. 广播事件

一旦你定义了一个事件并用 ShouldBroadcast 接口标记了它，你只需要使用事件的 dispatch 方法来触发事件。事件调度器会注意到该事件被标记为 ShouldBroadcast 接口，并将该事件排队以进行广播：

```
use App\Events\OrderShipmentStatusUpdated;

OrderShipmentStatusUpdated::dispatch($order);
```

5.2.7.1. 只广播给他人

在构建使用事件广播的应用程序时，您可能偶尔需要将事件广播给给定频道的所有订阅者，但当前用户除外。 您可以使用 broadcast 帮助程序和 toOthers 方法来完成此操作：

```
use App\Events\OrderShipmentStatusUpdated;

broadcast(new OrderShipmentStatusUpdated($update))->toOthers();
```

为了更好地理解何时需要使用 toOthers 方法，让我们想象一个任务列表应用程序，其中用户可以通过输入任务名称来创建新任务。要创建任务，您的应用程序可能会向 /task URL 发出请求，该 URL 会广播任务的创建并返回新任务的 JSON 表示。当您的 JavaScript 应用程序接收到来自端点的响应时，它可能会直接将新任务插入到其任务列表中，如下所示：

```
axios.post('/task', task)

    .then((response) => {
```

```
        this.tasks.push(response.data);
    });
```

但是，请记住，我们还广播了任务的创建。如果您的 JavaScript 应用程序也在侦听此事件以将任务添加到任务列表中，那么您的列表中将有重复的任务：一个来自端点，一个来自广播。您可以通过使用 toOthers 方法来指示广播者不要将事件广播给当前用户来解决此问题。

5.2.8. 接收广播

5.2.8.1. 事件监听

一旦你安装并实例化了 Laravel Echo，你就可以开始监听从你的 Laravel 应用程序广播的事件了。首先，使用 channel 方法获取通道实例，然后调用 listen 方法监听指定事件：

```
Echo.channel('orders')
    .listen('OrderShipped', (e) => {
        console.log(e.order.name);
    });
```

如果您想在私人频道上监听事件，请改用 private 方法。 您可以继续调用 listen 方法以在单个通道上侦听多个事件：

```
Echo.private(`orders.${this.order.id}`)
    .listen(...)
    .listen(...)
    .listen(...);
```

5.2.8.2. 退出频道

要退出频道，你可以在 Echo 实例上调用 leaveChannel 方法：

```
Echo.leaveChannel(`orders.${this.order.id}`);
```

5.2.8.3. 命令空间

您可能已经注意到，在上面的示例中，我们没有为事件类指定完整的 App\Events 命名空间。这是因为 Echo 会自动假定事件位于 App\Events 命名空间中。但是，您可以在实例化 Echo 时通过传递 namespace 配置选项来配置根命名空间：

```
window.Echo = new Echo({
    broadcaster: 'pusher',
    // ...
    namespace: 'App.Other.Namespace'
});
```

5.2.9. Presence 频道

Presence 频道建立在私有频道安全性的基础上，同时公开了知道谁订阅了频道的附加功能。这使得构建强大的协作应用程序功能变得容易，例如在另一个用户查看同一页面时通知用户或列出聊天室的居民。

5.2.9.1. 授权 Presence 频道

所有在线频道也是私有频道；因此，用户必须 被授权才能访问。但是，在为 Presence 频道定义授权回调时，如果用户有权加入该频道，则不会返回 true。 相反，你应该返回有关用户的数据数组。

5.2.9.2. 加入 Presence 频道

加入 Presence 频道，你可以使用 Echo 的 join 方法。 join 方法将返回一个 PresenceChannel 实现，它与 listen 方法一起展示，允许你订阅 here，joining 和 leaving 事件。

5.2.9.3. 广播到 Presence 频道

Presence 频道可以像公共或私私有频道一样接收事件。 使用聊天室的示例，我们可能希望将 NewMessage 事件广播到房间的 Presence 频道。

5.2.10. 模型广播

在创建、更新或删除应用程序的 Eloquent 模型 时广播事件是很常见的。当然，这可以通过手动为 Eloquent 模型状态更改定义自定义事件 并使用 ShouldBroadcast 接口标记这些事件来轻松完成。

但是，如果您在应用程序中没有将这些事件用于任何其他目的，则仅出于广播它们的目的而创建事件类可能会很麻烦。为了解决这个问题，Laravel 允许您指示 Eloquent 模型应该自动广播其状态更改。

5.2.10.1.广播约定模型

广播约定

您可能已经注意到，上面模型示例中的 broadcastOn 方法没有返回 Channel 实例。相反，直接返回了 Eloquent 模型。如果模型的 broadcastOn 方法返回 Eloquent 模型实例（或包含在该方法返回的数组中），Laravel 将使用模型的类名和主键标识符作为通道自动为模型实例化一个私有通道实例 姓名。

因此，id 为 1 的 App\Models\User 模型将被转换为名称为 App.Models.User.1 的 Illuminate\Broadcasting\PrivateChannel 实例。当然，除了从模型的 broadcastOn 方法返回 Eloquent 模型实例之外，您还可以返回完整的 Channel 实例，以便完全控制模型的通道名称：

use Illuminate\Broadcasting\PrivateChannel;

```
/**
 * 获取模型事件应该广播的频道。
 *
 * @param  string  $event
 * @return \Illuminate\Broadcasting\Channel|array
```

```
 */
public function broadcastOn($event)
{
    return [new PrivateChannel('user.'.$this->id)];
}
```

5.2.10.2.监听模型广播

一旦你为你的模型添加了 BroadcastsEvents 特征并定义了模型的 broadcastOn 方法，你就可以开始在客户端应用程序中监听广播的模型事件了。在开始之前，您可能希望查阅有关 listening for events 的完整文档。

5.2.11. 客户端事件

有时您可能希望将事件广播给其他连接的客户端，而根本不影响您的 Laravel 应用程序。这对于诸如「键入」通知之类的事情特别有用，您希望提醒应用程序的用户另一个用户正在给定屏幕上键入消息。

5.2.12. 消息通知

通过将事件广播与 notifications 配对，您的 JavaScript 应用程序可以在新通知发生时接收它们，而无需刷新页面。在开始之前，请务必阅读有关使用 [广播通知频道] 的文档（/docs/laravel/9.x/notifications#broadcast-notifications）。

一旦你配置了一个使用广播频道的通知，你就可以使用 Echo 的 notification 方法来监听广播事件。请记住，通道名称应与接收通知的实体的类名称匹配：

```
Echo.private(`App.User.${userId}`)
    .notification((notification) => {
        console.log(notification.type);
    });
```

5.3. 缓存

5.3.1. 简介

在某些应用中，一些查询数据或处理任务的操作会在某段时间里短时间内大量进行，或是一个操作花费好几秒钟。当出现这种情况时，通常会将检索到的数据缓存起来，从而为后面请求同一数据的请求迅速返回结果。这些缓存数据通常会储存在极快的存储系统中，例如 Memcached 和 Redis。

Laravel 为各种缓存后端提供了富有表现力且统一的 API，以便你利用它们极快的查询数据来加快你的应用。

5.3.2. 配置

缓存配置文件位于 config/cache.php。在这个文件中，你可以指定应用默认使用哪个缓存驱动。Laravel 支持的缓存后端包括 Memcached、Redis、DynamoDB，以及现成的关系型数据库。此外，还支持基于文件的缓存驱动，以及方便自动化测试的缓存驱动 array 和 null 。

缓存配置文件还包含文件中记录的各种其他选项，因此请务必阅读这些选项。默认情况下，Laravel 配置为使用 file 缓存驱动程序，它将序列化的缓存对象存储在服务器的文件系统上。对于较大的应用程序，建议您使用更健壮的驱动程序，例如 Memcached 或 Redis。您甚至可以为同一个驱动程序配置多个缓存配置。

5.3.2.1. 驱动的前提条件

数据库

当使用 database 缓存驱动时，你需要配置一个表来存放缓存数据。下面是构建缓存数据表结构的 Schema 声明示例：

```
Schema::create('cache', function ($table) {
    $table->string('key')->unique();
    $table->text('value');
    $table->integer('expiration');
});
```

Memcached

使用 Memcached 驱动需要安装 Memcached PECL 扩展包 。你可以把所有的 Memcached 服务器都列在 config/cache.php 配置文件中：

```
'memcached' => [
    [
        'host' => '127.0.0.1',
        'port' => 11211,
        'weight' => 100
    ],
],
```

5.3.3. 缓存使用

5.3.3.1. 获取缓存实例

要获取缓存存储实例，您可以使用 Cache Facade，我们将在本文档中使用它。Cache Facade 提供了对 Laravel 缓存合约底层实现的方便、简洁的访问：

```
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\Cache;

class UserController extends Controller
{
    /**
     * 显示应用程序的所有用户的列表。
     *
```

```
     * @return Response
     */
    public function index()
    {
        $value = Cache::get('key');

        //
    }
}
```

5.3.3.2. 从缓存中获取数据

Cache 门面的 get 方法用于从缓存中获取缓存项，如果缓存项不存在，则返回 null。如果需要的话您可以传递第二个参数到 get 方法指定不存在时的自定义默认值。

```
$value = Cache::get('key');

$value = Cache::get('key', 'default');
```

您甚至可以传递一个闭包作为默认值，如果缓存项不存在的话闭包的结果将会被返回。

```
$value = Cache::get('key', function() {

return DB::table(...)->get();

});
```

5.3.3.3. 在缓存中存储数据

您可以使用 Cache 门面上的 put 方法在缓存中存储数据。

```
Cache::put('key', 'value', $seconds);
```

如果没有传递缓存时间到 put 方法，则缓存存永久有效。

5.3.3.4. 从缓存中移移除数据

您可以使用 Cache 门面上的 forget 方法从缓存中移除项数据。

```
Cache::forget('key');
```

还可以通过设置缓存有效期为 0 或负数来移除缓存项。

5.3.3.5. 缓存辅助函数

除了可以使用 Cache 的门面以及 Cache 契约 外，你也可以使用全局辅助函数 cache 来获取和保存缓存数据。当只使用一个字符串参数调用 cache 函数时，这将返回给定键对应的值。

```
$value = cache('key');
```

5.3.4. 缓存标签

5.3.4.1. 存储缓存标签

缓存标签允许您在缓存中标记相关项目，然后刷新所有已分配给定标签的缓存值。您可以通过传入标记名称的有序数组来访问标记缓存。例如，让我们访问一个标记的缓存并将一个值「put」缓存中：

```
Cache::tags(['people', 'artists'])->put('John', $john, $seconds);

Cache::tags(['people', 'authors'])->put('Anne', $anne, $seconds);
```

5.3.4.2. 访问缓存标签

要检索标记的缓存项，请将相同的有序标签列表传递给 tags 方法，然后使用您要检索的键调用 get 方法：

```
$john = Cache::tags(['people', 'artists'])->get('John');
```

```
$anne = Cache::tags(['people', 'authors'])->get('Anne');
```

5.3.4.3. 移除缓存标签

您可以刷新所有分配了标签或标签列表的项目。例如，此语句将删除所有标记为「people」、「authors」或两者的缓存。 因此，Anne 和 John 都将从缓存中删除：

```
Cache::tags(['people', 'authors'])->flush();
```

5.3.5. 原子锁

5.3.5.1. 驱动程序先决条件

数据库

使用 “数据库” 缓存驱动程序时，您需要设置一个表来包含应用程序的缓存锁。您将在下表中找到一个示例 Schema 声明：

```
Schema::create('cache_locks', function ($table) {
    $table->string('key')->primary();
    $table->string('owner');
    $table->integer('expiration');
});
```

5.3.6. 添加自定义缓存驱动

5.3.6.1. 编写驱动

要创建我们的自定义缓存驱动程序，我们首先需要实现 Illuminate\Contracts\Cache\Store contract。 因此，MongoDB 缓存实现可能如下所示：

```
<?php
```

```
namespace App\Extensions;

use Illuminate\Contracts\Cache\Store;

class MongoStore implements Store
{

    public function get($key) {}

    public function many(array $keys);

    public function put($key, $value, $seconds) {}

    public function putMany(array $values, $seconds);

    public function increment($key, $value = 1) {}

    public function decrement($key, $value = 1) {}

    public function forever($key, $value) {}

    public function forget($key) {}

    public function flush() {}

    public function getPrefix() {}

}
```

我们只需要使用 MongoDB 连接来实现这些方法中的每一个。有关如何实现这些方法的示例，请查看 Laravel 框架源代码 中的 Illuminate\Cache\MemcachedStore。 一旦我们的实现完成，我们可以通过调用 Cache 门面的 extend 方法来完成我们的自定义驱动程序注册：

```
Cache::extend('mongo', function($app) {

    return Cache::repository(new MongoStore);

});
```

5.3.6.2. 注册驱动

要向 Laravel 注册自定义缓存驱动程序，我们将使用 Cache 门面的 extend 方法。

```
<?php

namespace App\Providers;

use App\Extensions\MongoStore;

use Illuminate\Support\Facades\Cache;

use Illuminate\Support\ServiceProvider;


class CacheServiceProvider extends ServiceProvider

{

    /**

    * Perform post-registration booting of services.

    *

    * @return void

    * @translator laravelacademy.org

    */

    public function boot()

    {

        Cache::extend('mongo', function($app) {

            return Cache::repository(new MongoStore);

        });

    }

    /**

    * Register bindings in the container.
```

```
    *
    * @return void
    */
    public function register()
    {
        //
    }
}
```

传递给 extend 方法的第一个参数是驱动名称。

5.3.6.3. 缓存事件

要在每个缓存操作上执行代码，您可以侦听缓存触发的 events。 通常，您应该将这些事件侦听器放在应用程序的 App\Providers\EventServiceProvider 类中：

```
/**
* The event listener mappings for the application.
*
* @var array
*/
protected $listen = [
    'Illuminate\Cache\Events\CacheHit' => [
    'App\Listeners\LogCacheHit',
],
'Illuminate\Cache\Events\CacheMissed' => [
    'App\Listeners\LogCacheMissed',
],
```

```
'Illuminate\Cache\Events\KeyForgotten' => [
    'App\Listeners\LogKeyForgotten',
],
'Illuminate\Cache\Events\KeyWritten' => [
    'App\Listeners\LogKeyWritten',
],
];
```

5.4. 集合

5.4.1. 简介

Illuminate\Support\Collection 类为处理数据数组提供了一个流畅、方便的包装器。例如，查看以下代码。我们将使用 collect 助手从数组中创建一个新的集合实例，对每个元素运行 strtoupper 函数，然后删除所有空元素：

```
$collection = collect(['taylor', 'abigail', null])->map(function
($name) {
    return strtoupper($name);
    })->reject(function ($name) {
    return empty($name);
});
```

如你所见，Collection 类允许你链接其方法以执行流畅的映射和减少底层数组。一般来说，集合是不可变的，这意味着每个 Collection 方法都会返回一个全新的 Collection 实例。

5.4.1.1. 创建集合

如上所述，collect 帮助器为给定数组返回一个新的 Illuminate\Support\Collection 实例。因此，创建一个集合非常简单：

```
$collection = collect([1, 2, 3]);
```

5.4.1.2. 扩展集合

集合是「可宏化的」，它允许你在运行时向 Collection 类添加其他方法。Illuminate\Support\Collection 类的 macro 方法接受一个闭包，该闭包将在调用宏时执行。宏闭包可以通过 $this 访问集合的其他方法，就像它是集合类的真实方法一样。例如，以下代码在 Collection 类中添加了 toUpper 方法：

```
use Illuminate\Support\Collection;
use Illuminate\Support\Str;

Collection::macro('toUpper', function () {
    return $this->map(function ($value) {
        return Str::upper($value);
    });
});

$collection = collect(['first', 'second']);
$upper = $collection->toUpper();
// ['FIRST', 'SECOND']
```

通常，你应该在 service provider 的 boot 方法中声明集合宏。

5.4.2. 集合方法

所对于剩余的大部分集合文档，我们将讨论 Collection 类中可用的每个方法。请记住，所有这些方法都可以链式调用，以便流畅地操作底层数组。

5.4.3. 方法列表

All 方法返回集合表示的底层数组。

Avg 方法返回所有集合项的平均值。

5.4.4. 高阶消息传递

集合还支持“高阶消息传递”，也就是在集合上执行通用的功能，支持高阶消息传递的方法包括。

5.4.5. 懒性集合

5.4.5.1. 介绍

为了补充已经强大的 Collection 类，LazyCollection 类利用 PHP 的 generators 允许你使用非常 大型数据集，同时保持较低的内存使用率。

5.5. 契约

5.5.1. 简介

Laravel 的 Contract 是一组接口，它们定义由框架提供的核心服务。例如，illuste\Contracts\Queue\Queue Contract 定义了队列所需的方法，而 illuste\Contracts\Mail\Mailer Contract 定义了发送邮件所需的方法。

每个契约都有由框架提供的相应实现。例如，Laravel 提供了一个支持各种驱动的队列实现，还有一个由 SwiftMailer 提供支持的邮件程序实现等等。

所有的 Laravel Contract 都存在于它们各自的 GitHub 仓库。这为所有可用的契约提供了一个快速的参考点，以及一个可以被包开发人员使用的独立的包。

5.5.2. 何时使用契约

使用 Contract 或 Facades 取决于个人喜好和开发团队的喜好。Contract 和 Facade 均可用于创建功能强大且经过良好测试的 Laravel 应用。Contract 和 Facade 并不是一道单选题，你可以在同一个应用内同时使用 Contract 和 Facade。

只要聚焦在类的职责应该单一上，您会发现 Contract 和 Facade 的实际差异其实很小。

通常情况下，大部分使用 Facade 的应用都不会在开发中遇到问题。但如果你在建立一个可以由多个 PHP 框架使用的扩展包，你可能会希望使用 illuminate/contracts 扩展包来定义该包和 Laravel 集成，而不需要引入完整的 Laravel 实现。

5.5.3. 如何使用契约

那么，如何实现契约呢？它其实很简单。

Laravel 中的许多类都是通过 服务容器 解析的，包括控制器、事件侦听器、中间件、队列任务，甚至路由闭包。因此，要实现契约，你只需要在被解析的类的构造函数中「类型提示」接口。

5.6. 事件

5.6.1. 简介

Laravel 的事件系统提供了一个简单的观察者模式的实现，允许你能够订阅和监听在你的应用中的发生的各种事件。事件类一般来说存储在 app/Events 目录，监听者的类存储在 app/Listeners 目录。不要担心在你的应用中没有看到这两个目录，因为通过 Artisan 命令行来创建事件和监听者的时候目录会同时被创建。

事件系统可以作为一个非常棒的方式来解耦你的系统的方方面面，因为一个事件可以有多个完全不相关的监听者。例如，你希望每当有订单发出的时候都给你发送一个 Slack 通知。你大可不必将你的处理订单的代码和发送 slack 消息的代码放在一起，你只需要触发一个 App\Events\OrderShipped 事件，然后事件监听者可以收到这个事件然后发送 slack 通知。

5.6.2. 注册事件/监听器

在系统的服务提供者 App\Providers\EventServiceProvider 中提供了一个简单的方式来注册你所有的事件监听者。属性 listen 包含所有的事件 (作为键) 和对应的监听器 (值)。你可以添加任意多系统需要的监听器在这个数组中，让我们添加一个 OrderShipped 事件：

```
use App\Events\OrderShipped;
use App\Listeners\SendShipmentNotification;


/**
 * 系统中的事件和监听器的对应关系。
 *
 * @var array
 */
protected $listen = [
    OrderShipped::class => [
        SendShipmentNotification::class,
    ],
];
```

5.6.2.1. 生成事件和监听器

当然，为每个事件和监听器手动创建文件是很麻烦的。相反，将监听器和事件添加到 EventServiceProvider 并使用 event:generate Artisan 命令。此命令将生成 EventServiceProvider 中列出的、尚不存在的任何事件或侦听器：

```
php artisan event:generate
```

5.6.2.2. 手动注册事件

通常，事件应该通过 EventServiceProvider $listen 数组注册；但是，你也可以在 EventServiceProvider 的 boot 方法中手动注册基于类或闭包的事件监听器：

```
use App\Events\PodcastProcessed;
use App\Listeners\SendPodcastNotification;
use Illuminate\Support\Facades\Event;

/**
 * 注册任意的其他事件和监听器。
 *
 * @return void
 */
public function boot()
{
    Event::listen(
        PodcastProcessed::class,
        [SendPodcastNotification::class, 'handle']
    );

    Event::listen(function (PodcastProcessed $event) {
        //
    });
}
```

5.6.2.3. 事件发现

你可以启用自动事件发现，而不是在 EventServiceProvider 的 $listen 数组中手动的注册事件和监听器。当事件发现启用，Laravel 将自动发现和注册你的事件和监听器扫描你的应用程序的 Listeners 目录。此外，在 EventServiceProvider 中列出的任何显式定义的事件仍将被注册。

5.6.3. 定义事件

事件类本质上是一个数据容器，它保存与事件相关的信息。例如，让我们假设一个 App\Events\OrderShipped 事件接收到一个 Eloquent ORM 对象：

```
<?php

namespace App\Events;

use App\Models\Order;

use Illuminate\Broadcasting\InteractsWithSockets;

use Illuminate\Foundation\Events\Dispatchable;

use Illuminate\Queue\SerializesModels;


class OrderShipped

{

    use Dispatchable, InteractsWithSockets, SerializesModels;


    public $order;


    /**

     * 创建一个事件实例

     *
```

```
     * @param  \App\Models\Order  $order
     * @return void
     */
    public function __construct(Order $order)
    {
        $this->order = $order;
    }
}
```

如您所见，这个事件类不包含逻辑。它是一个被购买的 App\Models\Order 实例容器。 如果事件对象是使用 PHP 的 SerializesModels 函数序列化的，事件使用的 SerializesModels trait 将会优雅地序列化任何 Eloquent 模型，比如在使用队列侦听器。

5.6.4. 定义监听器

接下来，让我们看一下示例事件的侦听器。事件监听器在其 handle 方法中接收事件实例。 artisan 命令 event:generate 和 make:listener 会自动导入正确的事件类，并在 handle 方法中注入提示事件。

5.6.5. 事件监听器队列

如果侦听器执行缓慢的任务如发送电子邮件或发出 HTTP 请求，你可以将任务丢给队列处理。在开始使用队列监听器之前，请确保在你的服务器或者本地开发环境中能够 配置队列 并启动一个队列监听器。

5.6.5.1. 手动访问队列

如果你需要手动访问监听器下面队列任务的 delete 和 release 方法，可以使用 Illuminate\Queue\InteractsWithQueuetrait 进行访问。

5.6.5.2. 处理失败任务

有时，你的事件监听器队列可能会失败。如果监听器的队列任务超过了队列中定义的最大尝试次数，则会在监听器上调用 failed 方法。

5.6.6. 调度事件

要分派一个事件，你可以在事件上调用静态的 dispatch 方法。这个方法是通过 Illuminate\Foundation\Events\Dispatchable 特性提供给事件的。

5.6.7. 事件订阅者

5.6.7.1. 构建事件订阅者

事件订阅者是可以从订阅者类本身中订阅多个事件的类，允许你在单个类中定义多个事件处理程序。订阅者应该定义一个 subscribe 方法，它将被传递一个事件分派器实例。

5.6.7.2. 注册事件订阅者

当编写完订阅者后，你已经准备好为事件分发器注册它们了。你可以使用 EventServiceProvider 上的 $subscribe 属性来注册订阅者。

5.7. 文件存储

5.7.1. 简介

Laravel 提供了一个强大的文件系统抽象概念，这得益于 Frank de Jonge 强大的 Flysystem 扩展包。Laravel 文件系统集成为使用本地文件系统、SFTP 和 Amazon S3 提供了简单易用的驱动程序。 更棒的是，由于每个系统的 API 保持不变，所以在这些存储选项之间切换是非常简单的。

5.7.2. 配置

Laravel 文件系统的配置文件位于 config/filesystems.php。在这个文件中你可以配置所有的「磁盘」。每个磁盘代表特定的存储驱动及存储位置。每种支持的驱动程序的示例配置都包含在配置文件中。因此，只需要修改配置即可应用你的存储偏好和凭据。

local 驱动用于操作本地服务器的文件，而 s3 驱动用于操作在 Amazon S3 云存储服务上的文件。

5.7.2.1. 本地驱动

使用 local 驱动时，所有文件操作都与 filesystems 配置文件中定义的 root 目录相关。 默认情况下，此值设置为 storage/app 目录。因此，以下方法会把文件存储在 storage/app/example.txt 中：

```
use Illuminate\Support\Facades\Storage;

Storage::disk('local')->put('example.txt', 'Contents');
```

5.7.2.2. 公共磁盘

在 filesystems 配置文件中定义的 public 磁盘适用于要公开访问的文件。默认情况下， public 磁盘使用 local 驱动，并且将这些文件存储在 storage/app/public 目录下。

为了让它们能通过网络访问，你需要创建从 public/storage 到 storage/app/public 的符号链接。这种方式能把可公开访问文件都保留在同一个目录下，以便在使用零停机时间部署系统如 Envoyer 的时候，就可以轻松地在不同的部署之间共享这些文件。

5.7.2.3. 驱动要求

S3 驱动配置

在使用 S3 驱动之前，你需要通过 Composer 安装相应的软件包：

```
composer require -W league/flysystem-aws-s3-v3 "^3.0"
```

5.7.3. 获取磁盘实例

Storage Facade 可用于与所有已配置的磁盘进行交互。例如，你可以使用 Facade 中的 put 方法将头像存储到默认磁盘。如果你使用 Storage Facade 时并没有使用 disk 方法，那么所有的方法调用将会自动传递给默认的磁盘：

```
use Illuminate\Support\Facades\Storage;
```

```
Storage::put('avatars/1', $content);
```

5.7.3.1. 按需配置磁盘

有时你可能希望在运行时使用给定配置创建磁盘，而该配置实际上不存在于应用程序的「文件系统」配置文件中。

5.7.4. 检索文件

get 方法可以用于检索文件的内容，此方法返回该文件的原始字符串内容。切记，所有文件路径的指定都应该相对于为磁盘配置的「root」目录：

```
$contents = Storage::get('file.jpg');
```

5.7.4.1. 下载文件

download 方法可用于生成响应，该响应强制用户的浏览器在给定路径下下载文件。download 方法接受文件名作为该方法的第二个参数，它将确定下载文件的用户看到的文件名。最后，你可以将 HTTP 标头数组作为该方法的第三个参数传递：

```
return Storage::download('file.jpg');

return Storage::download('file.jpg', $name, $headers);
```

5.7.4.2. 文件 URLs

你可以使用 url 方法来获取给定文件的 URL。如果你使用的是 local 驱动，一般只是在给定的路径上加上 /storage 并返回一个相对的 URL 到那个文件。如果使用的是 s3 或者是 rackspace 驱动，会返回完整的远程 URL：

```
use Illuminate\Support\Facades\Storage;

$url = Storage::url('file.jpg');
```

5.7.4.3. 文件元数据

除了读写文件外，Laravel 还可以提供有关文件本身的信息，例如，size 方法可用来获取文件的大小（以字节为单位）：

```
use Illuminate\Support\Facades\Storage;

$size = Storage::size('file.jpg');
```

5.7.5. 存储文件

put 方法可用于将原始文件内容保存到磁盘上。你也可以传递 PHP 的 resource 给 put 方法，它将使用文件系统下的底层流支持。强烈建议在处理大文件时使用此方法：

```
use Illuminate\Support\Facades\Storage;

Storage::put('file.jpg', $contents);

Storage::put('file.jpg', $resource);
```

5.7.5.1. 文件上传

在 web 应用程序中，最常用到的文件存储的场景的地方就是上传头像，照片和文件。Laravel 上传文件的实例方法 store 可以轻松的处理文件上传存储问题。你只需要调用带有文件保存路径作为参数的 store 方法即可：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use Illuminate\Http\Request;


class UserAvatarController extends Controller

{
```

```
    /**
     * 更新用户头像
     *
     * @param  Request  $request
     * @return Response
     */
    public function update(Request $request)
    {
        $path = $request->file('avatar')->store('avatars');

        return $path;
    }
}
```

5.7.5.2. 文件可见性

在 Laravel 集成的文件系统中，「可见性」是一个对多个平台的文件权限的抽象概念。文件可以声明为 public 或 private。如果一个文件被声明为 public，意味着其他人可以访问。例如，使用 S3 驱动时，就可以检索声明为 public 的文件。

你可以通过 put 方法设置文件的可见性：

```
use Illuminate\Support\Facades\Storage;

Storage::put('file.jpg', $contents, 'public');
```

5.7.6. 删除文件

delete 方法接收一个文件名或一个文件名数组来将其从磁盘中删除：

```
use Illuminate\Support\Facades\Storage;

Storage::delete('file.jpg');

Storage::delete(['file.jpg', 'file2.jpg']);
```

5.7.7. 目录

获取目录下的所有的文件

files 方法返回指定目录下的所有文件。如果你想检索指定目录（包括子目录）中所有的文件列表，可以使用 allFiles 方法：

```
use Illuminate\Support\Facades\Storage;

$files = Storage::files($directory);

$files = Storage::allFiles($directory);
```

5.7.8. 自定义文件系统

Laravel 内置的文件系统提供了一些开箱即用的驱动；当然，它不仅仅是这些，它还提供了与其他存储系统的适配器。通过这些适配器，你可以在你的 Laravel 应用中创建自定义驱动。

5.8. 辅助函数

5.8.1. 简介

Laravel 包含各种各样的全局 PHP 「辅助」函数，框架本身也大量的使用了这些功能函数；如果你觉的方便，你可以在你的应用中任意使用这些函数。

5.8.2. 方法列表

Arr::accessible() {.collection-method .first-collection-method}

Arr::accessible 函数检查给定的值是否可数组式访问：

5.9. HTTP 客户端

5.9.1. 简介

Laravel 为 Guzzle HTTP 客户端 提供了一套语义化且轻量的 API，让你可用快速地使用 HTTP 请求与其他 Web 应用进行通信。该 API 专注于其在常见用例中的快速实现以及良好的开发者体验。

在开始之前，你需要确保你的项目已经安装了 Guzzle 包作为依赖项。默认情况下，Laravel 已经包含了 Guzzle 包。但如果你此前手动移除了它，你也可以通过 Composer 重新安装它：

```
composer require guzzlehttp/guzzle
```

5.9.2. 创建请求

你可以通过 get、 post、 put、 patch 和 delete 方法来创建请求。首先，让我们先看一下如何发出一个基础的 GET 请求：

```
use Illuminate\Support\Facades\Http;

$response = Http::get('http://example.com');
```

get 方法返回一个 Illuminate\Http\Client\Response 的实例，该实例提供了大量的方法来检查请求的响应：

```
$response->body() : string;

$response->json($key = null) : array|mixed;
```

```
$response->object() : object;

$response->collect($key = null) : Illuminate\Support\Collection;

$response->status() : int;

$response->ok() : bool;

$response->successful() : bool;

$response->redirect(): bool;

$response->failed() : bool;

$response->serverError() : bool;

$response->clientError() : bool;

$response->header($header) : string;

$response->headers() : array;
```

5.9.2.1. 请求数据

大多数情况下，POST、 PUT 和 PATCH 携带着额外的请求数据是相当常见的。所以，这些方法的第二个参数接受一个包含着请求数据的数组。默认情况下，这些数据会使用 application/json 类型随请求发送。

```
$response = Http::post('http://example.com/users', [
    'name' => 'Steve',
    'role' => 'Network Administrator',
]);
```

5.9.2.2. 请求头

你可以通过 withHeaders 方法添加请求头。该方法接受一个数组格式的键值对。

```
$response = Http::withHeaders([
    'X-First' => 'foo',
    'X-Second' => 'bar'
])->post('http://example.com/users', [
    'name' => 'Taylor',
]);
```

5.9.2.3. 认证

你可以使用 withBasicAuth 和 withDigestAuth 方法来分别指定使用 Basic 或是 Digest 认证方式：

```
// Basic 认证...
$response = Http::withBasicAuth('taylor@laravel.com', 'secret')->post(...);

// Digest 认证...
$response = Http::withDigestAuth('taylor@laravel.com', 'secret')->post(...);
```

5.9.2.4. 超时

该 timeout 方法用于指定响应的最大等待秒数：

```
$response = Http::timeout(3)->get(...);
```

5.9.2.5. 重试

如果你希望你的 HTTP 客户端在发生错误时自动重新发送请求，你可以使用

retry 方法。该方法接受两个参数：重新尝试次数以及重试等待时间（毫秒）：

```
$response = Http::retry(3, 100)->post(...);
```

5.9.2.6. 错误处理

跟 Guzzle 的默认行为不同，Laravel HTTP 客户端并不会在客户端或服务端错误时抛出异常（400 及 500 状态码）。你可以通过 successful、 clientError 或是 serverError 方法来判断是否发生错误：

```
// 确认状态码是否在 200 到 300 之间（包含 200）
$response->successful();


// 确认是否发生了 400 级别的错误（以 4 开头的状态码）
$response->clientError();


// 确认是否发生了 500 级别的错误（以 5 开头的状态码）
$response->serverError();
```

5.9.3. 测试

许多 Laravel 服务都提供了帮助您轻松且富有表现力地编写测试的功能，Laravel 的 HTTP 包装器也不例外。 Http 门面的 fake 方法允许您指示 HTTP 客户端在发出请求时返回存根 / 虚拟响应。

5.9.3.1. 伪造响应

例如，要指示 HTTP 客户端为每个请求返回空的 200 状态码响应，您可以调用不带参数的 fake 方法：

```
use Illuminate\Support\Facades\Http;

Http::fake();
```

```
$response = Http::post(...);
```

5.9.3.2. 检查请求

在伪造响应时，您可能偶尔希望检查客户端收到的请求，以确保您的应用程序正在发送正确的数据或标头。您可以在调用 Http::fake 后调用 Http::assertSent 方法来完成此操作。

5.9.4. 事件

Laravel 在发送 HTTP 请求的过程中会触发三个事件。 RequestSending 事件在发送请求之前触发，而 ResponseReceived 事件在收到给定请求的响应后触发。 如果没有收到给定请求的响应，则会触发 ConnectionFailed 事件。

5.10. 本地化

5.10.1. 简介

Laravel 的本地化功能提供了一种方便的方法来检索各种语言的字符串，从而使你可以轻松地在应用程序中支持多种语言。

Laravel 提供了两种管理翻译字符串的方法。首先，语言字符串可以存储在 lang 目录里的文件中。在此目录中，可能存在应用程序支持的每种语言的子目录。

5.10.1.1.配置语言环境

应用程序的默认语言存储在 config/app.php 配置文件的 locale 配置选项中。你可以随意修改此值以适合你的应用程序的需求。

5.10.2. 定义翻译字符串

5.10.2.1.使用短键

通常，翻译字符串存储在 lang 目录中的文件中。在这个目录中，应用程序支持的每种语言都应该有一个子目录。

5.10.3. 检索翻译字符串

5.10.3.1. 替换翻译字符串中的参数

如果愿意，可以在翻译字符串中定义占位符。所有占位符的前缀都是 :。

5.10.4. 覆盖扩展包的语言文件

有些包可能随自己的语言文件一起装运。你可以将文件放置在 lang/vendor/{package}/{locale} 目录中，而不是更改扩展包的核心文件来调整这些行。

例如，如果需要重写位于名为 skyrim/hearthfire 的包的 messages.php 文件内容，应将语言文件放在：lang/vendor/hearthfire/en/messages.php 在这个文件中，你应该只定义要覆盖的翻译字符串。任何未重写的翻译字符串仍将从包的原始语言文件中加载。

5.11. 邮件

5.11.1. 简介

发送邮件并不复杂。Laravel 基于 Symfony Mailer 组件提供了一个简洁、简单的邮件 API。Laravel 和 Symfony 为 Mailer SMTP 、Mailgun 、Postmark 、Amazon SES 、 及 sendmail （发送邮件的方式）提供驱动，允许你通过本地或者云服务来快速发送邮件。

5.11.1.1. 配置

Laravel 的邮件服务可以通过 config/mail.php 配置文件进行配置。

邮件中的每一项都在配置文件中有单独的配置项，甚至是独有的「传输方式」，允许你的应用使用不同的邮件服务发送邮件。例如，你的应用程序在使用 Amazon SES 发送批量邮件时，也可以使用 Postmark 发送事务性邮件。

在你的 mail 配置文件中，你将找到 mailers 配置数组。该数组包含 Laravel 支持的每个邮件 驱动程序 / 传输方式 配置，而 default 配置值确定当您的应用

程序需要发送电子邮件时，默认情况下将使用哪个邮件驱动。

5.11.1.2.驱动前提

基于 API 的驱动，如 Mailgun 和 Postmark ，通常比 SMTP 服务器更简单快速。如果可以的话， 我们建议您使用下面这些驱动。

5.11.2. 生成 Mailables

在构建 Laravel 应用程序时，应用程序发送的每种类型的电子邮件都表示为一个 mailable 类。

5.11.3. 编写 Mailables

一旦你生成了一个 mailable 的类，打开它，这样我们就可以探索它的内容了。首先，请注意所有可邮寄类的配置都是在 build 方法中完成的。 在此方法中，您可以调用各种方法，例如 from、subject、view 和 attach 来配置电子邮件的呈现和传递。

5.11.3.1.配置发件人

首先，我们来看一下邮件发件人的配置。

5.11.3.2.配置视图

您可以在可邮寄类的 build 方法中使用 virw 方法来指定渲染邮件内容时使用哪个视图模板，由于每封邮件通常使用 Blade 模板来渲染内容，所以您可以在构建邮件 HTML 时使用 Blade 模板引擎提供的所有功能。

5.11.3.3.视图数据

通常，我们需要传递一些数据到渲染邮件的 HTML 视图以供使用。有两种方式将数据传递到视图，第一种是可邮寄类的公共（public）属性在视图中自动生效。

5.11.3.4.附件

要添加附件以邮件，可以在可邮寄类的 build 方法中使用 attach 方法。

5.11.3.5.内部附件

要嵌入内联图片，在邮件视图中使用$message 变量上的 embed 方法即可。

5.11.3.6.自定义 SwiftMailer 消息

该 Mailable 基类的 withSymfonyMessage 方法允许您注册一个闭包，在发送消息之前将使用 Symfony 消息实例调用该闭包。

5.11.4. Markdonw 格式邮件

Maikdown 邮件消息允许您在可邮寄类中利用预置模板和邮件通知组件。因为这些消息以 Markdown 格式编写，框架还可以为它们渲染出高颜值、响应式的 HTML 模板，同时自动生成纯文本的副本。

5.11.5. 发送邮件

若要发送邮件，使用 Mail 门面 的方法。该 to 方法接受 邮件地址、用户实例或用户集合。如果传递一个对象或者对象集合，mailer 在设置收件人时将自动使用它们的 email 和 name 属性，因此请确保对象的这些属性可用。

5.11.6. 渲染可邮寄类

有时您可能希望捕获邮件的 HTML 内容而不发送它。为此，可以调用邮件类的 render 方法。此方法将以字符串形式返回邮件类的渲染内容:

```
use App\Mail\InvoicePaid;

use App\Models\Invoice;



$invoice = Invoice::find(1);

return (new InvoicePaid($invoice))->render();
```

5.11.7. 本地化邮件

Laravel 允许您在请求的当前语言环境之外的语言环境中发送邮件，如果邮件在排队，它甚至会记住这个语言环境。

5.11.8. 邮件&本地开发

在开发发送电子邮件的应用程序时，您可能不希望实际将电子邮件发送到实时电子邮件地址。 Laravel 提供了几种在本地开发期间「禁用」实际发送电子邮件的方法。

5.11.9. 事件

Laravel 在发送邮件消息的过程中会触发两个事件。MessageSending 事件在消息发送之前触发，而 MessageSent 事件在消息发送后触发。请记住，这些事件是在邮件发送时触发的，而不是在排队时触发的。你可以在你的 App\Providers\EventServiceProvider 服务提供者中为这个事件注册事件监听器：

```
/**
 * 为应用映射事件侦听器。
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Mail\Events\MessageSending' => [
        'App\Listeners\LogSendingMessage',
    ],
    'Illuminate\Mail\Events\MessageSent' => [
        'App\Listeners\LogSentMessage',
    ],
];
```

5.12. 通知

5.12.1. 简介

除了支持 发送邮件 之外，Laravel 还支持通过多种频道发送通知，包括邮件、短信（通过 Vonage，原来叫 Nexmo），以及 Slack。此外，已经创建了各种各样的 社区通知频道，来在十几个不同的频道中发送通知！通知还能存储到数据库以便后续在 Web 页面中显示。

通常，通知应该是简短的信息性消息，用于通知用户应用中发生的事情。例如，如果你正在编写一个账单应用，则可以通过邮件和短信频道向用户发送一个「支付凭证」通知。

5.12.2. 创建通知

Laravel 中，通常每个通知都由一个存储在 app/Notifications 目录下的一个类表示。如果在你的应用中没有看到这个目录，不要担心，当运行 make:notification 命令时它将为您创建：

```
php artisan make:notification InvoicePaid
```

。这个命令会在 app/Notifications 目录下生成一个新的通知类。每个通知类都包含一个 via 方法以及一个或多个消息构建的方法比如 toMail 或 toDatabase，它们会针对特定的渠道把通知转换为对应的消息。

5.12.3. 发送通知

5.12.3.1.使用 Notifiable Trait

通知可以通过两种方式发送： 使用 Notifiable 特性的 notify 方法或使用 Notification 门面。该 Notifiable 默认包含在应用程序的 App\Models\User 模型中：

```
<?php

namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
```

```
use Illuminate\Notifications\Notifiable;


class User extends Authenticatable
{
    use Notifiable;
}
```

此 notify 方法需要接收一个通知实例参数：

```
use App\Notifications\InvoicePaid;
$user->notify(new InvoicePaid($invoice));
```

5.12.3.2.使用 Notification Facade

另外，你可以通过 Notification facade 来发送通知，它主要用在当你需要给多个可接收通知的实体发送的时候，比如给用户集合发送通知。使用 Facade 发送通知的话，要把可接收通知实例和通知实例传递给 send 方法：

```
use Illuminate\Support\Facades\Notification;
Notification::send($users, new InvoicePaid($invoice));
```

5.12.4. 邮件通知

5.12.4.1.格式化邮件消息

如果通知支持以邮件方式发送，你需要在通知类上定义一个 toMail 方法。该方法会接收一个 $notifiable 实体并返回 Illuminate\Notifications\Messages\MailMessage 实例。邮件消息可以包含多行文本以及对「动作的调用」。 让我们来看一个 toMail 方法的示例:

```
/**
 * 获取通知对应的邮件.
```

```
     *
     * @param  mixed  $notifiable
     * @return \Illuminate\Notifications\Messages\MailMessage
     */
    public function toMail($notifiable)
    {
        $url = url('/invoice/'.$this->invoice->id);

        return (new MailMessage)
                    ->greeting('Hello!')
                    ->line('One of your invoices has been paid!')
                    ->action('View Invoice', $url)
                    ->line('Thank you for using our application!');
    }
```

5.12.4.2.自定义发送人

默认情况下，电子邮件的发件人 / 发件人地址 在 config/mail.php 配置文件中定义。 但是，您可以使用 from 方法指定特定通知的发件人地址：

```
    /**
     * Get the mail representation of the notification.
     *
     * @param  mixed  $notifiable
     * @return \Illuminate\Notifications\Messages\MailMessage
     */
```

```
public function toMail($notifiable)
{
    return (new MailMessage)
                ->from('test@example.com', 'Example')
                ->line('...');
}
```

5.12.4.3.自定义收件人

通过 mail 通道发送通知时，通知系统会自动在被通知实体上查找 email 属性，你可以通过在该实体上定义一个 routeNotificationForMail 方法来自定义用于发送通知的电子邮件地址：

```
<?php
namespace App;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * 邮件通道通知的路由。
     *
```

```
        * @param  \Illuminate\Notifications\Notification
$notification
        * @return array|string
        */
       public function routeNotificationForMail($notification)
       {
           // 只返回邮件地址...
           return $this->email_address;

           // 返回名字和邮件地址...
           return [$this->email_address => $this->name];
       }
   }
```

5.12.4.4.自定义主题

默认情况下，电子邮件的主题是格式为「标题风格」的通知的类名。因此，如果您的通知类名为 InvoicePaid，则电子邮件的主题将是 Invoice Paid。

5.12.5. Markdown 邮件通知

Markdown 邮件通知允许您利用预先构建的邮件通知模板，同时让您更自由地编写更长的自定义消息。 由于消息是用 Markdown 编写的，Laravel 能够为消息呈现漂亮的响应式 HTML 模板，同时还可以自动生成纯文本副本。

5.12.5.1.生成消息

要使用相应的 Markdown 模板生成通知，您可以使用 make:notification Artisan 命令的 --markdown 选项：

```
    php artisan make:notification InvoicePaid --
markdown=mail.invoice.paid
```

5.12.5.2.编写消息

Markdown 邮件通知使用 Blade 组件和 Markdown 语法的组合，允许您在利用 Laravel 预先制作的通知组件的同时轻松构建通知：

```
    @component('mail::message')
    # Invoice Paid


    Your invoice has been paid!


    @component('mail::button', ['url' => $url])
    View Invoice
    @endcomponent


    Thanks,<br>
    {{ config('app.name') }}
    @endcomponent
```

5.12.5.3.自定义组件

您可以将所有 Markdown 通知组件导出到您自己的应用程序以进行自定义。要导出组件，请使用 vendor:publish Artisan 命令发布 laravel-mail 资产标签：

```
    php artisan vendor:publish --tag=laravel-mail
```

此命令会将 Markdown 邮件组件发布到 resources/views/vendor/mail 目录。

mail 目录将包含一个 html 和一个 text 目录，每个目录都包含它们各自对每个可用组件的表示。 您可以随意自定义这些组件。

5.12.6. 数据库通知

5.12.6.1.先决条件

database 通知通道将通知信息存储在数据库表中。此表将包含通知类型等信息以及描述通知的 JSON 数据结构。

您可以查询该表以在应用程序的用户界面中显示通知。但是，在您这样做之前，您需要创建一个数据库表来保存您的通知。您可以使用 notifications:table 命令生成具有正确表模式的 migration：

```
php artisan notifications:table

php artisan migrate
```

5.12.6.2.格式化数据库通知

如果通知支持存储在数据库表中，则应在通知类上定义 toDatabase 或 toArray 方法。这个方法将接收一个 $notifiable 实体并且应该返回一个普通的 PHP 数组。 返回的数组将被编码为 JSON 并存储在 notifications 表的 data 列中。让我们看一个示例 toArray 方法：

```
/**
 * Get the array representation of the notification.
 *
 * @param  mixed  $notifiable
 * @return array
 */
public function toArray($notifiable)
```

```
    {
        return [
            'invoice_id' => $this->invoice->id,
            'amount' => $this->invoice->amount,
        ];
    }
```

5.12.6.3.访问通知

一旦通知存储在数据库中，您需要一种方便的方式从您的通知实体访问它们。Illuminate\Notifications\Notifiable trait 包含在 Laravel 的默认 App\Models\User 模型中，包括一个 notifications Eloquent 关系，它返回 实体的通知。要获取通知，您可以像访问任何其他 Eloquent 关系一样访问此方法。默认情况下，通知将按 created_at 时间戳排序，最近的通知位于集合的开头：

```
$user = App\User::find(1);
foreach ($user->notifications as $notification) {
    echo $notification->type;
}
```

5.12.6.4.标记通知已读

通常，您希望在用户查看通知时将其标记为“已读”。Illuminate\Notifications\Notifiable trait 提供了 markAsRead 方法，它更新通知数据库记录中的 read_at 列：

```
$user = App\User::find(1);
foreach ($user->unreadNotifications as $notification) {
    $notification->markAsRead();
```

```
}
```

5.12.7. 广播通知

5.12.7.1.先决条件

在广播通知之前，您应该配置并熟悉 Laravel 的 事件广播 服务。事件广播提供了一种从 JavaScript 驱动的前端对服务器端 Laravel 事件做出反应的方法。

5.12.7.2.格式化广播通知

broadcast 频道使用 Laravel 的 事件广播 服务广播通知，允许您的 JavaScript 驱动的前端实时捕获通知。如果通知支持广播，您可以在通知类上定义一个 toBroadcast 方法。此方法将接收一个 $notifiable 实体并应返回一个 BroadcastMessage 实例。如果 toBroadcast 方法不存在，toArray 方法将用于收集应该广播的数据。返回的数据将被编码为 JSON 并广播到您的 JavaScript 驱动的前端。让我们看一个示例 toBroadcast 方法：

```
use Illuminate\Notifications\Messages\BroadcastMessage;


/**
 * Get the broadcastable representation of the notification.
 *
 * @param  mixed  $notifiable
 * @return BroadcastMessage
 */
public function toBroadcast($notifiable)
{
    return new BroadcastMessage([
        'invoice_id' => $this->invoice->id,
```

```
            'amount' => $this->invoice->amount,
        ]);
    }
```

5.12.7.3.监听通知

通知将在使用 {notifiable}.{id} 约定格式化的私人频道上广播。因此，如果您要向 ID 为 1 的 App\Models\User 实例发送通知，则通知将在 App.Models.User.1 私有频道上广播。使用 Laravel Echo 时，您可以使用 notification 方法轻松地监听频道上的通知：

```
Echo.private('App.User.' + userId)
    .notification((notification) => {
        console.log(notification.type);
    });
```

5.12.8. 短信

5.12.8.1.先决条件

在 Laravel 中发送 SMS 通知由 Vonage（以前称为 Nexmo）提供支持。在通过 Vonage 发送通知之前，您需要安装 laravel/vonage-notification-channel 和 guzzlehttp/guzzle 包：

```
composer require laravel/vonage-notification-channel
guzzlehttp/guzzle
```

该软件包包括一个 [配置文件]（https://github.com/laravel/vonage-notification-channel/blob/3.x/config/vonage.php）。 但是，您不需要将此配置文件导出到您自己的应用程序。您可以简单地使用 VONAGE_KEY 和 VONAGE_SECRET 环境变量来定义您的 Vonage 公钥和密钥。

5.12.8.2.格式化短信通知

如果通知支持以短信方式发送，那么你需要在通知类上定义一个 toNexmo 方法。该方法接收一个 $notifiable 实体并返回 Illuminate\Notifications\Messages\NexmoMessage 实例：

```
/**
 * 获取通知的 Vonage / SMS 表示。
 *
 * @param  mixed  $notifiable
 * @return \Illuminate\Notifications\Messages\VonageMessage
 */
public function toVonage($notifiable)
{
    return (new VonageMessage)
                ->content('Your SMS message content');
}
```

5.12.8.3.自定义发件人号码

如果您想从与您的 VONAGE_SMS_FROM 环境变量指定的电话号码不同的电话号码发送一些通知，您可以在 VonageMessage 实例上调用 from 方法：

```
/**
 * 获取通知的 Vonage / SMS 表示。
 *
 * @param  mixed  $notifiable
 * @return \Illuminate\Notifications\Messages\VonageMessage
```

```
     */
    public function toVonage($notifiable)
    {
        return (new VonageMessage)
                    ->content('您的短信内容')
                    ->from('15554443333');
    }
```

5.12.9. Slack 通知

5.12.9.1.先决条件

在您可以通过 Slack 发送通知之前，您必须通过 Composer 安装 Slack 通知通道：

```
composer require laravel/slack-notification-channel
```

您还需要为您的团队创建一个 Slack 应用程序。 创建应用程序后，您应该为工作区配置一个「传入 Webhook」。 然后，Slack 将为您提供一个 webhook URL，您可以在 路由 Slack 通知 时使用该 URL。

5.12.9.2.格式化 Slack 通知

如果通知支持作为 Slack 消息发送，则应在通知类上定义 toSlack 方法。此方法将接收一个 $notifiable 实体并应返回一个 Illuminate\Notifications\Messages\SlackMessage 实例。 Slack 消息可能包含文本内容以及格式化附加文本或字段数组的 “附件”。

5.12.9.3.Slack 附件

您还可以在邮件中添加「附件」。附件提供了比简单文本消息更丰富的格式选项。

5.12.10. 本地化通知

Laravel 允许您在 HTTP 请求的当前语言环境之外的语言环境中发送通知，如果通知在排队，它甚至会记住这个语言环境。

为此，Illuminate\Notifications\Notification 类提供了一个 locale 方法来设置所需的语言。应用程序将在评估通知时更改为此区域设置，然后在评估完成后恢复为先前的区域设置：

```
$user->notify((new InvoicePaid($invoice))->locale('es'));
```

5.12.10.1. 用户首选语言环境

有时，应用程序存储每个用户的首选语言环境。通过在你的通知模型上实现 HasLocalePreference 合约，你可以指示 Laravel 在发送通知时使用这个存储的语言环境：

```
use Illuminate\Contracts\Translation\HasLocalePreference;

class User extends Model implements HasLocalePreference
{
    /**
     * 获取用户的首选语言环境。
     *
     * @return string
     */
    public function preferredLocale()
    {
        return $this->locale;
    }
}
```

5.12.11. 通知事件

通知发送事件

发送通知时，通知系统会调度 Illuminate\Notifications\Events\NotificationSending event。 这包含“可通知”实体和通知实例本身。您可以在应用程序的 EventServiceProvider 中为此事件注册监听器：

```
/**
 * 应用程序的事件监听器映射。
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Notifications\Events\NotificationSending' => [
        'App\Listeners\CheckNotificationStatus',
    ],
];
```

5.12.12. 自定义通道

Laravel 为我们提供了多种通知通道，但你可能想编写自己的驱动程序来通过其他通道传递通知。 Laravel 让它变得简单。首先，定义一个包含 send 方法的类。该方法应该接收两个参数：一个 $notifiable 和一个 $notification。

在 send 方法中，您可以调用通知上的方法来检索您的频道可以理解的消息对象，然后将通知发送到 $notifiable 实例，但是您希望：

```
<?php
```

```
namespace App\Channels;

use Illuminate\Notifications\Notification;


class VoiceChannel
{
    /**
     * 发送指定的通知.
     *
     * @param  mixed  $notifiable
     * @param  \Illuminate\Notifications\Notification
$notification
     * @return void
     */
    public function send($notifiable, Notification $notification)
    {
        $message = $notification->toVoice($notifiable);


        // Send notification to the $notifiable instance...
    }
}
```

5.13. 扩展包开发

5.13.1. 简介

包是向 Laravel 添加功能的主要方式。 包可能是处理日期的好方法，例如 Carbon，也可能是允许您将文件与 Eloquent 模型相关联的包，例如 Spatie 的 Laravel 媒体库。

有不同类型的包。 有些包是独立的，这意味着它们可以与任何 PHP 框架一起使用。 Carbon 和 PHPUnit 是独立包的示例。 任何这些包都可以通过在你的 composer.json 文件中的要求来与 Laravel 一起使用。

另一方面，其他软件包专门用于 Laravel。 这些包可能包含专门用于增强 Laravel 应用程序的路由、控制器、视图和配置。 本指南主要涵盖了那些特定于 Laravel 的包的开发。

5.13.1.1.关于 Facades

在编写 Laravel 应用程序时，通常使用契约（Contracts）还是门面（Facades）并不重要，因为两者都提供了基本相同的可测试性级别。 但是，在编写包时，您的包通常无法访问 Laravel 的所有测试助手。 如果您希望能够像将包安装在典型的 Laravel 应用程序中一样编写包测试，您可以使用 Orchestral Testbench 包。

5.13.2. 包自动发现

在 Laravel 应用的 config/app.php 配置文件中， providers 选项定义了能够被 Laravel 加载的服务提供者列表。 当有人安装您的扩展包时，通常会希望此列表中包含你的服务提供者。你可以在扩展包的 composer.json 文件中的 extra 部分定义服务提供者，而不是让用户手动将你的服务提供者添加到列表中。除了服务提供者，你还可以列出你想要注册的所有 facades ：

```
"extra": {
    "laravel": {
        "providers": [
```

```
            "Barryvdh\\Debugbar\\ServiceProvider"
        ],
        "aliases": {
            "Debugbar": "Barryvdh\\Debugbar\\Facade"
        }
    }
},
```

一旦你的包被配置为发现，Laravel 将在安装时自动注册它的服务提供者和门面，为你的包的用户创造一个方便的安装体验。

5.13.2.1.选择退出包发现

如果您是一个包的消费者并且想要禁用包的包发现，您可以在应用程序的 composer.json 文件的 extra 部分列出包名：

```
"extra": {
    "laravel": {
        "dont-discover": [
            "barryvdh/laravel-debugbar"
        ]
    }
},
```

您可以使用应用程序的 dont-discover 指令中的 * 字符禁用所有包的包发现：

```
"extra": {
    "laravel": {
        "dont-discover": [
```

```
            "*"
        ]
    }
},
```

5.13.3. 服务提供者

服务提供者 是你的包和 Laravel 之间的连接点。 服务提供者负责将事物绑定到 Laravel 的 服务容器 并通知 Laravel 在哪里加载包资源，例如视图、配置和本地化文件。

服务提供者扩展了 Illuminate\Support\ServiceProvider 类并包含两个方法：register 和 boot。 基本的 ServiceProvider 类位于 illuminate/support Composer 包中，您应该将其添加到您自己的包的依赖项中。 要了解有关服务提供者的结构和目的的更多信息，请查看 他们的文档。

5.13.4. 资源

5.13.4.1.配置

通常，您需要将包的配置文件发布到应用程序的 config 目录。 这将允许您的包的用户轻松覆盖您的默认配置选项。 要允许发布配置文件，请从服务提供者的 boot 方法中调用 publishes 方法：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
```

```
    $this->publishes([
        __DIR__.'/path/to/config/courier.php' =>
config_path('courier.php'),
    ]);
}
```

5.13.4.2.路由

如果你的包包含路由，你可以使用 loadRoutesFrom 方法加载它们。 此方法将自动确定应用程序的路由是否被缓存，如果路由已被缓存，则不会加载您的路由文件：

```
/**
 * Bootstrap any package services.
 *
 * @return void
 */
public function boot()
{
    $this->loadRoutesFrom(__DIR__.'/../routes/web.php');
}
```

5.13.4.3.迁移

如果您的扩展包包含数据库迁移，你可以使用 loadMigrationsFrom 方法告诉 Laravel 如何加载它们。 loadMigrationsFrom 方法接受包迁移的路径作为其唯一参数：

```
/**
```

```
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->loadMigrationsFrom(__DIR__.'/path/to/migrations');
}
```

5.13.4.4.语言包

如果您的扩展包包含翻译文件，您可以使用 LoadTranslationsFrom 方法告诉框架如何加载它们。例如，如果你的扩展包名为 courier，你需要将下面的内容加入到服务提供者的 boot 方法中：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->loadTranslationsFrom(__DIR__.'/path/to/translations',
'courier');
}
```

5.13.4.5.视图

想要在 Laravel 中注册你的扩展包的 视图 ， 需要告知 Laravel 视图文件的位置。 你可以使用服务提供者的 loadViewsFrom 方法来实现。loadViewsFrom 方法接收两个参数：视图模板的路径和扩展包名。例如，如果你的扩展包名为 courier ，你需要将下面的内容加入到服务提供者的 boot 方法中：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->loadViewsFrom(__DIR__.'/path/to/views', 'courier');
}
```

5.13.4.6.视图组件

如果您的扩展包中包含视图组件 ， 你需要使用 loadViewComponentsAs 方法告知 Laravel 如何加载它们。 loadViewComponentsAs 方法接受两个参数：视图组件的标签前缀和视图组件类组成的数组。例如，如果包的前缀是 courier ，并且具有 Alert 和 Button 视图组件，则可以将以下内容添加到服务提供者的 boot 方法中：

```
/**
 * 启动应用服务
 *
 * @return void
```

```
 */
public function boot()
{
    $this->loadViewComponentsAs('courier', [
        Alert::class,
        Button::class,
    ]);
}
```

一旦在服务提供者中注册了视图组件，就可以直接在视图中引用它们，如下所示：

```
<x-courier-alert />

<x-courier-button />
```

5.13.5. 命令

要在 Laravel 中注册你的包的 Artisan 命令，你可以使用 commands 方法。此方法需要一个命令类名称数组。 注册命令后，您可以使用 Artisan CLI 执行它们：

```
use Courier\Console\Commands\InstallCommand;
use Courier\Console\Commands\NetworkCommand;

/**
 * Bootstrap any package services.
 *
```

```
 * @return void
 */
public function boot()
{
    if ($this->app->runningInConsole()) {
        $this->commands([
            InstallCommand::class,
            NetworkCommand::class,
        ]);
    }
}
```

5.13.6. 发布前端资源

您的扩展包可能包含 JavaScript 、CSS 和图片之类的资源文件。要将这些资源发布到应用的 public 目录，可以使用服务提供者的 publishes 方法。在下面的例子中，我们也可以添加一个 public 资源组标签，该标签可用于发布相关资源组：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
```

```
    $this->publishes([
        __DIR__.'/path/to/assets' => public_path('vendor/courier'),
    ], 'public');
}
```

5.13.7. 发布文件组

您可能希望单独发布包资产和资源组。例如，您可能希望允许用户发布包的配置文件，而不是被迫发布包的全部资产。您可以在包的服务提供者中调用 publishes 方法时定义「标签」来做到这一点。例如，让我们使用标签在包的服务提供者的 boot 方法中为 courier 包定义两个发布组（courier-config 和 courier-migrations）：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.'/../config/package.php' =>
config_path('package.php')
    ], 'config');

    $this->publishes([
```

```
        __DIR__.'/../database/migrations/' =>
database_path('migrations')
    ], 'migrations');
}
```

5.14. 队列

5.14.1. 简介

在构建 Web 应用程序时，你可能需要执行一些任务（例如解析和存储上传的 CSV 文件），但这些任务在典型的 Web 请求中花费的时间太长。幸运的是，Laravel 允许你轻松地创建可在后台排队处理的任务作业。通过将耗时的任务移到队列中，你的应用程序可以以超快的速度响应 Web 请求，并为客户提供更好的用户体验。

Laravel 队列提供了可以跨各种不同队列后台的统一 API，例如 Amazon SQS, Redis 甚至关系数据库。

队列配置文件存储在 config/queue.php 中。 在这个文件中，你可以找到框架中包含的每个队列驱动程序的连接配置，其中包括数据库，Amazon SQS, Redis, 和 Beanstalkd 驱动，一个同步驱动程序（供本地使用）。还包括一个用于丢弃排队任务的 null 队列驱动。

5.14.1.1.连接 Vs 队列

在开始使用 Laravel 队列之前，理解「连接」和「队列」之间的区别非常重要。在 config/queue.php 配置文件中，有一个 connections 配置选项。此选项定义到后端服务（如 Amazon SQS、Beanstalk 或 Redis）的特定连接。然而，任何给定的队列连接都可能有多个队列，这些队列可能被认为是不同的堆栈或成堆的排队任务。

请注意，queue 配置文件中的每个连接配置示例都包含一个 queue 属性。这

是将任务发送到给定连接时将被分配到的默认队列。换句话说，如果你没有显式地定义任务应该被发送到哪个队列，那么该任务将被放置在连接配置的 queue 属性中定义的队列上：

```
use App\Jobs\ProcessPodcast;

// 这个任务将被推送到默认队列...

ProcessPodcast::dispatch();

// 这个任务将被推送到 "emails" 队列...

ProcessPodcast::dispatch()->onQueue('emails');
```

有些应用程序可能不需要将任务推到多个队列中，而是倾向于使用一个简单的队列。然而，如果希望对任务的处理方式进行优先级排序或分段时，将任务推送到多个队列就显得特别有用，因为 Laravel 队列工作程序允许您指定哪些队列应该按优先级处理。

5.14.1.2.驱动程序说明和先决条件

数据库

要使用 database 队列驱动程序，你需要一个数据库表来保存任务。要生成创建此表的迁移，请运行 queue:table Artisan 命令。一旦迁移已经创建，你可以使用 migrate 命令迁移你的数据库:

```
php artisan queue:table

php artisan migrate
```

Redis

要使用 Redis 队列驱动程序，需要在 config/database.php 配置文件中配置一个 redis 数据库连接。

Redis 集群

如果你的 Redis 队列连接使用一个 Redis 集群，那么你的队列名称就必须包含一个 key hash tag。这是为了确保一个给定列表的所有 Redis 键都被放在同一个哈希插槽：

```
'redis' => [
    'driver' => 'redis',
    'connection' => 'default',
    'queue' => '{default}',
    'retry_after' => 90,
],
```

5.14.2. 创建任务

5.14.2.1.生成任务类

默认情况下，应用程序的所有的可排队任务都被存储在了 app/Jobs 目录中。如果 app/Jobs 目录不存在，当您运行 make:job Artisan 命令时，将会自动创建该目录：

```
php artisan make:job ProcessPodcast
```

生成的类将会实现 Illuminate\Contracts\Queue\ShouldQueue 接口，告诉 Laravel，该任务应该推入队列以异步的方式运行。

5.14.2.2.任务类结构

任务类非常简单，通常只包含一个 handle 方法，在队列处理任务时将会调用它。让我们看一个任务类的示例。在这个例子中，我们假设我们管理一个 podcast 服务，并且需要在上传的 podcast 文件发布之前对其进行处理：

```
<?php
namespace App\Jobs;
use App\Models\Podcast;
use App\Services\AudioProcessor;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class ProcessPodcast implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable,
SerializesModels;

    protected $podcast;

    /**
     * 创建一个新的任务实例。
     *
     * @param  Podcast  $podcast
     * @return void
     */
    public function __construct(Podcast $podcast)
    {
        $this->podcast = $podcast;
    }

    /**
     * 运行任务
```

```
     *
     * @param  AudioProcessor  $processor
     * @return void
     */
    public function handle(AudioProcessor $processor)
    {
        // Process uploaded podcast...
    }
}
```

在本示例中，请注意，我们能够将一个 Eloquent model 直接传递到已排队任务的构造函数中。由于任务所使用的 SerializesModels，在任务处理时，Eloquent 模型及其加载的关系将被优雅地序列化和反序列化。

如果你的队列任务在其构造函数中接受一个 Eloquent 模型，那么只有模型的标识符才会被序列化到队列中。当实际处理任务时，队列系统将自动重新从数据库中获取完整的模型实例及其加载的关系。这种用于模型序列化的方式允许将更小的作业有效负载发送给你的队列驱动程序。

5.14.2.3.Unique Jobs

有时，您可能希望确保在任何时间点队列中只有一个特定任务的实例。你可以通过在你的工作类上实现 ShouldBeUnique 接口来做到这一点。这个接口不需要你在你的类上定义任何额外的方法：

```
<?php

use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Contracts\Queue\ShouldBeUnique;


class UpdateSearchIndex implements ShouldQueue, ShouldBeUnique
```

```
{
    ...
}
```

在上面的示例中，UpdateSearchIndex 任务是唯一的。因此，如果任务的另一个实例已经在队列中并且尚未完成处理，则不会分派该任务。

5.14.3. 任务中间件

任务中间件允许你围绕排队任务的执行封装自定义逻辑，从而减少了任务本身的样板代码。

5.14.3.1.访问限制

尽管我们刚刚演示了如何编写自己的访问限制的任务中间件，但 Laravel 实际上内置了一个访问限制中间件，你可以利用它来限制任务。与 路由限流器一样，任务访问限制器是使用 RateLimiter facade 的 for 方法定义的。

例如，你可能希望允许用户每小时备份一次数据，但不对高级客户施加此类限制。为此，可以在 AppServiceProvider 的 boot 方法中定义 RateLimiter ：

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

/**
 * 注册应用程序服务。
 *
 * @return void
 */
public function boot()
{
    RateLimiter::for('backups', function ($job) {
```

```
        return $job->user->vipCustomer()
                    ? Limit::none()
                    : Limit::perHour(1)->by($job->user->id);
    });
}
```

在上面的例子中，我们定义了一个小时访问限制；但是，你可以使用 perMinute 方法轻松定义基于分钟的访问限制。此外，您可以将任何值传递给访问限制的 by 方法，但是，这个值通常用于按客户来区分不同的访问限制：

```
return Limit::perMinute(50)->by($job->user->id);
```

5.14.4. 调度任务

一旦你写好了你的任务类，你可以使用任务本身的 dispatch 方法来调度它。传递给 dispatch 方法的参数将被提供给任务的构造函数：

```
<?php

namespace App\Http\Controllers;


use App\Http\Controllers\Controller;

use App\Jobs\ProcessPodcast;
use App\Models\Podcast;

use Illuminate\Http\Request;


class PodcastController extends Controller
{
    /**
```

```
     * 存储一个新的广播
     *
     * @param  Request  $request
     * @return Response
     */
    public function store(Request $request)
    {
        $podcast = Podcast::create(...);

        ProcessPodcast::dispatch($podcast);
    }
}
```

如果您想有条件地分派任务，您可以使用 dispatchIf 和 dispatchUnless 方法：

```
ProcessPodcast::dispatchIf($accountActive === true, $podcast);

ProcessPodcast::dispatchUnless($accountSuspended === false,
$podcast);
```

5.14.4.1.延迟调度

如果您想指定任务不应立即可供队列工作人员处理，您可以在调度任务时使用 “延迟” 方法。例如，让我们指定一个任务在分派后 10 分钟内不能用于处理：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
```

```
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;


class PodcastController extends Controller
{
    /**
     * 存储一个新的 podcast
     *
     * @param  Request  $request
     * @return Response
     */
    public function store(Request $request)
    {
        // 创建 podcast...

        ProcessPodcast::dispatch($podcast)
                ->delay(now()->addMinutes(10));
    }
}
```

5.14.4.2.同步调度

如果您想要立即 (同步地) 调度任务，您可以使用 dispatchSync 方法。当使用此方法时，任务将不会排队，并将立即运行在当前进程：

```
<?php
```

```
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use App\Jobs\ProcessPodcast;

use Illuminate\Http\Request;


class PodcastController extends Controller

{

    /**

     * 存储一个新的 podcast

     *

     * @param  Request  $request

     * @return Response

     */

    public function store(Request $request)

    {

        // 创建 podcast...


        ProcessPodcast::dispatchSync($podcast);

    }

}
```

5.14.4.3.任务链

任务链允许您指定一组应在主任务成功执行后按顺序运行的排队任务。如果序列中的一个任务失败，其余的任务将不会运行。要执行一个排队的任务链，你

可以使用 Bus facade 提供的 chain 方法：

```
use App\Jobs\OptimizePodcast;
use App\Jobs\ProcessPodcast;
use App\Jobs\ReleasePodcast;
use Illuminate\Support\Facades\Bus;

Bus::chain([
    new ProcessPodcast,
    new OptimizePodcast,
    new ReleasePodcast,
])->dispatch();
```

除了链接作业任务实例，您还可以链接闭包：

```
Bus::chain([
    new ProcessPodcast,
    new OptimizePodcast,
    function () {
        Podcast::update(...);
    },
])->dispatch();
```

5.14.4.4.自定义队列和连接

分派到特定队列

通过将任务推送到不同的队列，您可以对排队的任务进行「分类」，甚至可以优先考虑分配给各个队列的工作人员数量。请记住，这不会将任务推送到队列

配置文件定义的不同队列「连接」，而只会推送到单个连接中的特定队列。

5.14.4.5.指定任务最大尝试次数/超时值

最大尝试次数

如果你的一个队列任务遇到了错误，你可能不希望无限制的重试。因此 Laravel 提供了各种方法来指定一个任务可以尝试多少次或多长时间。

指定任务可尝试的最大次数的其中一个方法是，通过 Artisan 命令行上的 --tries 开关。这将适用于调度作业的所有任务，除非正在处理的任务指定了最大尝试次数。

```
php artisan queue:work --tries=3
```

你可以采取更细化的方法来定义任务类本身的最大尝试次数。如果在任务上指定了最大尝试次数，它将优先于命令行上提供的 --tries 开关设定的值：

```
<?php

namespace App\Jobs;


class ProcessPodcast implements ShouldQueue
{
    /**
     * 任务尝试次数
     *
     * @var int
     */
    public $tries = 5;
}
```

基于时间的尝试

除了定义任务失败前尝试的次数之外，还可以定义任务应该超时的时间。这允许在给定的时间范围内尝试任意次数的任务。要定义任务超时的时间，请在任务类中添加 retryUntil 方法：

```
/**
 * 确定任务应该超时的时间
 *
 * @return \DateTime
 */
public function retryUntil()
{
    return now()->addMinutes(10);
}
```

5.14.4.6.错误处理

如果在处理任务时抛出异常，任务将自动释放回队列，以便再次尝试。 任务将继续发布，直到尝试达到你的应用程序允许的最大次数为止。最大尝试次数由 queue:work Artisan 命令中使用的 --tries 开关定义。或者，可以在任务类本身上定义最大尝试次数。有关运行队列处理器的更多信息 可以在下面找到。

5.14.5. 任务批处理

Laravel 的任务批处理功能允许你轻松地执行一批任务，然后在这批任务执行完毕后执行一些操作。 在开始之前，你应该创建一个数据库迁移来构建一个表来包含有关你的任务批次的元信息，例如它们的完成百分比。这个迁移可以使用 queue:batches-table Artisan 命令生成：

```
php artisan queue:batches-table
```

```
php artisan migrate
```

5.14.5.1.定义可批处理任务

要定义可批处理任务，您应该像往常一样创建可排队任务；但是，您应该将 Illuminate\Bus\Batchable 特征添加到任务类中。

5.14.5.2.调度批次

要调度一批任务，您应该使用 Bus 门面的 batch 方法。 当然，与完成回调结合使用时，批处理主要是有用的。因此，您可以使用 then、catch 和 finally 方法来定义批处理的完成回调。

5.14.6. 队列闭包

除了将任务类分派到队列之外，您还可以分派一个闭包。这对于需要在当前请求周期之外执行的快速、简单的任务非常有用。当向队列分派闭包时，闭包的代码内容是加密签名的，因此它不能在传输过程中被修改：

```
$podcast = App\Podcast::find(1);


dispatch(function () use ($podcast) {
    $podcast->publish();
});
```

使用 catch 方法，您可以提供一个闭包，如果队列中的闭包在耗尽所有队列的配置重试尝试后未能成功完成，则应执行该闭包：

```
use Throwable;


dispatch(function () use ($podcast) {
    $podcast->publish();
```

```
})->catch(function (Throwable $e) {
    // 任务失败了
});
```

5.14.7. 运行队列工作者

5.14.7.1.queue:work 命令

Laravel 包含一个 Artisan 命令，该命令将启动队列进程并在新任务被推送到队列时处理它们。 您可以使用 queue:work Artisan 命令运行任务进程。请注意，一旦 queue:work 命令启动，它将继续运行，直到手动停止或关闭终端：

```
php artisan queue:work
```

请记住，队列任务者是长期存在的进程，并将启动的应用程序状态存储在内存中。 因此，它们在启动后不会注意到您的代码库中的更改。 因此，在您的部署过程中，请务必重新启动您的队列进程。 此外，请记住，您的应用进程创建或修改的任何静态状态都不会在任务之间自动重置。

或者，您可以运行 queue:listen 命令。 使用 queue:listen 命令时，当您想要重新加载更新的代码或重置应用进程状态时，您不必手动重启 worker； 但是，此命令的效率明显低于 queue:work 命令：

```
php artisan queue:listen
```

5.14.7.2.队列优先级

有时您可能希望优先处理队列的处理方式。例如，在 config/queue.php 配置文件中，您可以将 redis 连接的默认 queue 设置为 low。 但是，有时您可能希望将作业推送到 high 优先级队列，如下所示：

```
dispatch((new Job)->onQueue('high'));
```

要启动一个进程，在继续处理 low 队列上的任何任务之前验证所有 high 队列任务是否已处理，请将队列名称的逗号分隔列表传递给 work 命令：

```
php artisan queue:work --queue=high,low
```

5.14.8. Supervisor 配置

在生产中，您需要一种方法来保持 queue:work 进程运行。 queue:work 进程可能会因多种原因停止运行，例如超过 worker 超时或执行 queue:restart 命令。

出于这个原因，您需要配置一个进程监视器，它可以检测您的 queue:work 进程何时退出并自动重新启动它们。此外，进程监视器可以让您指定要同时运行多少个 queue:work 进程。Supervisor 是 Linux 环境中常用的进程监视器，我们将在下面的文档中讨论如何配置它。

安装 Supervisor

Supervisor 是 Linux 操作系统的进程监视器，如果它们失败，它将自动重新启动您的 queue:work 进程。要在 Ubuntu 上安装 Supervisor，您可以使用以下命令：

```
sudo apt-get install supervisor
```

Supervisor 配置

Supervisor 配置文件通常存储在 /etc/supervisor/conf.d 目录。在此目录中，你可以创建任意数量的配置文件，这些配置文件将指示 supervisor 如何监事你的进程。

5.14.9. 处理失败任务

有时，您队列任务会失败。别担心，事情并不总是按计划进行！ Laravel 提供了一种方便的方法来 指定应尝试任务的最大次数。在异步任务超过此尝试次数后，它将被插入到 failed_jobs 数据库表中。 同步调度的任务 不存储在此表中，其异常由应用程序立即处理。

创建 failed_jobs 表的迁移通常已经存在于新的 Laravel 应用程序中。但是，如果您的应用程序不包含此表的迁移，您可以使用 queue:failed-table 命令来创

建迁移：

```
php artisan queue:failed-table

php artisan migrate
```

5.14.9.1.任务失败后清理

当特定任务失败时，您可能希望向用户发送警报或恢复该任务部分完成的任何操作。为此，您可以在任务类上定义一个 failed 方法。

5.15. 限流

5.15.1. 简介

Laravel 包含了一个开箱即用的，基于缓存实现的限流器，提供了一个简单的方法来限制指定时间内的任何操作。

5.15.1.1.缓存配置

通常情况下，限流器使用你默认的缓存驱动，由 cache 配置文件中的 default 键定义。你也可以通过在你的应用程序的 cache 配置文件中定义一个 limiter 来指定限流器应该使用哪一个缓存来驱动：

```
'default' => 'memcached',

'limiter' => 'redis',
```

5.15.2. 基础用法

可以通过 Illuminate\Support\Facades\RateLimiter 来操作限流器。限流器提供的最简单的方法是 attempt 方法，它将一个给定的回调函数执行次数限制在一个给定的秒数内。

5.15.2.1.手动配置尝试次数

如果您想手动与限流器交互，可以使用多种方法。例如，您可以调用 tooManyAttempts 方法来确定给定的限流器是否超过了每分钟允许的最大尝试次数。

```
use Illuminate\Support\Facades\RateLimiter;


if (RateLimiter::tooManyAttempts('send-message:'.$user->id,
$perMinute = 5)) {
    return 'Too many attempts!';
}
```

5.15.2.2.清除尝试次数

您可以使用 clear 方法重置给定速率限制键的尝试次数。例如，当接收者读取给定消息时，您可以重置尝试次数:

```
use App\Models\Message;
use Illuminate\Support\Facades\RateLimiter;


/**
 * 将邮件标记为已读。
 *
 * @param  \App\Models\Message  $message
 * @return \App\Models\Message
 */
public function read(Message $message)
{
```

```
    $message->markAsRead();

    RateLimiter::clear('send-message:'.$message->user_id);

    return $message;
}
```

5.16. 任务调度

5.16.1. 简介

过去，你可能需要在服务器上为每一个调度任务去创建 Cron 条目。因为这些任务的调度不是通过代码控制的，你要查看或新增任务调度都需要通过 SSH 远程登录到服务器上去操作，所以这种方式很快会让人变得痛苦不堪。

Laravel 的命令行调度器允许你在 Laravel 中清晰明了地定义命令调度。在使用这个任务调度器时，你只需要在你的服务器上创建单个 Cron 入口。你的任务调度在 app/Console/Kernel.php 的 schedule 方法中进行定义。为了帮助你更好的入门，这个方法中有个简单的例子。

5.16.2. 定义调度

你可以在 App\Console\Kernel 类的 schedule 方法中定义所有的调度任务。在开始之前，我们来看一个例子：我们计划每天午夜执行一个 闭包，这个 闭包会执行一次数据库语句去清空一张表：

```
<?php

namespace App\Console;

use Illuminate\Console\Scheduling\Schedule;

use Illuminate\Foundation\Console\Kernel as ConsoleKernel;
```

```
use Illuminate\Support\Facades\DB;


class Kernel extends ConsoleKernel
{


    /**
     * 定义应用中的命令调度
     *
     * @param  \Illuminate\Console\Scheduling\Schedule  $schedule
     * @return void
     */
    protected function schedule(Schedule $schedule)
    {
        $schedule->call(function () {
            DB::table('recent_users')->delete();
        })->daily();
    }
}
```

除了调用闭包这种方式来调度外，你还可以调用 可调用对象. 可调用对象是简单的 PHP 类，包含一个 __invoke 方法：

```
$schedule->call(new DeleteRecentUsers)->daily();
```

5.16.2.1.调度 Artisan 命令

调度方式不仅有调用闭包，还有调用 Artisan commands 和操作系统命令。

例如，你可以给 command 方法传递命令名称或类来调度一个 Artisan 命令：

当使用命令类名调度 Artisan 命令时，你可以通过一个数组传递附加的命令行参数，且这些参数需要在命令触发时提供：

```
use App\Console\Commands\SendEmailsCommand;

$schedule->command('emails:send Taylor --force')->daily();

$schedule->command(SendEmailsCommand::class, ['Taylor', '--force'])->daily();
```

5.16.2.2.调度队列任务

Job 方法可用于调度一个队列任务。此方法提供了一种快捷方式来调度任务，而无需使用 call 方法创建闭包来调度任务：

```
use App\Jobs\Heartbeat;

$schedule->job(new Heartbeat)->everyFiveMinutes();
```

5.16.2.3.调度 Shell 命令

exec 方法可发送命令到操作系统：

```
$schedule->exec('node /home/forge/script.js')->daily();
```

5.16.2.4.调度频率选项

我们已经看到了几个如何设置任务在指定时间间隔运行的例子。不仅如此，你还有更多的任务调度频率可选：

方法	描述
->cron('* * * * *');	自定义 Cron 计划执行任务

->everyMinute();	每分钟执行一次任务
->everyTwoMinute();	每两分钟执行一次任务
->hourly();	每小时执行一次任务
->everyTwoHours();	每两小时执行一次任务
->daily();	每天 0 点执行一次任务
->weeky();	每周日 00：00 点执行一次任务
->monthly();	每月第一天 00：00 点执行一次任务
->yearly();	每年第一天 00：00 点执行一次任务

5.16.2.5.时区

timezone 方法可指定在某一时区的时间执行计划任务：

```
$schedule->command('report:generate')
         ->timezone('America/New_York')
         ->at('02:00')
```

5.16.2.6.避免任务重复

默认情况下，即使之前的任务实例还在执行，调度内的任务也会执行。为避免这种情况的发生，你可以使用 withoutOverlapping 方法：

```
$schedule->command('emails:send')->withoutOverlapping();
```

在此例中，若 emails:send Artisan 命令还未执行，那它将会每分钟执行一次。当你的任务执行时间非常不确定，导致你无法准确预测任务的执行时间，那 withoutOverlapping 方法会特别有用。

5.16.2.7.任务只运行在单台服务器上

如果你的应用运行在多台服务器上，可能需要限制调度任务只在某台服务器上运行。 例如，假设你有一个每个星期五晚上生成新报告的调度任务，如果任务调度器运行在三台服务器上，调度任务会在三台服务器上运行并且生成三次报告，不够优雅！

要指示任务应仅在一台服务器上运行，请在定义计划任务时使用 onOneServer 方法。第一台获取到该任务的服务器会给任务上一把原子锁以阻止其他服务器同时运行该任务:

```
$schedule->command('report:generate')
                ->fridays()
                ->at('17:00')
                ->onOneServer();
```

5.16.2.8.后台任务

默认情况下，同时运行多个任务将根据它们在 schedule 方法中定义的顺序执行。如果你有一些长时间运行的任务，将会导致后续任务比预期时间更晚启动。如果你想在后台运行任务，以便它们可以同时运行，则可以使用 runInBackground 方法:

```
$schedule->command('analytics:report')
         ->daily()
         ->runInBackground();
```

5.16.2.9.维护模式

当应用处于 维护模式 时，Laravel 的队列任务将不会运行。因为我们不想调度任务干扰到服务器上可能还未完成的维护项目。不过，如果你想强制任务在维护模式下运行，你可以使用 evenInMaintenanceMode 方法：

```
$schedule->command('emails:send')->evenInMaintenanceMode();
```

5.16.3. 运行调度程序

现在，我们已经学会了如何定义计划任务，接下来让我们讨论如何真正在服务器上运行它们。schedule:run Artisan 命令将评估你的所有计划任务，并根据服务器的当前时间决定它们是否运行。

因此，当使用 Laravel 的调度程序时，我们只需要向服务器添加一个 cron 配置项，该项每分钟运行一次 schedule:run 命令。如果你不知道如何向服务器添加 cron 配置项，请考虑使用 Laravel Forge 之类的服务来为你管理 cron 配置项：

```
* * * * cd /你的项目路径 && php artisan schedule:run >> /dev/null 2>&1
```

5.16.4. 本地运行调度程序

通常，你不会直接将 cron 配置项添加到本地开发计算机。你反而可以使用 schedule:work Artisan 命令。该命令将在前台运行，并每分钟调用一次调度程序，直到你终止该命令为止：

```
php artisan schedule:work
```

5.16.5. 任务输出

Laravel 调度器提供了一些简便方法来处理调度任务生成的输出。首先，你可以使用 sendOutputTo 方法将输出发送到文件中以便后续检查：

```
$schedule->command('emails:send')

    ->daily()

    ->sendOutputTo($filePath);
```

如果希望将输出追加到给定文件，可以使用 appendOutputTo 方法：

```
$schedule->command('emails:send')
        ->daily()
        ->appendOutputTo($filePath);
```

使用 emailOutputTo 方法，你可以将输出发送到指定邮箱。在使用邮件发送之前，你需要配置 Laravel 的邮件服务：

```
$schedule->command('report:generate')
        ->daily()
        ->sendOutputTo($filePath)
        ->emailOutputTo('taylor@example.com');
```

5.16.6. 任务钩子

使用 before 和 after 方法，你可以决定在调度任务执行前或者执行后来运行代码：

```
$schedule->command('emails:send')
        ->daily()
        ->before(function () {
            // 任务即将开始...
        })
        ->after(function () {
            // 任务完成...
        });
```

使用 onSuccess 和 onFailure 方法，你可以决定在调度任务成功或者失败运行代码。失败表示 Artisan 或系统命令以非零退出码终止：

```
$schedule->command('emails:send')
```

```
        ->daily()
        ->onSuccess(function () {
            // 任务执行成功...
        })
        ->onFailure(function () {
            // 任务执行失败...
        });
```

5.16.7. 事件

如果需要，您可以监听调度程序调度的 事件。

第六章 安全相关

6.1. 用户认证

6.1.1. 简介

许多 web 应用程序为其用户提供了一种通过应用程序进行身份验证和「登录」的方法。在 web 应用程序中实现此功能可能是一项复杂且具有潜在风险的工作。因此，Laravel 努力为你提供了一套快速、安全、轻松地实现身份验证的工具。

在其核心，Laravel 的用户认证是由「看守器」和「提供器」。看守器定义如何对每个请求的用户进行身份验证。例如，Laravel 附带了一个 session 守护程序，它使用 session 存储和 cookie 来维护状态。

提供器定义如何从持久存储中检索用户。Laravel 支持使用 Eloquent 和数据库查询生成器检索用户。不仅如此，你甚至可以根据应用程序的需要自由定制其

他提供程序。

应用程序的身份验证配置文件位于 config/auth.php。这个文件包含几个记载了的选项，用于调整 Laravel 的身份验证服务的行为。

6.1.1.1. 入门套件

想快点开始吗？在新的 Laravel 应用程序中安装 Laravel 入门套件 。执行迁移数据库后，将浏览器导航到 /register 或分配给应用程序的任何其他 URL。初学者工具包将守护好你的整个认证系统！

6.1.1.2. 数据库注意事项

默认情况下，Laravel 在 App/Models 目录中包含一个 App\Models\User Eloquent model。此模型可与默认的 Eloquent 身份验证驱动程序一起使用。如果你的应用程序未使用 Eloquent，则可以使用 Laravel 查询生成器的 database 身份验证提供程序。

为 App\Models\User 模型构建数据库架构时，请确保密码列的长度至少为 60 个字符。当然，新的 Laravel 应用程序中包含的 users 表迁移文件已经创建了一个超过此长度的列。

此外，还应验证 users (或等效表) 是否包含一个可为空的字符串 remember_token ，该列包含 100 个字符。此列将用于为在登录到应用程序时选择「记住我」选项的用户存储令牌。同样，新的 Laravel 应用程序中包含的默认 users 表迁移文件已经包含此列。

6.1.2. 快速入门

6.1.2.1. 安装入门套件

首先，你应该 安装一个 Laravel 应用程序启动程序工具包。我们目前的入门套件，Laravel Breeze 和 Laravel Jetstream，提供了设计精美的起点，可将身份验证纳入你的全新 Laravel 应用程序。

Laravel Breeze 是 Laravel 所有身份验证功能的最简单的实现，包括登录、

注册、密码重置、电子邮件验证和密码确认。 Laravel 的视图层由简单的 Blade 模板 和 Tailwind CSS 组成。Breeze 还使用 Vue 或 React 提供了基于 Inertia 的脚手架选项。

Laravel Jetstream 是一个更强大的应用程序启动工具包，包括对使用 Livewire 或 Inertia.js and Vue 构建应用程序的支持。 此外，Jetstream 还提供可选的双因素身份验证支持、团队、配置文件管理、浏览器会话管理、通过 Laravel Sanctum 的 API 支持、帐户删除等。

6.1.2.2. 获取已验证用户

在安装身份验证初学者工具包并允许用户注册应用程序并对其进行身份验证之后，你通常需要与当前通过身份验证的用户进行交互。在处理传入请求时，您可以通过 Auth facade 的 user 方法访问通过身份验证的用户：

```
use Illuminate\Support\Facades\Auth;


// 获取当前通过认证的用户...
$user = Auth::user();


//  获取当前通过认证的用户 ID...
$id = Auth::id();
```

6.1.2.3. 路由保护

路由中间件 只能用于允许经过身份验证的用户访问给定的路由。Laravel 附带了一个 auth 中间件，它引用了 Illuminate\Auth\Middleware\Authenticate 类。由于此中间件已在应用程序的 HTTP 内核中注册，因此你只需将中间件连接到路由定义即可：

```
Route::get('/flights', function () {
    // 只有经过身份验证的用户才能访问此路由 ...
```

```
})->middleware('auth');
```

6.1.2.4. 登录限流

如果您使用的是 Laravel Breeze 或 Laravel Jetstream 入门套件 ，那么在尝试登录的时候将自动应用速率限制。默认情况下，如果用户在多次尝试后未能提供正确的凭据，一分钟内将无法登录。该限制对与用户的用户名 /email 及其 IP 地址是唯一的。

6.1.3. 手动验证用户

你并非一定要使用 Laravel 应用入门套件 附带的身份验证框架。如果选择不使用此支架，则需要直接使用 Laravel 身份验证类来管理用户身份验证。别担心，这也很容易！

可以借助 Auth facade 访问 Laravel 服务，因此需要在类的开头导入 Auth 。

6.1.3.1. 记住用户

许多 web 应用程序在其登录窗体上提供「记住我」复选框。如果希望在应用程序中提供「记住我」功能，可以将布尔值作为第二个参数传递给 attempt 方法。

当此值为 true 时，Laravel 将无限期地对用户进行身份验证，或者直到用户手动注销。users 表必须包含字符串 remember_token 列，该列将用于存储「记住我」标记。

6.1.3.2. 其他认证方法

如果需要将现有用户实例设置为当前经过身份验证的用户，可以将该用户实例传递给 Auth facade 的 login 方法。 给定的用户实例必须是 Illuminate\Contracts\Auth\Authenticatable contract 的实现。

6.1.4. HTTP 基本认证

HTTP Basic 用户认证 提供了一种快速验证应用程序用户身份的方法，无需设置专用的「登录」页面。要开始，请附加 auth.basic 中间件 到路由。

6.1.4.1. 无状态 HTTP 基本认证

你也可以使用 HTTP Basic 身份验证，而无需在 session 中设置用户标识符 cookie。如果你选择使用 HTTP 身份验证对应用程序 API 的请求进行身份验证，这将非常有用。为此， 定义一个中间件 调用 onceBasic 方法。

6.1.5. 退出登录

要在应用程序中手动注销用户，可以使用 Auth facade 提供的 logout 方法。这将从用户的 session 中删除身份验证信息，以便后续请求不会得到身份验证。

6.1.5.1. 其他设备的会话无效

Laravel 还提供了这样一种机制，可以使用户在其他设备上处于活动状态置为失效并「注销」，而不会使其当前设备上的会话失效。当用户正在更改或更新其密码，并且你希望在保持当前设备身份验证的同时使其他设备上的会话无效时，通常会使用此功能。

6.1.6. 添加自定义保护

你可以使用 Auth facade 的 extend 方法来定义自己的身份验证看守器。你应该在 服务提供器 中调用 extend 方法。

6.1.6.1. 关闭请求保护

实现基于 HTTP 请求的自定义身份验证系统的最简单方法，是使用 Auth::viaRequest 方法。此方法允许您使用单个闭包来快速定义身份验证过程。

首先，在 AuthServiceProvider 的 boot 方法中调用 Auth::viaRequest 方法。viaRequest 方法接受一个看守器名称作为其第一个参数。此名称可以是描述你自定义看守器的任何字符串。传递给该方法的第二个参数应该是一个闭包函数，它接收传入的 HTTP 请求并返回一个用户实例，或者，如果验证失败，则为 null。

6.1.7. 添加自定义用户提供者

如果不使用传统的关系数据库来存储用户，则需要使用自己的身份验证用户提供程序来扩展 Laravel。

6.1.7.1. 用户提供程序契约

Illuminate\Contracts\Auth\UserProvider 实现负责从持久性存储系统（如 MySQL、MongoDB 等）中提取 Illuminate\Contracts\Auth\Authenticatable 实例。无论用户数据是如何存储的，这两个接口可以保障 Laravel 身份验证机制持续工作，或者可以使用任意类型的类来表示经过身份验证的用户

6.1.7.2. 可认证的契约

我们已经研究了 UserProvider 上的每个方法，现在让我们看看 Authenticatable 契约。

6.1.8. 事件

在身份验证过程中，Laravel 调度各种 事件。您可以在自己的 EventServiceProvider 中监听这些事件。

6.2. 用户授权

6.2.1. 简介

除了提供内置的 authentication （身份验证） 服务外，Laravel 还提供了一种可以很简单就进行使用的方法，来对用户与资源的授权关系进行管理。 它很安全，即使用户已经通过了「身份验证（authentication)」，用户也可能无权对应用程序中重要的模型或数据库记录进行删除或更改。简单、条理化的系统性，是 Laravel 对授权管理的特性。

Laravel 主要提供了两种授权操作的方法: 拦截器 和 策略。可以把拦截器（gates）和策略（policies）想象成路由和控制器。拦截器（Gates）提供了一种轻便的基于闭包函数的授权方法，像是路由。而策略（policies)，就像是一个控制器，对特定模型或资源，进行分组管理的逻辑规则。 在本文档中，我们将首先

探讨拦截器（gates），然后研究策略（policies)。

您在构建应用程序时，不用为是仅仅使用拦截器（gates）或是仅仅使用策略（policies）而担心，并不需要在两者中进行唯一选择。大多数的应用程序都同时包含两个方法，并且同时使用两者，能够更好的进行工作。拦截器（gates），更适用于没有与任何模型或资源有关的授权操作，例如查看管理员仪表盘。与之相反，当您希望为特定的模型或资源进行授权管理时，应该使用策略（policies) 方法。

6.2.2. Gates

6.2.2.1. 编写 Gates

Gates 是用来决定用户是否授权执行给予动作的一个闭包函数，并且典型的做法就是在 App\Providers\AuthServiceProvider 中使用 Gate 来定义. Gates 总是接收一个用户实例作为第一个参数，并且可以接收可选参数，比如相关的 Eloquent 模型：

```
/**
 * 注册任意用户认证、用户授权服务。
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Gate::define('edit-settings', function ($user) {
        return $user->isAdmin;
    });
```

```
    Gate::define('update-post', function ($user, $post) {
        return $user->id === $post->user_id;
    });
}
```

6.2.2.2. 授权动作

使用 Gate 来授权动作的时候， 你应该使用 allows 或者 denies 方法。 注意，不需要将当前已认证用户传递给这些方法。 Laravel 会自动处理好已经认证通过的用户，然后传递给 Gate 闭包函数：

```
if (Gate::allows('edit-settings')) {
    // 指定当前用户可以编辑设置
}

if (Gate::allows('update-post', $post)) {
    // 指定当前用户可以进行更新...
}

if (Gate::denies('update-post', $post)) {
    // 指定当前用户不能更新...
}
```

6.2.2.3. Gate 响应

到目前为止，我们只检查了返回简单布尔值的 Gate 。但是，有时你可能希

望返回更详细的响应，包括错误消息。为此，你可以从你的 Gate 返回 illuminate\auth\access\response。

```
use Illuminate\Auth\Access\Response;
use Illuminate\Support\Facades\Gate;

Gate::define('edit-settings', function ($user) {
    return $user->isAdmin
                ? Response::allow()
                : Response::deny('You must be a super
administrator.');
});
```

6.2.2.4. Gate 拦截检查

有时，你可能希望将所有能力授予特定用户。所以你可以在所有其他授权检查之前使用 before 方法来定义运行的回调：

```
Gate::before(function ($user, $ability) {
    if ($user->isSuperAdmin()) {
        return true;
    }
});
```

6.2.3. 创建策略

6.2.3.1. 生成策略

策略是在特定模型或者资源中组织授权逻辑的类。

6.2.3.2. 注册策略

一旦策略存在，就需要注册它。

6.2.4. 编写策略

6.2.4.1. 策略方法

一旦注册了策略，你可以为它授权的每个操作添加方法。

6.2.4.2. 策略响应

到目前为止，我们只研究了返回简单布尔值的策略方法。然而，有时您可能希望返回更详细的响应，包括错误消息。为此，您可以从您的策略方法返回一个 \Auth\Access\Response。

6.2.4.3. 不包含模型方法

一些策略方法只接收当前经过身份验证的用户，而不接收它们授权的模型的实例。这种情况在授权 create 操作时最为常见。

6.2.4.4. Guest 用户

默认情况下，如果传入的 HTTP 请求不是由经过身份验证的用户发起的，所有的门和策略都会自动返回 false。但是，您可以通过声明一个「可选」类型提示或为用户参数定义提供一个 null 默认值，从而允许这些授权检查通过您的门和策略。

6.2.4.5. 策略过滤器

对于某些用户，您可能希望授权给定策略中的所有操作。为此，在策略上定义一个 before 方法。before 方法将在策略上的任何其他方法之前执行，从而使您有机会在实际调用预期的策略方法之前授权操作。

6.2.5. 使用策略授权动作

6.2.5.1. 通过用户模型

Laravel 应用程序中包含的 User 模型包括两个用于授权操作的有用方法：can 和 cant。can 方法接收您希望授权的操作和相关模型。

6.3. 邮箱验证

6.3.1. 简介

很多 Web 应用会要求用户在使用之前进行 Email 地址验证。Laravel 不会强迫你在每个应用中重复实现它，而是提供了便捷的方法来发送和校验电子邮件的验证请求。

6.3.2. 模型准备

一旦这一接口被添加到模型中，新注册的用户将自动收到一封包含电子邮件验证链接的电子邮件。检查你的 App\Providers\EventServiceProvider 可以看到，Laravel 已经为 Illuminate\Auth\Events\Registered 事件附加了一个 SendEmailVerificationNotification 监听器 。

6.3.3. 数据库准备

接下来，你的 user 表必须包含一个 email_verified_at 字段用来存储 Email 地址通过验证的时间。默认情况下，Laravel 框架中 users 表的数据迁移已经包含了这个字段。所以，您需要做的就只是执行数据库迁移：

```
php artisan migrate
```

6.3.4. 路由

为了实现完整的电子邮件验证流程，你将需要定义三个路由。首先，需要定义一个路由向用户显示通知，告诉他们应该点击 Laravel 注册后向他们发送邮件中的验证链接。

其次，需要一个路由来处理用户点击邮件中验证链接时发来的请求。

第三，如果用户没有收到验证邮件，则需要一条路由来重新发送验证邮件。

6.4. 加密解密

6.4.1. 简介

Laravel 的加密机制使用的是 OpenSSL 所提供的 AES-256 和 AES-128 加密。强烈建议你使用 Laravel 内建的加密工具，而不是用其它的加密算法。所有 Laravel 加密之后的结果都会使用消息认证码 (MAC) 签名，使其底层值不能在加密后再次修改。

6.4.2. 设置

在使用 Laravel 的加密工具之前，你必须先设置 config/app.php 配置文件中的 key 配置项。该配置项由环境变量 APP_KEY 设定。你应当使用 php artisan key:generate 命令来生成该变量的值，key:generate 命令将使用 PHP 的安全随机字节生成器为你的应用程序构建加密安全密钥。通常情况下，在 安装 Laravel 中会为你生成 APP_KEY 环境变量的值。

6.4.3. 基本用法

加密一个值。

你可以使用 Crypt 门面提供的 encryptString 来加密一个值。所有加密的值都使用 OpenSSL 的 AES-256-CBC 来进行加密。此外，所有加密过的值都会使用消息认证码 (MAC) 来签名，以检测加密字符串是否被篡改过：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use App\Models\User;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Crypt;



class UserController extends Controller
```

```
{
    /**
     * Store a secret message for the user.
     *
     * @param  Request  $request
     * @param  int  $id
     * @return Response
     */
    public function storeSecret(Request $request, $id)
    {
        $user->fill([
            'secret' => Crypt::encryptString($request->secret),
        ])->save();
    }
}
```

解密一个值

你可以使用 Crypt 门面提供的 decryptString 来进行解密。

```
use Illuminate\Contracts\Encryption\DecryptException;
use Illuminate\Support\Facades\Crypt;

try {
    $decrypted = Crypt::decryptString($encryptedValue);
} catch (DecryptException $e) {
```

```
    //
}
```

6.5. 哈希

6.5.1. 简介

Laravel Hash facade 为存储用户密码提供了安全的 Bcrypt 和 Argon2 哈希加密方式。如果你在你的 Laravel 应用程序中使用了内置的 LoginController 和 RegisterController 类，那么它们默认使用 Bcrypt 进行注册和身份认证。

6.5.2. 配置

你可以在 config/hashing.php 配置文件中配置默认哈希驱动程序。目前支持三种驱动程序：Bcypt 和 Argon2 (Argon2i and Argon2id variants)。

6.5.3. 基本用法

你可以通过调用 Hash facade 的 make 方法来加密你的密码：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Hash;


class UpdatePasswordController extends Controller
{

    /**
     * 更新用户密码
     *
```

```
     * @param  Request  $request
     * @return Response
     */
    public function update(Request $request)
    {
        // 验证新密码的长度...

        $request->user()->fill([
            'password' => Hash::make($request->newPassword)
        ])->save();
    }
}
```

调整 Bcrypt 加密函数

如果使用 Bcypt 算法，你可以在 make 方法中使用 rounds 选项来配置该算法的加密系数。然而，对大多数应用程序来说，默认值就足够了：

```
$hashed = Hash::make('password', [
    'rounds' => 12,
]);
```

6.6. 密码重置

6.6.1. 简介

大多数 Web 应用程序都会为用户提供重置密码的功能。 Laravel 已经提供了便捷的服务来发送密码重置链接和安全重置密码，而不需要您为每个应用程序重新实现此功能。

6.6.1.1. 数据库准备

必须创建一个表来存储应用程序的密码重置令牌。此表的迁移包含在默认的 Laravel 应用程序中，因此您只需迁移您的数据库即可创建此表：

```
php artisan migrate
```

6.6.2. 路由

为了正确实现对允许用户重置密码的支持，我们需要定义几个路由。首先，我们需要一对路由来处理允许用户通过他们的电子邮件地址请求密码重置链接。其次，一旦用户访问通过电子邮件发送给他们的密码重置链接并完成密码重置表单，我们将需要一对路由来处理实际重置密码

6.6.2.1. 请求密码重置链接

首先，我们将定义请求密码重置链接所需的路由。

6.6.2.2. 重置密码

接下来，我们将定义用户点击重置密码邮件中的链接，进行重置密码所需要的一些路由。

6.6.3. 自定义

您可以轻松修改用于向用户发送密码重置链接的通知类。 首先，覆盖您的 App\Models\User 模型上的 sendPasswordResetNotification 方法。

第七章 数据库

7.1. 快速入门

7.1.1. 简介

几乎所有的应用程序都需要和数据库进行交互。Laravel 为此提供了一套非常简单易用的数据库交互方式。开发者可以使用原生 SQL、查询构造器以及 Eloquent ORM 等方式与数据库交互。目前，Laravel 为以下四种数据库提供了官方支持：

MySQL 5.7+ (版本策略)

PostgreSQL 9.6+ (版本策略)

SQLite 3.8.8+

SQL Server 2017+ (版本策略)

7.1.1.1. 配置

数据库的配置文件在 config/database.php 文件中。你可以在这个文件中配置所有的数据库连接，并指定默认的数据库连接。该文件中的大部分配置都基于项目的环境变量，且提供了大部分 Laravel 所支持的数据库配置示例。

在默认情况下，Laravel 的示例 环境配置 使用了 Laravel Sail，Laravel Sail 是一种用于在本地开发 Laravel 应用的 Docker 配置。但你依然可以根据本地数据库的需要修改数据库配置。

7.1.1.1.1. SQLite 配置

SQLite 数据库本质上只是一个存在你文件系统上的文件。你可以通过 touch 命令来建立一个新的 SQLite 数据库，如： touch database/database.sqlite. 建立数据库之后，你就可以很简单地使用数据库的绝对路径来配置 DB_DATABASE 环境变量，使其指向这个新创建的数据库：

```
DB_CONNECTION=sqlite
```

```
DB_DATABASE=/absolute/path/to/database.sqlite
```

7.1.1.1.2. URL 形式配置

通常，数据库连接使用多个配置项进行配置，例如 host、database、username、password 等。这些配置项都拥有对应的环境变量。这意味着你需要在生产服务器上管理多个不同的环境变量。

部分数据库托管平台（如 AWS 和 Heroku）会提供了包含所有连接信息的数据库「URL」。它们通常看起来像这样：

```
mysql://root:password@127.0.0.1/forge?charset=UTF-8
```

7.1.1.2. 读写分离

有时候你可能会希望使用一个数据库连接来执行 SELECT 语句，而 INSERT、UPDATE 和 DELETE 语句则由另一个数据库连接来执行。在 Laravel 中，无论你是使用原生 SQL 查询、查询构造器 或是 Eloquent ORM，都能轻松实现读写分离。

为了弄明白如何配置读写分离，我们先来看个例子：

```
'mysql' => [
    'read' => [
        'host' => [
            '192.168.1.1',
            '196.168.1.2',
        ],
    ],
    'write' => [
```

```
        'host' => [
            '196.168.1.3',
        ],
    ],
    'sticky' => true,
    'driver' => 'mysql',
    'database' => 'database',
    'username' => 'root',
    'password' => '',
    'charset' => 'utf8mb4',
    'collation' => 'utf8mb4_unicode_ci',
    'prefix' => '',
],
```

注意在以上的例子中，配置数组中增加了三个键，分别是 read、write 和 sticky。read 和 write 都包含一个键为 host 的数组。而 read 和 write 的其他数据库选项都在键为 mysql 的数组中。

如果你想重写主数组中的配置，只需要修改 read 和 write 数组即可。所以，这个例子中：192.168.1.1 和 192.168.1.2 将作为续连接主机，而 192.168.1.3 将作为写连接主机。

7.1.2. 运行原生 SQL 查询

配置好数据库连接后，就可以使用 DB 门面来运行查询。DB 门面为每种操作提供了相应方法。select, update, insert, delete 和 statement。

执行 Select 查询

你可以使用 DB 门面的 select 方法来运行基础的查询语句：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use Illuminate\Support\Facades\DB;


class UserController extends Controller
{
    /**
     * 显示应用程序中所有用户的列表
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::select('select * from users where active = ?',
[1]);


        return view('user.index', ['users' => $users]);
    }
}
```

传递给 select 方法的第一个参数就是一个原生的 SQL 查询，而第二个参数则是需要绑定到查询中的参数值。

7.1.2.1. 使用多个数据库连接

如果你在配置文件 config/database.php 中定义了多个数据库连接的话，你可

以通过 DB Facade 的 connection 方法来使用它们。传递给 connection 方法的连接名称应该是你在 config/database.php 里或者通过 config 助手函数在运行时配置的连接之一：

```
use Illuminate\Support\Facades\DB;

$users = DB::connection('sqlite')->select(...);
```

你也可以使用一个连接实例上的 getPdo 方法访问底层的 PDO 实例：

```
$pdo = DB::connection()->getPdo();
```

7.1.2.2. 监听查询事件

如果你想要获取程序执行的每一条 SQL 语句，可以使用 listen 方法。该方法对查询日志和调试非常有用，你可以在 服务提供者 中注册查询监听器。

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\DB;

use Illuminate\Support\ServiceProvider;


class AppServiceProvider extends ServiceProvider
{
    /**
     * 注册所有应用的服务
     *
     * @return void
     */
    public function register()
```

```
    {
        //
    }


    /**
     * 引导所有应用的服务
     *
     * @return void
     */
    public function boot()
    {
        DB::listen(function ($query) {
            // $query->sql
            // $query->bindings
            // $query->time
        });
    }
}
```

7.1.3. 数据库事务

想要在数据库事务中运行一系列操作，你可以使用 DB 门面的 transaction 方法。如果在事务的闭包中出现了异常，事务将会自动回滚。如果闭包执行成功，事务将会自动提交。在使用 transaction 方法时不需要手动回滚或提交：

```
use Illuminate\Support\Facades\DB;
```

```
DB::transaction(function () {
    DB::update('update users set votes = 1');

    DB::delete('delete from posts');
});
```

7.1.3.1. 处理死锁

transaction 方法接受一个可选的第二次参数，该参数用来表示事务发生死锁重复执行的次数。一旦定义的次数尝试完毕，就会抛出一个异常：

```
DB::transaction(function () {
    DB::table('users')->update(['votes' => 1]);
    DB::table('posts')->delete();
}, 5);
```

7.1.3.2. 手动处理事务

如果你想要手动处理事务并完全控制回滚和提交，可以使用 DB 门面提供的 beginTransaction 方法：

```
DB::beginTransaction();
```

您可以通过 rollBack 方法回滚事务。

```
DB::rollBack();
```

最后，您可以通过 commit 方法提交事务。

```
DB::commit();
```

7.1.4. 连接到数据库 CLI

如果您想连接到数据库的 CLI，则可以使用 db Artisan 命令：

```
php artisan db
```

如果需要，可以指定数据库连接名称以连接到不是默认连接的数据库连接：

```
php artisan db mysql
```

7.2. 查询构建器

7.2.1. 简介

Laravel 的数据库查询生成器为创建和运行数据库查询提供了方便、流畅的界面。它可以用于执行应用程序中的大多数数据库操作，并与 Laravel 支持的所有数据库系统完美配合。

Laravel 9 查询生成器使用 PDO 参数绑定来保护应用程序免受 SQL 注入攻击。无需清理或清理作为查询绑定传递给查询生成器的字符串。

7.2.2. 获取结果集

我们可以从 DB 门面的 table 方法开始，table 方法为给定表返回一个流式查询构建器实例，该实例允许你在查询上链接多个约束条件并返回最终查询结果。

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use Illuminate\Support\Facades\DB;


class UserController extends Controller

{

    /**
```

```
     * 显示所有应用程序的用户列表。
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::table('users')->get();

        return view('user.index', ['users' => $users]);
    }
}
```

get 方法返回一个包含 Illuminate\Support\Collection 实例的结果，其中每一条记录都是 PHP stdClass 对象的一个实例。你可以通过对象属性的方式来获取每个字段的值。

```
foreach ($users as $user) {
    echo $user->name;
}
```

7.2.2.1. 分块结果

如果您需要处理成千上万的数据库记录，请考虑使用 DB 提供的 chunk 方法。这个方法一次检索一小块结果，并将每个块反馈到闭包函数中进行处理。例如，让我们以一次 100 条记录的块为单位检索整个 users 表。

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {
    foreach ($users as $user) {
```

```
        //
    }
});
```

也可以通过 Closure 中返回 false 来终止继续获取分块结果。

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {
    // Process the records...
    return false;
});
```

7.2.2.2. 聚合

查询构建器还提供了多种检索聚合值的方法，例如 count, max, min, avg, 和 sum。您可以在构建查询后调用这些方法中的任何一个：

```
$users = DB::table('users')->count();
$price = DB::table('orders')->max('price');
```

当然，你也可以将这些聚合方法与其他的查询语句相结合：

```
$price = DB::table('orders')
                ->where('finalized', 1)
                ->avg('price');
```

判断记录是否存在

除了通过 count 方法可以确定查询条件的结果是否存在之外，还可以使用 exists 和 doesntExist 方法。

```
return DB::table('orders')->where('finalized', 1)->exists();
return DB::table('orders')->where('finalized', 1)->doesntExist();
```

7.2.3. 查询（Select）

指定一个 Select 语句

我们可能不是总是想要获取数据表的所有有列。使用 select 方法，你可以自定义一个 select 查询语句来查询指定的字段。

```
$users = DB::table('users')->select('name', 'email as
user_email')->get();
```

distinct 方法会强制让查询返回的结果不重复。

```
$users = DB::table('users')->distinct()->get();
```

7.2.4. 原生表达式

有时可能需要在查询中插入任意字符串。要创建原始字符串表达式，可以使用 DB facade 提供的 raw 方法：

```
$users = DB::table('users')
    ->select(DB::raw('count(*) as user_count, status'))
    ->where('status', '<>', 1)
    ->groupBy('status')
    ->get();
```

7.2.5. 连接（Joins）

7.2.5.1. 内连接

要实现一个简单的“内连接”，您可以使用查询构建器实例上的 join 方法，传递给 join 方法的第一个参数是您需要连接到的表名，剩余的其他参数则是为连接指定的列约束，当然，正如您所看到的，您可以在单个查询中连接多张表。

```
$users = DB::table('users')
```

```
    ->join('contacts', 'users.id', '=', 'contacts.user_id')
    ->join('orders', 'users.id', '=', 'orders.user_id')
    ->select('users.*', 'contacts.phone', 'orders.price')
    ->get();
```

7.2.5.2. 左连接/右连接

如果您想要执行左连接或右连接，而不是内连接，可以使用 leftJoin 或 rightJoin 方法。这些方法和 join 方法的用法一样。

```
$users = DB::table('users')
    ->leftJoin('posts', 'users.id', '=', 'posts.user_id')
    ->get();
$users = DB::table('users')
    ->rightJoin('posts', 'users.id', '=', 'posts.user_id')
    ->get();
```

7.2.5.3. 交叉连接

要执行交叉连接可以使用 crossJoin 方法，传递您想要交叉连接的表名到该方法即可。交叉连连接在第一张表和被连接表之间生成一个笛卡尔积。

```
$users = DB::table('sizes')
    ->crossJoin('colours')
    ->get();
```

7.2.5.4. 高级连接语句

您还可以指定更多的高级连接子句，传递一个闭包到 join 方法作为第二个参

数，该闭包将会接收一个 JoinClause 对象用于指定 join 子句约束。

```
DB::table('users')
    ->join('contacts', function ($join) {
        $join->on('users.id', '=', 'contacts.user_id')
        ->orOn(...);
    })
    ->get();
```

7.2.5.5. 子查询连接

您可以使用 joinSub、leftJoinSub 和 rigthJoinSub 方法将查询和一个子查询进行连接，每个方法都接收三个参数——子查询、表别名和定义关联字段的闭包。

```
$latestPosts = DB::table('posts')
       ->select('user_id', DB::raw('MAX(created_at) as
last_post_created_at'))
       ->where('is_published', true)
       ->groupBy('user_id');
$users = DB::table('users')
       ->joinSub($latestPosts, 'latest_posts', function($join) {
           $join->on('users.id', '=', 'latest_posts.user_id');
       })->get();
```

7.2.6. 联合（Union）

查询构建器还提供了联合两个查询的快捷方式，比如，您可以先创建一个查询，然后使用 union 方法将其和第二个查询进行联合。

```
$first = DB::table('users')
    ->whereNull('first_name');
$users = DB::table('users')
    ->whereNull('last_name')
    ->union($first)
    ->get();
```

注：unionAll 方法也是有效的，并且和 union 使用方式相同。

7.2.7. Where 子句

简单的 Where 语句

在构造 where 查询实例中，你可以使用 where 方法。调用 where 最基本的方式是需要传递三个参数：第一个参数列名，第二个参数是任意一个数据库系统支持的运算符，第三个是该列要比较的值。

例如，下面是一个要验证 votes 字段的值等于 100 的查询：

```
$users = DB::table('users')->where('votes', '=', 100)->get();
```

7.2.7.1. 参数分组

有时候你需要创建更高级的 where 子句，例如「where exists」或者嵌套的参数分组。 Laravel 的查询构造器也能够处理这些。下面，让我们看一个在括号中进行分组约束的例子：

```
$users = DB::table('users')
           ->where('name', '=', 'John')
           ->where(function ($query) {
               $query->where('votes', '>', 100)
                     ->orWhere('title', '=', 'Admin');
```

```
        })
        ->get();
```

你可以看到，通过使用闭包写入 where 的方法来创建一个查询构造器，用来约束一个分组。这个闭包接收一个查询实例，你可以使用这个实例来设置应该包含的约束。上面的例子将生成以下 SQL 语句。

```
select * from users where name = 'John' and (votes > 100 or title = 'Admin')
```

7.2.7.2. Where exists 子句

whereExists 方法允许你使用 where exists SQL 语句。whereExists 方法接收一个 Closure 参数，该闭包获取一个查询构建器实例从而允许你定义放置在 exists 字句中查询：

```
$users = DB::table('users')
        ->whereExists(function ($query) {
            $query->select(DB::raw(1))
                  ->from('orders')
                  ->whereRaw('orders.user_id = users.id');
        })
        ->get();
```

上述查询将产生如下的 SQL 语句：

```
select * from users
where exists (
    select 1 from orders where orders.user_id = users.id
)
```

7.2.7.3. 子查询 Where 子句

有时你可能需要构造一个 where 语句，将子查询的结果与一个给定值进行比较。你可以通过给 where 方法传递一个闭包以及给定值实现。举例来说，下面的查询语句将会获取到所有属于最近一个会员类型的所有用户。

```
use App\User;

$users = User::where(function ($query) {
    $query->select('type')
        ->from('membership')
        ->whereColumn('user_id', 'users.id')
        ->orderByDesc('start_date')
        ->limit(1);
}, 'Pro')->get();
```

7.2.7.4. JSON Where 子句

Laravel 也支持查询 JSON 类型的字段（仅在对 JSON 类型支持的数据库上）。目前，本特性支持包括 MySQL 5.7、PostgreSQL、SQL Server 2016、和 SQLite 3.9.0 (with the JSON1 extension)。

```
$users = DB::table('users')
                ->where('options->language', 'en')
                ->get();


$users = DB::table('users')
                ->where('preferences->dining->meal', 'salad')
                ->get();
```

7.2.8. 排序、分组

7.2.8.1. orderBy

orderBy 方法允许你通过给定字段对结果集进行排序。 orderBy 的第一个参数应该是你希望排序的字段，第二个参数控制排序的方向，可以是 asc 或 desc：

```
$users = DB::table('users')
                ->orderBy('name', 'desc')
                ->get();
```

如果你需要使用多个字段进行排序，你可以多次引用 orderBy：

```
$users = DB::table('users')
                ->orderBy('name', 'desc')
                ->orderBy('email', 'asc')
                ->get();
```

7.2.9. 条件子句

有时候你可能想要子句只适用于某个情况为真时才执行查询。例如你可能只想给定值在请求中存在的情况下才应用 where 语句。你可以通过使用 when 方法来实现。

```
$role = $request->input('role');

$users = DB::table('users')
                ->when($role, function ($query, $role) {
                    return $query->where('role_id', $role);
                })
```

```
            ->get();
```

when 方法只有在每一个参数为 true 的时候才执行给的闭包。如果第一个参数为 false，那么这个闭包将不会被执行。

你可以传递另一个闭包作为 when 方法的第三个参数。该闭包会在第一个参数为 false 的情况下执行。为了说明如何使用这个特征，我们来配置一个查询的默认排序。

```
$sortBy = null;


$users = DB::table('users')
            ->when($sortBy, function ($query, $sortBy) {
                return $query->orderBy($sortBy);
            }, function ($query) {
                return $query->orderBy('name');
            })
            ->get();
```

7.2.10. 插入

查询构造器还提供了 insert 方法用来插入记录到数据库中。insert 方法接收数组形式的字段名和字段值进行插入操作。

```
DB::table('users')->insert(
    ['email' => 'john@example.com', 'votes' => 0]
);
```

你甚至可以将二维数组传递给 insert 方法，依次将多个记录插入到表中。

```
DB::table('users')->insert([
```

```
    ['email' => 'taylor@example.com', 'votes' => 0],
    ['email' => 'dayle@example.com', 'votes' => 0],
]);
```

自增 IDs

如果数据表有自增 ID，使用 insertGetId 方法来插入记录可以返回 ID 值。

```
$id = DB::table('users')->insertGetId(
    ['email' => 'john@example.com', 'votes' => 0]
);
```

7.2.11. 更新

当然，除了插入记录到数据库中，查询构造器也可以通过 udpate 方法更新已有的记录。update 方法和 insert 方法一样，接受包含要更新的字段及值的数组。你可以通过 where 子句对 update 查询进行约束。

```
$affected = DB::table('users')
              ->where('id', 1)
              ->update(['votes' => 1]);
```

7.2.11.1.更新或新增

有时你可能希望更新数据库的现有记录，或者如果不存在匹配记录则创建它。在这种情况下，可以使用 updateOrInsert 方法。updateOrInsert 方法接受两个参数：一个用于查找记录的条件数组，以及一个包含要更改记录的键值对数组。

updateOrInsert 方法将首先尝试使用第一个参数的键和值对来查询匹配的数据库记录。如果记录存在，则使用第二个参数中的值去更新记录。如果找不到记录，将插入一个新记录，新增的数据是两个数组的集合。

```
DB::table('users')
```

```
    ->updateOrInsert(
        ['email' => 'john@example.com', 'name' => 'John'],
        ['votes' => '2']
    );
```

7.2.11.2.更新 JSON

更新 JSON 字段时，你可以使用 -> 语法访问 JSON 对象中相应的值。注意，此操作只能支持 MySQL 5.7+ 和 PostgreSQL 9.5+

```
$affected = DB::table('users')
              ->where('id', 1)
              ->update(['options->enabled' => true]);
```

7.2.11.3.自增自减

查询构造器还为给定字段的递增或递减提供了便捷的方式。此方法提供了一个比手动编写 update 语句更具表达力且更精炼的接口。

这两种方法都至少接收一个参数：需要修改的列。可选的第二个参数用于控制列递增或递减的量：

```
DB::table('users')->increment('votes');

DB::table('users')->increment('votes', 5);

DB::table('users')->decrement('votes');

DB::table('users')->decrement('votes', 5);
```

7.2.12. 删除

查询构造器也可以使用 delete 方法从表中删除记录。在使用 delete 前，可以添加 where 子句来约束 delete 语法。

```
DB::table('users')->delete();


DB::table('users')->where('votes', '>', 100)->delete();
```

你可以使用 truncate 方法来清空整个表，这将移除所有的数据并重置所有的自增 ID 为 0。

```
DB::table('users')->truncate();
```

7.2.13. 悲观锁

查询构造器也包含了一些能够帮助你在 select 语句中实现 「悲观锁」的函数。要执行一个含有 「共享锁」的语句，你可以在查询中使用 sharedLock 方法。共享锁可防止指定的数据列被篡改，直到事务被提交为止。

```
DB::table('users')->where('votes', '>', 100)->sharedLock()->get();
```

7.2.14. 调试

在绑定查询的时候，你可以使用 dd 或 dump 方法来输出查询绑定和 SQL。dd 方法将会显示调试信息并终止执行请求，而 dump 方法则会显示调试信息并允许请求继续执行。

```
DB::table('users')->where('votes', '>', 100)->dd();


DB::table('users')->where('votes', '>', 100)->dump();
```

7.3. 分页

7.3.1. 简介

在其他框架中，分页可能非常痛苦。我们希望 Laravel 的分页方法能带来一股新鲜空气。Laravel 的分页器集成了 query builder 和 Eloquent ORM，并提供了方便、易于使用的数据库记录分页，无需配置。

默认情况下，由分页器生成的 HTML 与 Tailwind CSS 框架 兼容，然而，引导分页支持也是可用的。

7.3.2. 基本用法

7.3.2.1. 查询构造器分页

有几种方法可以对结果进行分页，最简单的方法是在 query builder 或 Eloquent query。上使用 paginate 方法 ，paginate 方法根据用户查看的当前页面自动设置查询的 「limit」 和 「offset」 ， 默认情况下，通过 HTTP 请求中的 page 查询字符串参数的值检测当前页面， Laravel 会自动检测这个值，它也会自动插入到分页器生成的链接中。

在下面的例子中，传递给 paginate 方法的唯一参数是你想要在一页中显示的记录数。在此例中，我们希望「每页」显示 15 条数据：

```
<?php

namespace App\Http\Controllers;


use App\Http\Controllers\Controller;

use Illuminate\Support\Facades\DB;


class UserController extends Controller

{

    /**
```

```
     * 显示所有用户列表.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        return view('user.index', [
            'users' => DB::table('users')->paginate(15)
        ]);
    }
}
```

如果你只需要在分页视图中简单地显示 「下一页」和「上一页」的链接，你可以使用 simplePaginate 方法来执行更高效地查询。这在数据量很大且不需要在渲染视图时显示每页的页码时非常有用。

```
$users = DB::table('users')->simplePaginate(15);
```

7.3.2.2. Eloquent 分页

你也可以对 Eloquent 查询结果进行分页。在下面的例子中，我们将 App\Models\User 模型按每页 15 条记录进行分页。如你所见，其语法与查询构造器分页基本相同：

```
use App\Models\User;

$users = User::paginate(15);
```

当然，你也可以在调用 paginate 方法之前为查询添加其他约束，例如 where 子句：

```
$users = User::where('votes', '>', 100)->paginate(15);
```

7.3.2.3. 游标分页

虽然 paginate 和 simplePaginate 使用 SQL「offset」 子句创建查询，但游标分页通过构造「where」子句来工作，这些子句比较查询中包含的有序列的值，提供所有可用的最有效的数据库性能 Laravel 的分页方法。 这种分页方法特别适合大型数据集和「无限」滚动用户界面。

与基于偏移量的分页在分页器生成的 URL 的查询字符串中包含页码不同，基于游标的分页在查询字符串中放置一个「游标」字符串。游标是一个编码字符串，包含下一个分页查询应该开始分页的位置和它应该分页的方向：

```
http://localhost/users?cursor=eyJpZCI6MTUsI19wb2ludHNUb05leHRJdGVtcyI6dHJ1ZX0
```

7.3.2.4. 手动创建分页

有时你可能希望手动创建分页，并传递一个包含数据的数组给它。这可以通过手动创建 Illuminate\Pagination\Paginator 或者 Illuminate\Pagination\LengthAwarePaginator 实例来实现，这取决于你的需要。

Paginator 不需要知道数据的总数。然而，你也无法通过 Paginator 获取最后一页的索引。而 LengthAwarePaginator 接受和 Paginator 几乎相同的参数，不过，它会计算数据的总数。

或者说，Paginator 相当于查询构造器或者 Eloquent ORM 分页的 simplePaginate 方法，而 LengthAwarePaginator 相当于 paginate 方法。

7.3.3. 显示分页视图

当调用 paginate 方法时，你会得到一个 Illuminate\Pagination\LengthAwarePaginator 实例。而调用 simplePaginate 方法时，你会得到 Illuminate\Pagination\Paginator 的实例。

7.3.3.1. 调整分页链接接口

在使用分页器展示分页链接时，将展示当前页前后各三页的链接。如果有需要，你可以通过 onEachSide 方法来控制每侧显示多少个链接：

```
{{ $users->onEachSide(5)->links() }}
```

7.3.3.2. 将结果转换为 JSON

Laravel 分页器类实现了 Illuminate\Contracts\Support\Jsonable 接口契约，提供了 toJson 方法，所以可以方便的将分页结果转换为 JSON 。你也可以通过来自路由或控制器的操作返回分页实例来将其转换为 JSON：

```
use App\Models\User;

Route::get('/users', function () {
    return User::paginate();
});
```

7.3.4. 自定义分页视图

默认情况下，分页器渲染的视图与 Tailwind CSS 兼容。不过，如果你并非使用 Tailwind，你也可以自由地定义用于渲染这些链接的视图。

7.3.4.1. 使用 Bootstrap

Laravel 同样包含使用 Bootstrap CSS 构建的分页视图。要使用这些视图来替代默认的 Tailwind 视图，你可以在 App\Providers\AppServiceProvider 服务提供者中的 boot 方法内调用分页器的 useBootstrapFour 或 useBootstrapFive 方法：

```
use Illuminate\Pagination\Paginator;



/**
 * Bootstrap any application services.
```

```
 *
 * @return void
 */
public function boot()
{
    Paginator::useBootstrapFive();
    Paginator::useBootstrapFour();
}
```

7.3.5. 分页器实例方法

每个分页器实例都可以通过以下方法提供更多分页信息。

方法	描述
`$results->count()`	获取当前面页数据的数量
`$results->currentPage()`	获取当前页页码
`$results->firstItem()`	获取结果集中第一条数据的结果编号

7.4. 迁移

7.4.1. 简介

迁移就像是数据库的版本控制，让你的团队能够轻松地去定义和共享程序的数据库结构。迁移通常配合 Laravel 的结构生成器，可以轻松生成应用程序的数据库结构。如果团队中有成员在他的本地数据库环境中手动的添加了某个字段，那么你将会面对如何解决数据库迁移的问题。

Laravel 9 Schema facade 提供了数据库相关的支持，可以在所有 Laravel 支持的数据库管理系统中创建和操作表。

7.4.2. 生成迁移

你可以使用 make:migration Artisan command 生成数据库迁移。新的迁移将放在你的 database/migrations 目录每个迁移文件名都包含一个时间戳，允许 Laravel 确定迁移的顺序：

```
php artisan make:migration create_users_table
```

7.4.2.1. 整合迁移

在构建应用程序时，可能会随着时间的推移积累越来越多的迁移。这可能会导致你的 database/migrations 目录因为数百次迁移而变得臃肿。你如果愿意的话，可以将迁移「压缩」到单个 SQL 文件中。如果你想这样做，请先执行 schema:dump 命令：

```
php artisan schema:dump

# 转储当前数据库架构并删除所有现有迁移...

php artisan schema:dump --prune
```

7.4.3. 迁移结构

迁移类包含两个方法：up 和 down 。up 方法用于向数据库中添加新表、列或索引，而 down 方法用于撤销 up 方法执行的操作。.

在这两种方法中，可以使用 Laravel 模式构建器来富有表现力地创建和修改表。要了解 Schema 构建器上可用的所有方法，查看其文档。例如，以下迁移会创建一个 flights 表：

```
<?php

use Illuminate\Database\Migrations\Migration;

use Illuminate\Database\Schema\Blueprint;

use Illuminate\Support\Facades\Schema;
```

```
class CreateFlightsTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('airline');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
```

```
    {
        Schema::drop('flights');
    }
}
```

7.4.4. 运行迁移

执行 Artisan 命令 migrate，来运行所有未执行过的迁移：

```
php artisan migrate
```

7.4.4.1. 回滚迁移

如果要回滚最后一次迁移操作，可以使用 Artisan 命令 rollback。该命令会回滚最后「一批」的迁移，这可能包含多个迁移文件：

```
php artisan migrate:rollback
```

7.4.5. 数据表

7.4.5.1. 创建数据表

接下来我们将使用 Schema 的 create 方法创建一个新的数据表。create 接受两个参数：第一个参数是表名，而第二个参数是一个闭包，该闭包接受一个用来定义新数据表的 Blueprint 对象：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::create('users', function (Blueprint $table) {
    $table->id();
```

```
    $table->string('name');

    $table->string('email');

    $table->timestamps();
});
```

创建表时，可以使用数据库结构构建器的 列方法 来定义表的列。

7.4.5.2. 更新数据表

Schema 门面的 table 方法可用于更新现有表。与 create 方法一样，table 方法接受两个参数：表的名称和接收可用于向表添加列或索引的 Blueprint 实例的闭包：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;


Schema::table('users', function (Blueprint $table) {
    $table->integer('votes');
});
```

7.4.5.3. 重命名/删除数据表

要重命名一个存在的数据库表，请使用 rename 方法：

```
use Illuminate\Support\Facades\Schema;

Schema::rename($from, $to);
```

7.4.6. 字段

7.4.6.1. 创建字段

门面 Schema 的 table 方法可用于更新表。与 create 方法一样， table 方法

接受两个参数：表名和一个闭包，该闭包接收一个 illumb\Database\Schema\Blueprint 实例，可以使用该实例向表中添加列：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::table('users', function (Blueprint $table) {
    $table->integer('votes');
});
```

7.4.6.2. 可用的字段类型

lluminate\Database\Schema\Blueprint 提供了多种方法，用来创建表中对应类型的列。下面列出了所有可用的方法：

bigIncrements

bigInteger

7.4.6.3. 字段修饰符

除了上面列出的列类型外，在向数据库表添加列时还有几个可以使用的「修饰符」。例如，如果要把列设置为要使列为「可空」，你可以使用 nullable 方法：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::table('users', function (Blueprint $table) {
    $table->string('email')->nullable();
```

```
});
```

7.4.6.4. 修改字段

在修饰字段之前，请确保你已经通过 Composer 包管理器安装了 doctrine/dbal 包。Doctrine DBAL 库用于确定字段的当前状态，并创建对该字段进行指定调整所需的 SQL 查询：

```
composer require doctrine/dbal
```

7.4.6.5. 删除字段

你可以使用结构生成器上的 dropColumn 方法来删除字段。如果你的应用程序使用的是 SQLite 数据库，你必须在调用 dropColumn 方法之前通过 Composer 包管理器安装了 doctrine/dbal 包：

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('votes');
});
```

7.4.7. 索引

7.4.7.1. 创建索引

结构生成器支持多种类型的索引。下面的例子中新建了一个值唯一的 email 字段。我们可以将 unique 方法链式地添加到字段定义上来创建索引：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::table('users', function (Blueprint $table) {
```

```
    $table->string('email')->unique();
});
```

7.4.7.2. 重命名索引

若要重命名索引，你需要调用 renameIndex 方法。此方法接受当前索引名称作为其第一个参数，并将所需名称作为其第二个参数:

```
$table->renameIndex('from', 'to');
```

7.4.7.3. 删除索引

若要删除索引，则必须指定索引的名称。Laravel 默认会自动将数据表名称、索引的字段名及索引类型简单地连接在一起作为名称。

7.4.7.4. 外键索引

Laravel 还支持创建用于在数据库层中的强制引用完整性的外键约束。例如，让我们在 posts 表上定义一个引用 users 表的 id 字段的 user_id 字段：

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::table('posts', function (Blueprint $table) {
    $table->unsignedBigInteger('user_id');

    $table->foreign('user_id')->references('id')->on('users');
});
```

7.4.8. 事件

为方便起见，每个迁移操作都会派发一个 事件。

7.5. 数据填充

7.5.1. 简介

Laravel 内置了一个可为你的数据库填充测试数据的填充类。所有的填充类都放在 database/seeds 目录下。Laravel 默认定义了一个 DatabaseSeeder 类。通过这个类，你可以用 call 方法来运行其他的 seed 类，从而控制数据填充的顺序。

7.5.2. 编写 Seeders

运行 Artisan 命令 make:seeder 生成 Seeder，框架生成的 seeders 都放在 database/seeds 目录下：

```
php artisan make:seeder UserSeeder
```

7.5.2.1. 使用模型工厂

当然，手动为每个模型填充指定属性很麻烦，你可以使用 模型工厂 轻松地生成大量数据库数据。首先，阅读 模型工厂文档 来学习如何定义工厂文件。

例如，创建 50 个用户并为每个用户创建关联：

```
use App\Models\User;


/**
 * 执行数据填充
 *
 * @return void
 */
```

```
public function run()
{
    User::factory()
            ->count(50)
            ->hasPosts(1)
            ->create();
}
```

7.5.2.2. 调用其他 Seeders

在 DatabaseSeeder 类中，你可以使用 call 方法来运行其它的 seed 类。使用 call 方法可以将数据填充拆分成多个文件，这样就不会使单个 seeder 变得非常大。只需简单传递要运行的 seeder 类名称即可：

```
/**
 * 执行数据填充
 *
 * @return void
 */
public function run()
{
    $this->call([
        UserSeeder::class,
        PostSeeder::class,
        CommentSeeder::class,
    ]);
```

```
}
```

7.5.3. 运行 Seeders

你可以使用 Artisan 命令 db:seed 来填充数据库。默认情况下， db:seed 命令将运行 Database\Seeders\DatabaseSeeder 类，这个类又可以调用其他 seed 类。不过，你也可以使用 --class 选项来指定一个特定的 seeder 类：

```
php artisan db:seed

php artisan db:seed --class=UserSeeder
```

7.6. Redis

7.6.1. 简介

Redis 是一个开源的，高级键值对存储数据库。 包含的数据类型有 字符串 ，hash, 列表 ，集合 ，和 有序集合.

在将 Redis 与 Laravel 一起使用前，我们鼓励你通过 PECL 安装并使用 PhpRedis 尽管扩展安装起来更复杂，但对于大量使用 Redis 的应用程序可能会产生更好的性能。

如果你正在使用 Laravel Sail, 这个扩展已经实现安装好在你的 Docker 容器中了。

如果你不能安装 PHPRedis 扩展，你或许可以使用 composer 安装 predis/predis 包。Predis 是一个完全用 PHP 编写的 Redis 客户端，不需要任何额外的扩展：

```
composer require predis/predis
```

7.6.2. 配置

在你的应用中配置 Redis 信息，你要在 config/database.php 文件中进行配置。

在该文件中，你将看到一个 Redis 数组包含了你的 Redis 配置信息。

```
'redis' => [

    'client' => env('REDIS_CLIENT', 'phpredis'),

    'default' => [
        'host' => env('REDIS_HOST', '127.0.0.1'),
        'password' => env('REDIS_PASSWORD'),
        'port' => env('REDIS_PORT', 6379),
        'database' => env('REDIS_DB', 0),
    ],

    'cache' => [
        'host' => env('REDIS_HOST', '127.0.0.1'),
        'password' => env('REDIS_PASSWORD'),
        'port' => env('REDIS_PORT', 6379),
        'database' => env('REDIS_CACHE_DB', 1),
    ],

],
```

7.6.2.1. 集群

如果你的应用使用 Redis 集群，你应该在 Redis 配置文件中用 clusters 键来定义集群。这个配置键默认没有，所以你需要在 config/database.php 配置文件

中手动创建：

```
'redis' => [
'client' => env('REDIS_CLIENT', 'phpredis'),
'clusters' => [
    'default' => [
        [
            'host' => env('REDIS_HOST', 'localhost'),
            'password' => env('REDIS_PASSWORD'),
            'port' => env('REDIS_PORT', 6379),
            'database' => 0,
        ],
    ],
],
```

默认情况下，集群可以在节点上实现客户端分片，允许你实现节点池以及创建大量可用内存。这里要注意，客户端共享不会处理失败的情况；因此，这个功能主要适用于从另一个主数据库获取的缓存数据。

7.6.2.2. Predis

要使用 Predis 扩展去连接 Redis， 请确保环境变量 REDIS_CLIENT 的值为 predis：

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'predis'),

    // Rest of Redis configuration...
],
```

7.6.2.3. PhpRedis

Laravel 默认使用 phpredis 扩展与 Redis 通信。Laravel 用于与 Redis 通信的客户端由 redis.client 配置项决定，这个配置通常为环境变量 REDIS_CLIENT 的值：

```
'redis' => [

    'client' => env('REDIS_CLIENT', 'phpredis'),

    // 重设 Redis 配置...

],
```

7.6.3. 与 Redis 交互

你可以通过调用 Redis facade 上的各种方法来与 Redis 进行交互。 Redis facade 支持动态方法，所以你可以在 facade 上调用各种 Redis 命令 ，这些命令将直接传递给 Redis。

7.6.3.1. 事务

Redis facade 上的 transaction 方法对 Redis 原生的 MULTI 和 EXEC 命令进行了封装。 transaction 方法接受一个闭包作为其唯一参数。这个闭包将接收一个 Redis 连接实例，并可能向这个实例发出想要的任何命令。闭包中发出的所有 Redis 命令都将在单个原子性事务中执行：

```
use Illuminate\Support\Facades\Redis;

Redis::transaction(function ($redis) {

    $redis->incr('user_visits', 1);

    $redis->incr('total_visits', 1);

});
```

7.6.3.2. 管道命令

当你需要执行很多个 Redis 命令时，你可以使用 pipeline 方法一次性提交所有命令，而不需要每条命令都与 Redis 服务器建立一次网络连接。pipeline 方法只接受一个参数：接收一个 Redis 实例的闭包。你可以将所有命令发给这个 Redis 实例，它们将同时发送到 Redis 服务器，以减少到服务器的网络访问。这些命令仍然会按照发出的顺序执行：

```
use Illuminate\Support\Facades\Redis;

Redis::pipeline(function ($pipe) {
    for ($i = 0; $i < 1000; $i++) {
        $pipe->set("key:$i", $i);
    }
});
```

7.6.4. 发布/订阅

Laravel 为 Redis 的 publish 和 subscribe 命令提供了方便的接口。你可以用这些 Redis 命令监听指定「频道」上的消息。你也可以从一个应用程序发消息给另一个应用程序，哪怕它是用其它编程语言开发的，让应用程序和进程之间能够轻松进行通信。

首先，用 subscribe 方法设置一个频道监听器。我们将这个方法调用放到一个 Artisan 命令 中，因为调用 subscribe 方法会启动一个常驻进程：

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;

use Illuminate\Support\Facades\Redis;
```

```
class RedisSubscribe extends Command
{
    /**
     * 控制台命令的名称和签名。
     *
     * @var string
     */
    protected $signature = 'redis:subscribe';

    /**
     * 控制台命令说明。
     *
     * @var string
     */
    protected $description = 'Subscribe to a Redis channel';

    /**
     * 执行控制台命令。
     *
     * @return mixed
     */
    public function handle()
    {
        Redis::subscribe(['test-channel'], function ($message) {
```

```
            echo $message;
        });
    }
}
```

现在我们可以使用 publish 方法将消息发布到频道：

```
use Illuminate\Support\Facades\Redis;


Route::get('/publish', function () {
    // ...


    Redis::publish('test-channel', json_encode([
        'name' => 'Adam Wathan'
    ]));
});
```

第八章 Eloquent ORM

8.1. 快速入门

8.1.1. 简介

Laravel 包含了 Eloquent，这是一个对象关系映射器（ORM），使与数据库的交互变得很愉快。使用 Eloquent 时，每个数据库表都有一个对应的「模型」，用于与该表进行交互。除了从数据库表中检索记录外，Eloquent 模型还允许您从表中插入，更新和删除记录。

8.1.2. 生成模型类

首先，让我们创建一个 Eloquent 模型。模型通常位于 app\Models 目录中，并继承 Illuminate\Database\Eloquent\Model 类。您可以使用 make:model Artisan 命令 来生成新模型类：

```
php artisan make:model Flight
```

如果要在生成模型的时候生成数据库迁移，可以使用 --migration 或 -m 选项。

```
php artisan make:model Flight --migration
```

8.1.3. Eloquent 模型约定

由 make:model 命令生成的模型会被放置在 app/Models 目录下。让我们检查一个基本的模型类并讨论 Eloquent 的一些关键约定：

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    //
}
```

8.1.3.1. 数据表名称

看了上面的例子，你可能已经注意到我们没有告诉 Eloquent 哪个数据库表对应我们的 Flight 模型。按照约定，除非明确指定另一个名称，类名称的下划线格式的复数形态将被用作表名。因此，在这个例子中，Eloquent 将假定 Flight 模

型将记录存储在 flights 表中，而 AirTrafficController 模型将记录存储在 air_traffic_controllers 表中。

如果你的模型对应的数据表不符合这个约定，你可以通过在模型上定义一个 table 属性来手动指定模型的表名：

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;


class Flight extends Model
{
    /**
     * 与模型关联的数据表.
     *
     * @var string
     */
    protected $table = 'my_flights';
}
```

8.1.3.2. 主键

Eloquent 还会假设每个模型对应的数据表都有一个名为 id 的列作为主键。如有必要，你可以在模型上定义一个受保护的 $primaryKey 属性，来指定一个不同的列名称用作模型的主键：

```
<?php

namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;


class Flight extends Model
{
    /**
     * 与数据表关联的主键.
     *
     * @var string
     */
    protected $primaryKey = 'flight_id';
}
```

此外，Eloquent 默认有一个 integer 值的主键，Eloquent 会自动转换这个主键为一个 integer 类型，如果你的主键不是自增或者不是数字类型，你可以在你的模型上定义一个 public 属性的 $incrementing ，并将其设置为 false：

```
<?php
class Flight extends Model
{
    /**
     * 指明模型的 ID 不是自增。
     *
     * @var bool
     */
    public $incrementing = false;
```

```
}
```

复合主键

Eloquent 要求每个模型至少有一个可以作为其主键的唯一标识 ID。它不支持「复合」主键。但是，除了表的唯一标识主键之外，还可以向数据库表添加额外的多列唯一索引。

8.1.3.3. 时间戳

默认情况下，Eloquent 期望 created_at 和 updated_at 列存在于模型对应的数据库表中。 创建或更新模型时，Eloquent 会自动设置这些列的值。如果您不希望 Eloquent 自动管理这些列，您应该在模型上定义一个 $timestamps 属性，其值为 false：

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;


class Flight extends Model
{
    /**
     * 指示模型是否主动维护时间戳。
     *
     * @var bool
     */
    public $timestamps = false;
}
```

8.1.3.4. 默认属性值

默认情况下，被实例化的模型不会包含任何属性值。如果想为模型的某些属性定义默认值，可以在模型上定义一个 $attributes 属性：

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
    * The model's default values for attributes.
    *
    * @var array
    */
    protected $attributes = [
    'delayed' => false,
    ];
}
```

8.1.4. 检索模型

一旦你创建了一个模型和 其关联的数据库表，就可以开始从数据库中检索数据了。可以将每个 Eloquent 模型视为一个强大的 查询构建器，让你能流畅地查询与该模型关联的数据库表。模型中的 all 方法将从模型的关联数据库表中检索所有记录：

```
use App\Models\Flight;
```

```
foreach (Flight::all() as $flight) {
    echo $flight->name;
}
```

8.1.4.1. 集合

正如我们所见，像 all() 和 get() 这样的 Eloquent 方法从数据库中检索出多条记录。但是，这些方法不会返回一个普通的 PHP 数组。相反，会返回一个 Illuminate\Database\Eloquent\Collection 的实例。

Eloquent Collection 类扩展了 Laravel 的 Illuminate\Support\Collection 基类，它提供了 大量的辅助方法 来与数据集合交互。例如，reject 方法可用于根据调用闭包的结果从集合中删除模型：

```
$flights = Flight::where('destination', 'Paris')->get();
$flights = $flights->reject(function ($flight) {
    return $flight->cancelled;
});
```

8.1.4.2. 结果分块

如果你尝试通过 all() 或 get() 方法加载数万条 Eloquent 记录，您的应用程序可能会耗尽内存。为了避免出现这种情况，「chunk」方法可以用来更有效地处理这些大量数据。

chunk 方法将传递 Eloquent 模型的子集，将它们交给闭包进行处理。由于一次只检索当前的 Eloquent 模型块的数据，所以当处理大量模型数据时，chunk 方法将显着减少内存使用：

```
use App\Models\Flight;
```

```
Flight::chunk(200, function ($flights) {
    foreach ($flights as $flight) {
        // TODO
    }
});
```

传递给「chunk」方法的第一个参数是每个分块检索的数据数量。第二个参数传递的闭包将方法将应用到每个分块，以数据库中查询到的分块结果来作为参数。

如果要根据一个字段来过滤「chunk」方法拿到的数据，同时，这个字段的数据在遍历的时候还需要更新的话，那么可以使用「chunkById」方法。在这种场景下如果使用「chunk」方法的话，得到的结果可能和预想中的不一样。在「chunkById」 方法的内部，默认会查询 id 字段大于前一个分块中最后一个模型的 id。

```
Flight::where('departed', true)
    ->chunkById(200, function ($flights) {
        $flights->each->update(['departed' => false]);
    }, $column = 'id');
```

8.1.4.3. 使用惰性集合进行分块

lazy 方法的工作方式类似于 chunk 方法，因为它在后台以块的形式执行查询。 然而，lazy 方法不是将每个块直接传递到回调中，而是返回 Eloquent 模型的扁平化 LazyCollection，它可以让你将结果作为单个流进行交互：

```
use App\Models\Flight;
foreach (Flight::lazy() as $flight) {
    //
```

```
}
```

8.1.4.4. 游标

与 lazy 方法类似，cursor 方法可用于在查询数万条 Eloquent 模型记录时减少内存的使用。

cursor 方法只会执行一次数据库查询；但是，各个 Eloquent 模型在实际迭代之前不会被数据填充。因此，在遍历游标时，在任何给定时间，只有一个 Eloquent 模型保留在内存中。

8.1.5. 高级子查询

8.1.5.1. selects 子查询

Eloquent 还提供高级子查询支持,你可以在单条语句中从相关表中提取信息。例如，假设我们有一个航班目的地表「destinations」和一个到达这些目的地的航班表「flights」。 flights 表包含一个 arrived_at 字段，指示航班何时到达目的地。

使用查询生成器可用的子查询功能 select 和 addSelect 方法，我们可以用单条语句查询全部目的地 destinations 和 抵达各目的地最后一班航班的名称：

```
use App\Models\Destination;
use App\Models\Flight;


return Destination::addSelect(['last_flight' =>
Flight::select('name')
    ->whereColumn('destination_id', 'destinations.id')
    ->orderByDesc('arrived_at')
    ->limit(1)
])->get();
```

8.1.5.2. 子查询排序

此外，查询构建器的 orderBy 也同样支持子查询。继续使用我们的航班为例，根据最后一次航班到达该目的地的时间对所有目的地进行排序。这同样可以在执行单个数据库查询时完成：

```
return Destination::orderByDesc(
    Flight::select('arrived_at')
        ->whereColumn('destination_id', 'destinations.id')
        ->orderByDesc('arrived_at')
        ->limit(1)
)->get();
```

8.1.6. 检索单个模型/聚合结果

除了检索与给定查询匹配的所有记录之外，还可以使用 find、first 或 firstWhere 方法检索单个记录。这些方法不是返回模型集合，而是返回单个模型实例：

```
use App\Models\Flight;

// 通过主键检索模型...
$flight = Flight::find(1);

// 检索与查询约束匹配的第一个模型...
$flight = Flight::where('active', 1)->first();

// 替代检索与查询约束匹配的第一个模型...
```

```
$flight = Flight::firstWhere('active', 1);
```

8.1.6.1. 检索或创建模型

firstOrCreate 方法将尝试使用给定的列 / 值对来查找数据库记录。如果在数据库中找不到该模型，则将插入一条记录，其中包含将第一个数组参数与可选的第二个数组参数合并后产生的属性：

firstOrNew 方法，类似 firstOrCreate，会尝试在数据库中找到与给定属性匹配的记录。如果没有找到，则会返回一个新的模型实例。请注意，由 firstOrNew 返回的模型尚未持久化到数据库中。需要手动调用 save 方法来保存它：

```
use App\Models\Flight;


// 按名称检索航班，如果不存在则创建它...
$flight = Flight::firstOrCreate([
    'name' => 'London to Paris'
]);


// 按名称检索航班或使用名称、延迟和到达时间属性创建它...
$flight = Flight::firstOrCreate(
    ['name' => 'London to Paris'],
    ['delayed' => 1, 'arrival_time' => '11:30']
);


// 按名称检索航班或实例化一个新的航班实例...
$flight = Flight::firstOrNew([
```

```
    'name' => 'London to Paris'
]);


// 按名称检索航班或使用名称、延迟和到达时间属性实例化...
$flight = Flight::firstOrNew(
    ['name' => 'Tokyo to Sydney'],
    ['delayed' => 1, 'arrival_time' => '11:30']
);
```

8.1.6.2. 检索单条唯一的模型

要检索一条表中唯一的记录，可以使用 sole() 方法，相比 first() 和 find() ，它会返回一条表中仅有的记录，如果查询结果有重复，会抛出一个断言异常。

8.1.6.3. 检索聚合

在与 Eloquent 模型交互时，您还可以使用 Laravel [查询构建器] 提供的 count、sum、max 和其他 聚合方法 。这些方法返回一个标量值而不是 Eloquent 模型实例：

```
$count = Flight::where('active', 1)->count();

$max = Flight::where('active', 1)->max('price');
```

8.1.7. 插入/更新模型

8.1.7.1. 插入

当然，在使用 Eloquent 时，我们不仅需要从数据库中检索模型。我们还需要插入新记录。Eloquent 让它变得简单。要将新记录插入数据库，只需要实例化一个新模型实例并在模型上设置属性。然后，在模型实例上调用 save 方法：

```php
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

use App\Models\Flight;

use Illuminate\Http\Request;


class FlightController extends Controller

{

    /**

     * 在数据库中存储一个新航班。

     *

     * @param  \Illuminate\Http\Request  $request

     * @return \Illuminate\Http\Response

     */

    public function store(Request $request)

    {

        // 验证请求...


        $flight = new Flight;


        $flight->name = $request->name;


        $flight->save();

    }
```

```
}
```

在此示例中，我们将传入 HTTP 请求中的「name」字段分配给 App\Models\Flight 模型实例的「name」属性。 当我们调用 save 方法时，一条记录将被插入到数据库中。模型的 created_at 和 updated_at 时间戳会在调用 save 方法时自动设置，无需手动设置。

8.1.7.2. 更新

save 方法也可以用来更新数据库中已经存在的模型。要更新模型，应该检索它并设置你想更新的任何属性。然后调用模型的 save 方法。 同样，updated_at 时间戳将自动更新，因此无需手动设置其值：

```
use App\Models\Flight;


$flight = Flight::find(1);

$flight->name = 'Paris to London';

$flight->save();
```

8.1.7.3. 批量赋值

你可以使用「create」方法使用单个 PHP 语句「保存」一个新模型。插入的模型实例将通过该方法返回：

```
use App\Models\Flight;


$flight = Flight::create([
    'name' => 'London to Paris',
]);
```

但是，在使用 create 方法之前，需要在模型类上指定 fillable 或 guarded 属性。 这些属性是必需的，因为默认情况下，所有 Eloquent 模型都受到保护，免

受批量分配漏洞的影响。

8.1.7.4. 其他创建方法

有时，如果不存在匹配的模型，您可能需要更新现有模型或创建新模型。与 firstOrCreate 方法一样，updateOrCreate 方法会持久化模型，因此无需手动调用 save 方法。

8.1.8. 删除模型

想删除模型，你可以调用模型实例的 delete 方法:

```
use App\Models\Flight;


$flight = Flight::find(1);

$flight->delete();
```

你可以调用 truncate 方法来删除所有模型关联的数据库记录。 truncate 操作还将重置模型关联表上的所有自动递增 ID：

```
Flight::truncate();
```

8.1.8.1. 软删除

除了实际从数据库中删除记录之外，Eloquent 还可以「软删除」。软删除不会真的从数据库中删除记录。相反，它在模型上设置了一个 deleted_at 属性，记录模型被「删除」的日期和时间。要为模型启用软删除，请将 Illuminate\Database\Eloquent\SoftDeletes trait 添加到模型中：

```
<?php

namespace App\Models;


use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\SoftDeletes;


class Flight extends Model
{
    use SoftDeletes;
}
```

8.1.8.2. 查询软删除模型

包括已软删除的模型

如上所述，软删除模型将自动从查询结果中排除。但是，你也可以通过在查询上调用 withTrashed 方法来强制将软删除模型包含在查询结果中：

```
use App\Models\Flight;


$flights = Flight::withTrashed()
                ->where('account_id', 1)
                ->get();
```

8.1.9. 修剪模型

有时你可能希望定期删除不再需要的模型。为此，您可以将 Illuminate\Database\Eloquent\Prunable 或 Illuminate\Database\Eloquent\MassPrunable trait 添加到要定期修剪的模型中。

8.1.10. 复制模型

可以使用 replicate 方法创建现有模型实例的未保存副本。在拥有共享许多相同属性的模型实例时，此方法特别有用：

```
$shipping = App\Address::create([
    'type' => 'shipping',
    'line_1' => '123 Example Street',
    'city' => 'Victorville',
    'state' => 'CA',
    'postcode' => '90001',
]);

$billing = $shipping->replicate()->fill([
    'type' => 'billing'
]);

$billing->save();
```

8.1.11. 查询作用域

8.1.11.1.全局作用域

全局范围可以为模型的所有查询添加约束。Laravel 的 软删除 功能就是利用全局范围仅从数据库中检索「未删除」模型。编写全局范围查询可以为模型的每个查询都添加约束条件。

8.1.11.2.本地作用域

局部作用域允许定义通用的约束集合以便在应用程序中重复使用。例如，你可能经常需要获取所有「流行」的用户。要定义这样一个范围，只需要在对应的 Eloquent 模型方法前添加 scope 前缀。

8.1.12. 模型比较

有时可能需要判断两个模型是否「相同」。is 和 isNot 方法可以用来快速校验两个模型是否拥有相同的主键、表和数据库连接：

```
if ($post->is($anotherPost)) {
    //
}


if ($post->isNot($anotherPost)) {
    //
}
```

8.1.13. 事件

Eloquent 模型触发几个事件，允许你挂接到模型生命周期的如下节点：retrieved、creating、created、updating、updated、saving、saved、deleting、deleted、restoring、restored、replicating。事件允许你每当特定模型保存或更新数据库时执行代码。每个事件通过其构造器接受模型实例。

当从数据库中检索到现有模型时，将调度 retrieved 事件。 当一个新模型第一次被保存时，creating 和 created 事件将被触发。 updating / updated 事件将在修改现有模型并调用 save 方法时触发。saving / saved 事件将在创建或更新模型时触发 - 即使模型的属性没有更改。以「-ing」结尾的事件名称在模型的任何更改被持久化之前被调度，而以「-ed」结尾的事件在对模型的更改被持久化之后被调度。

8.1.13.1.使用闭包

你可以注册在触发各种模型事件时执行的闭包，而不使用自定义事件类。通常，你应该在模型的 booted 方法中注册这些闭包：

```
<?php
```

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 模型的“引导”方法。
     *
     * @return void
     */
    protected static function booted()
    {
        static::created(function ($user) {
            //
        });
    }
}
```

8.1.13.2.观察者

定义观察者

如果在一个模型上监听了多个事件，可以使用观察者来将这些监听器组织到一个单独的类中。观察者类的方法名映射到你希望监听的 Eloquent 事件。这些方法都以模型作为其唯一参数。make:observer Artisan 命令可以快速建立新

的观察者类：

```
php artisan make:observer UserObserver --model=User
```

8.1.13.3.静默事件

也许有时候你会需要暂时将所有由模型触发的事件「静默」处理。使用 withoutEvents 达到目的。withoutEvents 方法接受一个闭包作为唯一参数。任何在闭包中执行的代码都不会被分配模型事件。

8.2. 关联关系

8.2.1. 简介

数据库表通常相互关联。例如，一篇博客文章可能有许多评论，或者一个订单对应一个下单用户。Eloquent 让这些关联的管理和使用变得简单，并支持多种常用的关联类型：

一对一

一对多

多对多

远程一对一

远程一对多

多态一对一

多态一对多

多态多对多

8.2.2. 定义关联关系

Eloquent 关联在 Eloquent 模型类中以方法的形式呈现。如同 Eloquent 模型本身，关联也可以作为强大的 查询语句构造器，使用，提供了强大的链式调用和查询功能。例如，我们可以在 posts 关联的链式调用中附加一个约束条件：

```
$user->posts()->where('active', 1)->get();
```

不过在深入使用关联之前，让我们先学习如何定义每种关联类型。

8.2.2.1. 一对一

一对一是最基本的数据库关系。 例如，一个 User 模型可能与一个 Phone 模型相关联。为了定义这个关联关系，我们要在 User 模型中写一个 phone 方法， 在 phone 方法中调用 hasOne 方法并返回其结果。hasOne 方法被定义在 Illuminate\Database\Eloquent\Model 这个模型基类中：

```
<?php

namespace App;


use Illuminate\Database\Eloquent\Model;


class User extends Model
{
    /**
     * 获取与用户相关的电话记录。
     */
    public function phone()
    {
        return $this->hasOne(phone::class);
    }
}
```

8.2.2.2. 一对多

当要定义一个模型是其他 （一个或者多个）模型的父模型这种关系时，可以使用一对多关联。例如，一篇博客可以有很多条评论。和其他模型关联一样，一对多关联也是在 Eloquent 模型文件中用一个方法来定义的：

```
<?php

namespace App;


use Illuminate\Database\Eloquent\Model;


class Post extends Model
{
    /**
     * 获取博客文章的评论
     */
    public function comments()
    {
        return $this->hasMany('App\Comment');
    }
}
```

8.2.2.3. 一对多(反向) / 属于

目前我们可以访问一篇文章的所有评论，下面我们可以定义一个关联关系，从而让我们可以通过一条评论来获取到它所属的文章。这个关联关系是 hasMany 的反向，可以在子模型中通过 belongsTo 方法来定义这种关联关系：

```
<?php
```

```
namespace App;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    /**
     * 获取此评论所属文章
     */
    public function post()
    {
        return $this->belongsTo('App\Post');
    }
}
```

8.2.3. 多对多关联

多对多关联比 hasOne 和 hasMany 关联稍微复杂些。举个例子，一个用户可以拥有多个角色，同时这些角色也可以分配给其他用户。例如，一个用户可是「作者」和「编辑」；当然，这些角色也可以分配给其他用户。所以，一个用户可以拥有多个角色，一个角色可以分配给多个用户。

8.2.4. 多态关系

多态关联允许目标模型借助单个关联从属于多个模型。例如，你正在构建一个允许用户共享博客文章和视频的应用程序，其中 Comment 模型可能同时从属于 Post 和 Video 模型。

8.2.4.1. 一对一(多态)

表结构

一对一多态关联与简单的一对一关联类似，不过，目标模型能够在一个关联上从属于多个模型。例如，博客 Post 和 User 可能共享一个关联到 Image 模型的关系。使用一对一多态关联允许使用一个唯一图片列表同时用于博客文章和用户账户。

8.2.4.2. 一对多(多态)

一对多多态关联与简单的一对多关联类似，不过，目标模型可以在一个关联中从属于多个模型。假设应用中的用户可以同时「评论」文章和视频。使用多态关联，可以用单个 comments 表同时满足这些情况。

8.2.4.3. 多对多(多态)

表结构

多对多多态关联比 morphOne 和 morphMany 关联略微复杂一些。例如，博客 Post 和 Video 模型能够共享关联到 Tag 模型的多态关系。使用多对多多态关联允许使用一个唯一标签在博客文章和视频间共享。

8.2.5. 关联查询

8.2.5.1. 关联方法 Vs 动态属性

如果你不需要向 Eloquent 关联查询添加额外的约束，您可以像访问属性一样访问关联。 例如，继续使用我们的 User 和 Post 示例模型，我们可以像这样访问用户的所有帖子：

```
use App\Models\User;


$user = User::find(1);
```

```
foreach ($user->posts as $post) {
    //
}
```

动态属性是 「懒加载」 的，只有实际访问到才会加载关联数据。因此，通常用 预加载 来准备模型需要用到的关联数据。预加载能大量减少因加载模型关联执行的 SQL 语句。

8.2.5.2. 查询已存在的关联

检索模型记录时，你可能希望根据关系的存在限制结果。例如，假设要检索至少有一条评论的所有博客文章。 你可以将关系的名称传递给 has 和 orHas 方法：

```
use App\Models\Post;
// 检索至少有一条评论的文章...
$posts = Post::has('comments')->get();
```

8.2.5.3. 查询不存在的关联

检索模型记录时，你可能会根据不存在关系来限制结果。例如，要检索没有有任何评论的所有博客文章。 可以将关系的名称传递给 doesntHave 和 orDoesntHave 方法：

```
use App\Models\Post;
$posts = Post::doesntHave('comments')->get();
```

8.2.5.4. 多态关联查询

要查询多态关联关系的存在，可以使用 whereHasMorph 和 whereDoesntHaveMorph 方法。这些方法接受关联名称作为它们的第一个参数。

8.2.6. 聚合关联模型

8.2.7. 预加载

8.2.8. 插入&更新关联模型

8.2.9. 更新父级时间戳

8.3. Eloquent 集合

8.3.1. 简介

Eloquent 返回的所有结果集都是 Illuminate\Database\Eloquent\Collection 对象的实例，包括通过 get 方法检索或通过访问关联关系获取到的结果。Eloquent 的集合对象继承了 Laravel 的 集合基类，因此它自然也继承了数十种能优雅地处理 Eloquent 模型底层数组的方法。

而且，所有的集合都可以作为迭代器，你可以像遍历简单的 PHP 数组一样来遍历它们：

```
$users = App\User::where('active', 1)->get();


foreach ($users as $user) {
    echo $user->name;
}
```

8.3.2. 可用的方法

所有 Eloquent 的集合都继承了 Laravel Collection 对象；因此，他们也继承

了所有集合基类提供的强大的方法。

另外，Illuminate\Database\Eloquent\Collection 类提供了一套上层的方法来帮你管理你的模型集合。大多数方法返回 Illuminate\Database\Eloquent\Collection 实例； 然而，也会有一些方法返回基于 Illuminate\Support\Collection 类的实例。

8.3.3. 自定义集合

如果你想在与模型交互时使用一个自定义的 Collection 对象，你可以通过在模型中定义 newCollection 方法来实现：

```
<?php

namespace App;


use App\CustomCollection;

use Illuminate\Database\Eloquent\Model;


class User extends Model

{

    /**

     * 创建一个 Eloquent 集合实例.

     *

     * @param  array  $models

     * @return \Illuminate\Database\Eloquent\Collection

     */

    public function newCollection(array $models = [])

    {

        return new CustomCollection($models);
```

```
    }
}
```

8.4. 访问器和修改器

8.4.1. 简介

当你在 Eloquent 模型实例中获取或设置某些属性值的时候，访问器和修改器允许你对 Eloquent 属性值进行格式化。例如，你可能需要使用 Laravel 加密器 来加密保存在数据库中的值,而在使用 Eloquent 模型访问该属性的时候自动进行解密其值。

或者，当通过 Eloquent 模型访问存储在数据库的 JSON 字符串时，你可能希望将其转换为数组。

8.4.2. 访问器&修改器

8.4.2.1. 定义一个访问器

访问器会在访问一个模型的属性时转换 Eloquent 值。要定义访问器，请在模型中创建一个受保护的「驼峰式」方法来表示可访问属性。此方法名称对应到真正的底层模型 属性/数据库字段 的表示。

8.4.2.2. 定义一个修改器

修改器会在设置属性时生效。要定义修改器，可以在定义属性时提供 set 参数。让我们为 first_name 属性定义一个修改器。这个修改器将会在我们修改 first_name 属性的值时自动调用：

```
<?php

namespace App\Models;



use Illuminate\Database\Eloquent\Casts\Attribute;

use Illuminate\Database\Eloquent\Model;
```

```
class User extends Model
{
    /**
     * 与 user 的first name 交互
     *
     * @param  string  $value
     * @return \Illuminate\Database\Eloquent\Casts\Attribute
     */
    protected function firstName(): Attribute
    {
        return new Attribute(
            get: fn ($value) => ucfirst($value),
            set: fn ($value) => strtolower($value),
        );
    }
}
```

8.4.3. 日期修改器

默认情况下，Eloquent 会将 created_at 和 updated_at 字段转换为 Carbon 实例，它继承了 PHP 原生的 DateTime 类并提供了各种有用的方法。你可以通过设置模型的 $dates 属性来添加其他日期属性：

```
<?php
```

```
namespace App;


use Illuminate\Database\Eloquent\Model;


class User extends Model
{
    /**
     * 应该转换为日期格式的属性.
     *
     * @var array
     */
    protected $dates = [
        'seen_at',
    ];
}
```

8.4.4. 属性转换

属性转换提供了类似于访问器和修改器的功能，且无需在模型上定义任何其他方法。模型中的 $casts 属性提供了一个便利的方法来将属性转换为常见的数据类型。

8.4.4.1. 数组 & JSON 转换

当你在数据库存储序列化的 JSON 的数据时， array 类型的转换非常有用。

8.4.5. 自定义类型转换

Laravel 有多种内置的、有用的类型转换； 如果需要自定义的强制转换类型。 可以通过定义一个实现 CastsAttributes 接口的类来实现这一点。

8.4.5.1. 值对象转换

你不仅可以将数据转换成原生的数据类型，还可以将数据转换成对象。两种自定义类型转换的定义方式非常类似。但是将数据转换成对象的自定义转换类中的 set 方法需要返回键值对数组，用于设置原始、可存储的值到对应的模型中。

8.4.5.2. 数组/ JSON 序列化

当使用 toArray 和 toJson 方法将 Eloquent 模型转换为数组或 JSON 时，自定义转换值对象通常会被序列化，只要它们实现 Illuminate\Contracts\Support\Arrayable 和 JsonSerializable 接口。 但是，在使用第三方库提供的值对象时，你可能无法将这些接口添加到对象中。

8.4.5.3. 入站转换

有时候，你可能只需要对写入模型的属性值进行类型转换而不需要对从模型中获取的属性值进行任何处理。一个典型入站类型转换的例子就是 「hashing」。

8.5. API 资源类

8.5.1. 简介

在构建 API 时，你往往需要一个转换层来联结你的 Eloquent 模型和实际返回给用户的 JSON 响应。比如，你可能希望显示部分用户属性而不是全部，或者你可能希望在模型的 JSON 中包括某些关系。Eloquent 的资源类能够让你以更直观简便的方式将模型和模型集合转化成 JSON。

当然，你可以始终使用 Eloquent 模型或集合的 toJson 方法将其转换为 JSON ；但是，Eloquent 的资源提供了对模型及其关系的 JSON 序列化更加精细和更加健壮的控制。

8.5.2. 生成资源

你可以使用 make:resource artisan 命令来生成一个资源类。默认情况下，资源将放在应用程序的 app/Http/Resources 目录下。资源继承自 Illuminate\Http\Resources\Json\JsonResource 类：

```
php artisan make:resource User
```

8.5.3. 核心概念

在深入了解如何定制化编写你的资源之前，让我们首先从高层次上了解 Laravel 中如何使用资源。一个资源类表示一个单一模型需要被转换成 JSON 格式。

8.5.3.1. 资源集合

如果你要返回一个资源集合或一个分页响应，你应该在路由或控制器中创建资源实例时使用你的资源类提供的 collection 方法：

```
use App\Http\Resources\UserResource;
use App\Models\User;

Route::get('/users', function () {
    return UserResource::collection(User::all());
});
```

8.5.4. 编写资源

从本质上说，资源的作用很简单，它只需将一个给定的模型转换为一个数组。因此，每个资源都包含一个 toArray 方法，这个方法会将模型的属性转换为一个 API 友好的数组，然后将该数组通过路由或控制器返回给用户。

8.5.4.1. 数据包裹

默认情况下，当资源响应被转换成 JSON 时，顶层资源将会被包裹在 data 键中。因此一个典型的资源集合响应如下所示：

```
{
    "data": [
        {
            "id": 1,
            "name": "Eladio Schroeder Sr.",
            "email": "therese28@example.com",
        },
        {
            "id": 2,
            "name": "Liliana Mayert",
            "email": "evandervort@example.com",
        }
    ]
}
```

8.5.4.2. 分页

你可以将 Laravel 分页实例传递给资源的 collection 方法或自定义资源集合：

```
use App\Http\Resources\UserCollection;
use App\User;

Route::get('/users', function () {
```

```
    return new UserCollection(User::paginate());
});
```

8.5.4.3. 条件属性

有些时候，你可能希望在给定条件满足时添加属性到资源响应里。例如，你可能希望如果当前用户是「管理员」时添加某个值到资源响应中。在这种情况下 Laravel 提供了一些辅助方法来帮助你解决问题。when 方法可以被用来有条件地向资源响应添加属性：

```
/**
 * 将资源转换成数组
 *
 * @param  \Illuminate\Http\Request  $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        'secret' => $this->when(Auth::user()->isAdmin(), 'secret-
value'),
        'created_at' => $this->created_at,
        'updated_at' => $this->updated_at,
```

```
    ];
}
```

8.5.4.4. 条件关联

除了有条件地加载属性之外，你还可以根据模型关联是否已加载来有条件地在你的资源响应中包含关联。这允许你在控制器中决定加载哪些模型关联，这样你的资源可以在模型关联被加载后才添加它们。最终，这样做可以使你的资源轻松避免「N+1」查询问题。

8.5.4.5. 添加元数据

一些 JSON API 标准需要你在资源和资源集合响应中添加元数据。这通常包括资源或相关资源的 links ，或一些关于资源本身的元数据。如果你需要返回有关资源的其他元数据，只需要将它们包含在 toArray 方法中即可。

8.5.5. 资源响应

就像你知道的那样， 资源可以直接在路由和控制器中被返回：

```
use App\Http\Resources\UserResource;
use App\Models\User;

Route::get('/user/{id}', function ($id) {
    return new UserResource(User::findOrFail($id));
});
```

8.6. 序列化

8.6.1. 简介

在使用 Laravel 构建 API 时，经常需要把模型和关联转化为数组或 JSON。针对这些操作，Eloquent 提供了一些便捷方法，以及对序列化中的属性控制。

8.6.2. 序列化模型&集合

8.6.2.1. 序列化为数组

要转化模型及其加载的 关联 为数组，可以使用 toArray 方法。这是一个递归的方法，因此所有的属性和关联（包括关联的关联）都将转化成数组：

```
use App\Models\User;


$user = User::with('roles')->first();

return $user->toArray();
```

8.6.2.2. 序列化为 JSON

您可以使用 toJson 方法将模型转化成 JSON。和 toArray 一样，toJson 方法也是递归的，因此所有属性和关联都会转化成 JSON, 您还可以指定由 PHP 支持的任何 JSON 编码选项：

```
use App\Models\User;


$user = User::find(1);

return $user->toJson();

return $user->toJson(JSON_PRETTY_PRINT);
```

8.6.3. 隐藏 JSON 属性

有时要将模型数组或 JSON 中的某些属性进行隐藏，比如密码。则可以在模型中添加 $hidden 属性。模型序列化后， $hidden 数组中列出的属性将不会被显示：

```
<?php

namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;


class User extends Model
{
    /**
     * 数组中的属性会被隐藏。
     *
     * @var array
     */
    protected $hidden = ['password'];
}
```

8.6.4. 追加 JSON 值

有时，需要在模型转换为数组或 JSON 时添加一些数据库中不存在字段的对应属性。要实现这个功能，首先要定义一个 访问器：

```
<?php
namespace App\Models;


use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;


class User extends Model
{
    /**
```

```
     * Determine if the user is an administrator.
     *
     * @return \Illuminate\Database\Eloquent\Casts\Attribute
     */
    protected function isAdmin(): Attribute
    {
        return new Attribute(
            get: fn () => 'yes',
        );
    }
}
```

8.6.5. 日期序列化

自定义默认日期格式

您可以通过重写 serializeDate 方法来自定义默认序列化格式。此方法不会影响日期在数据库中存储的格式：

```
/**
 * 为数组/ JSON 序列化准备一个日期。
 *
 * @param  \DateTimeInterface  $date
 * @return string
 */
protected function serializeDate(DateTimeInterface $date)
{
```

```
    return $date->format('Y-m-d');
}
```

第九章 测试系列

9.1. 快速入门

9.1.1. 简介

内置的 PHPUnit 对测试提供支持是开箱即用的。Laravel 默认就支持用 PHPUnit 来做测试，并为你的应用程序配置好了 phpunit.xml 文件。框架还提供了一些便利的辅助函数，让你可以更直观的测试你的应用程序。

默认情况，你的应用 tests 目录中包含两个子目录：Feature 和 Unit。单元测试是针对你的代码中非常少，而且相对独立的一部分代码来进行的测试。实际上，大部分单元测试都是针对单个方法进行的。功能测试是针对大面积代码进行的测试，包括多个对象之间的交互，甚至是对 JSON 端点的完整 HTTP 请求。

Feature 和 Unit 目录中都提供一个 ExampleTest.php 测试示例文件。安装一个新的 Laravel 应用程序之后，在命令行下运行 vendor/bin/phpunit 命令，即可运行测试。

9.1.2. 环境

运行测试时，由于 phpunit.xml 文件中定义了 环境变量 ，Laravel 会自动配置环境变量为 testing。Laravel 还会在测试时自动将会话和缓存配置到 array 驱动程序，这意味着在测试时不会持久化会话或缓存数据。

你可以根据需要自由定义其他测试环境配置值。 testing 环境变量可以在应用程序的 phpunit.xml 文件中配置，但请确保在运行测试之前使用 config:clear Artisan 命令清除配置缓存！

.env.testing 环境配置文件

此外，你可以在项目的根目录中创建一个 .env.testing 文件。 当运行 PHPUnit 测试或使用 --env=testing 选项执行 Artisan 命令时，将不会使用 .env 文件，而是使用此文件。

CreatesApplication Trait

Laravel 包含一个 CreatesApplication Trait，该 Trait 应用于应用程序的基类 TestCase 。这个 trait 包含一个 createApplication 方法，它在运行测试之前引导 Laravel 应用程序。 重要的是，应将此 trait 保留在其原始位置，因为某些功能（例如 Laravel 的并行测试功能）依赖于它。

9.1.3. 创建测试

要创建新的测试用例，请使用 Artisan 命令： make:test 。 默认情况下，测试将放置在 tests/Feature 目录中：

```
php artisan make:test UserTest
```

9.1.4. 运行测试

9.1.4.1. 并行运行测试

默认情况下，Laravel 和 PHPUnit 在执行测试时，是在单进程中按照先后顺序执行的。除此之外，通过多个进程同时运行测试，则可以大大减少运行测试所需的时间。

9.1.4.2. 报告测试覆盖率

在运行测试时，你可能需要确定测试用例是否真的测到了某些程序代码，以及在运行测试时究竟使用了多少应用程序代码。

9.2. HTTP 测试

9.2.1. 简介

Laravel 提供了一个非常流畅的 API，用于向应用程序发出 HTTP 请求并检

查响应。例如，看看下面定义的特性测试：

```
<?php

namespace Tests\Feature;


use Illuminate\Foundation\Testing\RefreshDatabase;

use Illuminate\Foundation\Testing\WithoutMiddleware;

use Tests\TestCase;


class ExampleTest extends TestCase

{

    /**

     * 基本的测试示例.

     *

     * @return void

     */

    public function test_a_basic_request()

    {

        $response = $this->get('/');


        $response->assertStatus(200);

    }

};
```

get 方法向应用程序发出 Get 请求，而 assertStatus 方法则断言返回的响应应该具有给定的 HTTP 状态代码。除了这个简单的断言之外，Laravel 还包

含各种用于检查响应头、内容、JSON 结构等的断言。

9.2.2. 创建请求

9.2.2.1. 自定义请求头

您可以使用此 withHeaders 方法自定义请求的标头，然后再将其发送到应用程序。这使您可以将任何想要的自定义标头添加到请求中：

```
<?php


class ExampleTest extends TestCase
{
    /**
     * 一个基本的功能测试示例。
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->withHeaders([
            'X-Header' => 'Value',
        ])->json('POST', '/user', ['name' => 'Sally']);


        $response
            ->assertStatus(201)
            ->assertJson([
                'created' => true,
```

```
            ]);
    }
}
```

9.2.2.2. 测试响应

测试请求通过应用程序后，dump 和 dumpHeaders 方法将会在控制台打印响应体和响应头，可以用来分析和调试响应 ：

```
<?php

namespace Tests\Feature;

use Tests\TestCase;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;

class ExampleTest extends TestCase
{
    /**
     * A basic test example.
     *
     * @return void
     */
    public function testBasicTest()
    {
```

```
        $response = $this->get('/');

        $response->dumpHeaders();

        $response->dump();
    }
}
```

9.2.3. Session/认证

Laravel 提供了几个可在 HTTP 测试时使用 Session 的辅助函数。首先，你需要传递一个数组给 withSession 方法来设置 Session 数据。这让你在应用程序的测试请求发送之前，先给数据加载 Session 变得简单：

```
<?php

class ExampleTest extends TestCase
{
    public function testApplication()
    {
        $response = $this->withSession(['foo' => 'bar'])
                         ->get('/');
    }
}
```

9.2.4. 测试 JSON APIs

Laravel 也提供了几个辅助函数来测试 JSON APIs 和他们的响应。 例如，json, get， post， put， patch，和 delete 可以被用于发送各种 HTTP 动作。 你也可以轻松地将数据和请求头传递到这些方法中。

```
<?php


class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->json('POST', '/user', ['name' =>
'Sally']);


        $response
            ->assertStatus(201)
            ->assertJson([
                'created' => true,
            ]);
    }
```

```
}
```

9.2.4.1. 验证 JSON 完全匹配

如果你想验证给定的数组 完全 匹配应用返回的 JSON 结果，你应该使用 assertExactJson 方法：

```
<?php


class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->json('POST', '/user', ['name' =>
'Sally']);


        $response
            ->assertStatus(201)
            ->assertExactJson([
                'created' => true,
            ]);
```

```
    }
}
```

9.2.4.2. 验证 JSON 路径

如果你想验证 JSON 响应是否包含指定路径上的某些给定数据，则应使用 assertJsonPath 方法：

```
<?php


class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->json('POST', '/user', ['name' =>
'Sally']);


        $response
            ->assertStatus(201)
            ->assertJsonPath('team.owner.name', 'foo')
    }
```

```
}
```

9.2.5. 测试文件上传

Illuminate\Http\UploadedFile 提供了一个 fake 方法用于生成虚拟的文件或者图像以供测试之用。 它可以和 Storage facade 的 fake 方法相结合， 大幅度简化了文件上传测试。举个例子，你可以结合这两者的功能非常方便地进行头像上传表单测试：

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testAvatarUpload()
    {
        Storage::fake('avatars');

        $file = UploadedFile::fake()->image('avatar.jpg');
```

```
        $response = $this->json('POST', '/avatar', [
            'avatar' => $file,
        ]);

        // Assert the file was stored...
        Storage::disk('avatars')->assertExists($file->hashName());

        // Assert a file does not exist...
        Storage::disk('avatars')->assertMissing('missing.jpg');
    }
}
```

9.3. 命令行测试

9.3.1. 简介

除了简化 HTTP 测试之外， Laravel 为测试用户输入的控制台应用提供了简单的 API。

9.3.2. 期望输入/输出

Laravel 使用 expectsQuestion 方法为控制台命令「模拟」用户输入。此外，还可以使用 assertExitCode 和 expectsOutput 方法指定控制台命令退出码和期望输出文本。举例如下：

```
Artisan::command('question', function () {
    $name = $this->ask('What is your name?');
```

```
    $language = $this->choice('Which language do you program in?', [
        'PHP',
        'Ruby',
        'Python',
    ]);

    $this->line('Your name is '.$name.' and you program in
'.$language.'.');
});
```

9.4. 数据库测试

9.4.1. 简介

Laravel 提供了各种有用的工具，可以更轻松地测试数据库驱动的应用程序。首先，你可以使用 assertDatabaseHas 辅助函数，来断言数据库中是否存在与指定条件互相匹配的数据。 例如，如果我们想要验证 users 表中是否存在 email 值为 sally@example.com 的数据，你可以执行以下操作：

```
public function testDatabase()
{
    // 调用应该程序...

    $this->assertDatabaseHas('users', [
        'email' => 'sally@example.com',
    ]);
}
```

9.4.2. 生成模型工厂

使用 make:factory Artisan command 命令可以创建一个模型工厂：

```
php artisan make:factory PostFactory
```

9.4.3. 每次测试后重置数据库

在每次测试后重置数据库是很有用的，这样前一次测试的数据不会干扰后续测试。 RefreshDatabase trait 会采用最优化的方法来迁移测试数据库，这取决于你使用的是内存数据库还是传统数据库。

9.4.4. 编写模型工厂

进行测试时，运行测试之前常常需要插入一些数据到数据库中。当你创建测试数据时，除了手动设置每个字段的值，Laravel 还可以使用 Eloquent 模型 的工厂来设置每个属性的默认值。在开始之前， 你可以先看一下应用程 database/factories/UserFactory.php 的文件。 开箱即用，这个文件包含一个模型工厂定义：

```
use Faker\Generator as Faker;
use Illuminate\Support\Str;

$factory->define(App\User::class, function (Faker $faker) {
    return [
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'email_verified_at' => now(),
```

```
        'password' =>
'$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro91lC/.og/at2.uheWG/igi', //
password
        'remember_token' => Str::random(10),
    ];
});
```

9.4.4.1. 工厂扩展

如果扩展了模型，则可能还希望扩展其工厂，以便在测试和数据填充期间利用子模型的工厂属性。 为此，你可以调用工厂制造商的 raw 方法来从任何给定工厂获得原始属性数组。

```
$factory->define(App\Admin::class, function (Faker\Generator $faker)
{
    return factory(App\User::class)->raw([
        // ...
    ]);
});
```

9.4.4.2. 工厂状态

工厂状态可以让你任意组合你的模型工厂，仅需要做出适当差异化的修改，就可以达到让模型拥有多种不同的状态。例如， 你的 User 模型中可以修改某个默认属性值来达到标识一种 delinquent 状态。你可以使用 state 方法来进行这种状态转换。对于简单的工厂状态，你可以传入要修改的属性数组：

```
$factory->state(App\User::class, 'delinquent', [
    'account_status' => 'delinquent',
```

```
]);
```

9.4.4.3. 工厂回调

工厂回调是使用 afterMaking 和 afterCreating 方法注册的，并且允许你在创建模型之后执行其他任务。例如，可以使用回调将附加模型与创建的模型相关联：

```
$factory->afterMaking(App\User::class, function ($user, $faker) {
    // ...
});

$factory->afterCreating(App\User::class, function ($user, $faker) {
    $user->accounts()->save(factory(App\Account::class)->make());
});
```

9.5. 模拟器

9.5.1. 简介

在 Laravel 应用程序测试中，你可能希望「模拟」应用程序的某些功能的行为，从而避免该部分在测试中真正执行。例如：在控制器执行过程中会触发事件，从而避免该事件在测试控制器时真正执行。这允许你在仅测试控制器 HTTP 响应的情况时，而不必担心触发事件。当然，你也可以在单独的测试中测试该事件逻辑。

Laravel 针对事件、任务和 Facades 的模拟，提供了开箱即用的辅助函数。这些函数基于 Mocker 封装而成，使用非常方便，无需手动调用复杂的 Mockery 函数。当然你也可以使用 Mockery 或者使用 PHPUnit 创建自己的模拟器。

9.5.2. 模拟对象

当模拟一个对象将通过 Laravel 的服务容器注入到应用中时，你将需要将模

拟实例作为 instance 绑定到容器中。这将告诉容器使用对象的模拟实例，而不是构造对象的真身：

```
use App\Service;
use Mockery;

$this->instance(Service::class, Mockery::mock(Service::class,
function ($mock) {
    $mock->shouldReceive('process')->once();
}));
```

9.5.3. 任务模拟

作为模拟的替代方式，你可以使用 Bus Facade 的 fake 方法来防止任务被真正分发执行。

9.5.4. 事件模拟

作为 mock 的替代方法，你可以使用 Event Facade 的 fake 方法来模拟事件监听，测试的时候并不会真正触发事件监听器。然后你就可以测试断言事件运行了，甚至可以检查他们接收的数据。

9.5.5. 邮件模拟

你可以是用 Mail Facade 的 fake 方法来模拟邮件发送，测试时不会真的发送邮件，然后你可以断言 mailables 发送给了用户，甚至可以检查他们收到的内容。使用 fakes 时，断言一般放在测试代码的后面。

9.5.6. 通知模拟

可以使用 Notification Facade 的 fake 方法来模拟通知的发送，测试时并不会真的发出通知。然后你可以断言 notifications 发送给了用户，甚至可以检查他们收到的内容。使用 fakes 时，断言一般放在测试代码后面。

第十章 附录：官方扩展包

10.1. 交易工具包（Stripe）

10.1.1. 简介

Laravel Cashier Stripe 为 Stripe 的订阅计费服务提供了一个富有表现力、流畅的接口。它处理了几乎所有你害怕编写的订阅计费样板代码。除了基本的订阅管理，Cashier 还可以处理优惠券、交换订阅、订阅 「数量」、取消宽限期，甚至生成发票 PDF。

10.1.2. 安装

首先，使用 Composer 为 Stripe 安装 Cashier 扩展包：

```
composer require laravel/cashier
```

10.1.2.1.数据库迁移

Cashier 的服务提供器注册了自己的数据库迁移目录，因此请记住在安装此包后迁移数据库。

10.2. 交易工具包（Paddle）

10.2.1. 简介

Laravel Cashier Paddle 为 Paddle's 订阅计费服务提供了一个富有表现力、流畅的界面。它几乎能够处理所有你所恐惧的各种订阅计费逻辑和代码。除了基本的订阅管理，Cashier 还可以处理：优惠券、交换订阅、订阅「数量」、取消宽限期等。

在使用 Cashier 时，推荐你回顾一下 Paddle 的用户手册 and API 文档。

10.2.2. 安装

首先，使用 Composer 包管理器安装 Paddle 的 Cashier 包：

```
composer require laravel/cashier-paddle
```

10.3. Envoy 部署工具

10.3.1. 简介

Laravel Envoy 是一套在远程服务器上执行日常任务的工具。使用了 Blade 风格语法，你可以轻松地配置部署任务、Artisan 命令的执行等。目前，Envoy 仅支持 Mac 和 Linux 操作系统。但是可以使用 WSL2 以实现在 Windows 上使用。

10.3.2. 安装

首先，运行 Composer 将 Envoy 安装到你的项目中：

```
composer require laravel/envoy --dev
```

10.4. Horizon 队列管理工具

10.4.1. 简介

Laravel Horizon 为你的 Laravel Redis queues. 提供了一个美观的仪表盘和代码驱动的配置；它可以方便的监控队列系统的关键指标：任务吞吐量、运行时间、作业失败情况。

在使用 Horizon 时，所有队列的 worker 配置都存储在一个简单的配置文件中。通过在受版本控制的文件中定义应用程序的 worker 配置，你可以在部署应用程序时轻松扩展或修改应用程序的队列 worker。

10.4.2. 安装

你可以使用 Composer 将 Horizon 安装到你的 Laravel 项目里：

```
composer require laravel/horizon
```

第十一章 补充说明

以上从官方文档翻译而来。

以下待删除。